Universidad Siglo 21



Seminario de Práctica de Informática TP4 - 2026

CLICKOBS

Sistema Inteligente de Gestión de Costos y Licenciamiento de Plataformas de Observabilidad basado en IA

Entrega: Trabajo Práctico 4
(versión final integradora: fases de Inicio, Elaboración y Construcción)

Alumno: JORGE CORREA ALVAREZ

Legajo: VINF02919

Fecha: noviembre 2026 | Repositorio y video: https://github.com/jorgejcorrea/clickobs

Docente/Tutor:
PABLO VIRGOLINI - JUAN OJEDA

Contenido

Resumen Ejecutivo..... 5
Datos clave del proyecto..... 6
Principales beneficios esperados..... 6
Alcance del presente documento..... 7
1. Introducción..... 8
1.1 Antecedentes..... 8
1.2 Descripción del área problemática..... 8
1.3 Formulación de la problemática..... 9
1.4 Marco FinOps de la solución..... 9
2. Justificación..... 10
Valor para la organización..... 10
Beneficios para los usuarios..... 10
Valor tecnológico..... 11
3. Definiciones del proyecto..... 11
3.1 Fines de la organización..... 11
3.2 Objetivo general del proyecto..... 11
3.3 Objetivos específicos del proyecto..... 11
3.4 Distinción entre fines, objetivo del proyecto y objetivo del sistema..... 12
4. Definiciones del sistema y elicitación..... 12
4.1 Objetivo general del sistema..... 12
4.2 Límites del proyecto y del sistema..... 12
4.3 Alcance..... 12
4.4 No contempla..... 13
4.5 Elicitación de requerimientos..... 13
5. Conocimiento del negocio..... 15
5.1 Descripción de la organización..... 15
5.2 Estructura organizacional..... 15
5.3 Modelo de negocio..... 15
5.4 Procesos de negocio principales..... 15
5.5 Cómo dará respuesta el sistema..... 16
6. Diagrama de dominio (UML)..... 16
6.1 Descripción de las clases del dominio..... 17
6.2 Relaciones principales..... 18
7. Diagramas de procesos de negocio..... 18
7.1 Proceso: ingesta y normalización de datos de consumo..... 18
7.2 Proceso: generación de recomendación IA y alerta de licenciamiento..... 19
8. Propuesta de solución..... 19
8.1 Metodología de desarrollo: Proceso Unificado de Desarrollo (PUD)..... 19
8.2 Propuesta funcional..... 20
8.3 Propuesta técnica..... 21
9. Requerimientos..... 24
9.1 Requerimientos funcionales (RF)..... 24
9.2 Requerimientos no funcionales (RNF)..... 25
10. Casos de uso..... 26
10.1 Identificación de actores..... 26
10.2 Diagrama de casos de uso..... 26
10.3 Especificación de casos de uso..... 27
11. Plan de proyecto (Gantt)..... 28

12. Criterios de diseño..... 30
12.1 Justificación antes que solución técnica.....30
12.2 Normalización orientada a las consultas más frecuentes.....30
12.3 IA como capa de interpretación, no de cálculo.....30
12.4 Simplicidad del prototipo frente a la arquitectura objetivo.....30
12.5 Trazabilidad entre interfaz y casos de uso.....30
13. Fundamento del uso de IA para el análisis de FinOps.....31
13.1 Aclaración sobre la arquitectura de IA.....31
13.2 Viabilidad práctica.....31
13.3 Salvaguardas de diseño.....31
14. Etapa de análisis: diagrama de secuencia.....33
15. Etapa de diseño: diagrama de clases.....34
16. Etapa de implementación.....35
16.1 Arquitectura del prototipo.....35
16.2 Requerimientos de implementación.....35
16.3 Prototipo de interfaz gráfica y su vinculación con los casos de uso.....36
17. Etapa de pruebas.....38
17.1 Plan de pruebas.....38
17.2 Casos de prueba (caja blanca y caja negra).....38
17.3 Tratamiento de defectos.....39
17.4 Evaluación de prueba.....39
18. Modelo de datos relacional.....39
18.1 Definición de base de datos para el sistema.....39
18.2 Decisión de diseño: unificación de Consumo y Costo.....40
18.3 Diagrama entidad-relación (DER).....40
18.4 Descripción de las tablas.....40
18.5 Verificación de reglas de negocio, normalización e integridad.....41
19. Scripts SQL.....42
19.1 Creación de tablas.....42
19.2 Inserción de datos de prueba.....44
19.3 Consulta de datos (CU-02: comparar costos entre herramientas).....46
19.4 Borrado de datos y verificación.....46
19.5 Presentación de las consultas SQL.....47
20. Requerimientos de comunicación del sistema.....47
20.1 Interacciones entre componentes.....48
20.2 Protocolos y estándares de comunicación.....48
20.3 Diagrama de despliegue y comunicaciones.....49
20.4 Entorno de red, infraestructura física y control de enlace de datos.....49
21. Repositorio del proyecto.....50
22. Desarrollo del prototipo en Java/JavaFX.....51
22.1 Alcance: producto mínimo viable.....51
22.2 Organización del código y decisión de capas.....51
22.3 Los cuatro pilares de la programación orientada a objetos.....53
22.4 Estructuras de datos y algoritmos.....53
22.5 Manejo de excepciones.....54
22.6 Tipos de datos y estructuras de control.....55
22.7 Interfaz de usuario y coherencia con los mockups.....55
22.8 Pruebas.....60
22.9 Decisiones y herramientas.....61
22.10 Trazabilidad con las entregas anteriores.....61
22.11 Repositorio y ejecución.....61
22.12 Fragmentos de código de referencia.....61

22.13 Conclusión....63
23. Versión final integradora del prototipo (TP4)....64
23.1 Alcance y entrega incremental....64
23.2 Corrección de las observaciones de las entregas anteriores....64
23.3 Análisis del modelo de negocios que justifica el proyecto....65
23.4 Aplicación del Proceso Unificado de Desarrollo....66
23.5 Selección del patrón de diseño y justificación....66
23.5.1 Patrón Modelo-Vista-Controlador: responsabilidades y verificación....69
23.6 Persistencia y consulta de datos en MySQL....70
23.7 Excepciones en la interacción con MySQL....72
23.8 Clases abstractas e interfaces....74
23.9 Arreglos y ArrayList de forma complementaria....75
23.10 Archivos para guardar y recuperar información....75
23.11 Pruebas de la iteración....76
23.12 Decisiones, límites y trabajo futuro....77
23.13 Repositorio y ejecución....77
23.14 Arquitectura del prototipo y arquitectura objetivo....78
23.15 Conclusión....80
Referencias....81
Anexo A. Paso a paso: publicar el proyecto en GitHub....81

Resumen Ejecutivo

CLICKOBS S.A. (en adelante, «la compañía») es una fintech regional (LATAM) de aproximadamente 280 empleados que enfrenta un problema operativo concreto: la gestión de sus herramientas de observabilidad (Dynatrace, Datadog, Grafana/Prometheus y Elastic) se realiza de forma descentralizada, sin gobierno de costos unificado entre las cuatro tribus de ingeniería que las adoptaron de manera independiente. El sistema que se desarrolla en este proyecto se denomina CLICKOBS (en adelante, «el sistema»).

Esta situación genera consecuencias concretas: solapamiento de cobertura entre herramientas (~35% de los servicios monitoreados por dos o más plataformas), imposibilidad de proyectar el gasto anual en observabilidad, ausencia de un responsable único del costo total y decisiones técnicas tomadas sin análisis de costo-beneficio.

Enfoque FinOps y necesidad de la solución

La observabilidad dejó de ser solo una decisión técnica y se volvió también una decisión financiera. Las herramientas de la compañía se licencian de maneras distintas (por host, por volumen de datos, por métrica o por almacenamiento) y combinan servicios contratados en la nube con plataformas operadas en infraestructura propia. Gestionar ese equilibrio es el campo de **FinOps**, definido por la FinOps Foundation (2026) como un marco operativo y una práctica cultural que maximiza el valor de negocio de la tecnología mediante decisiones oportunas basadas en datos y responsabilidad financiera compartida entre ingeniería, finanzas y negocio. Su alcance ya incluye categorías como SaaS, centros de datos propios e inteligencia artificial (Markanastasakis, 2026), que son los tres frentes que reúne el sistema.

La compañía no contaba con ninguna herramienta que integrara el consumo, el costo y el licenciamiento de su observabilidad, ni entre la nube y las plataformas propias ni entre las cuatro tribus. Cada equipo veía el costo de su herramienta, pero nadie veía cuánto costaba observar en total, cuánto se duplicaba ni cómo evolucionaría el gasto. Por eso contar con esta solución dejó de ser una mejora deseable y pasó a ser una **necesidad imperiosa** para la compañía.

El sistema busca sostener una ecuación de equilibrio entre el **valor operativo** (cobertura, retención de datos y capacidad de resolver incidentes) y el **costo financiero** (licencias, consumo, infraestructura y operación), con un objetivo claro: **prever** el gasto a partir del histórico y **reaccionar** a tiempo ante desvíos. Hoy el prototipo reacciona con umbrales mensuales configurables, detección de cobertura duplicada y recomendaciones con ahorro estimado; la detección de anomalías de consumo y la proyección anual están previstas para la Construcción final, junto con la extensión de la misma lógica al consumo global de servicios de IA.

Frente a este escenario, el proyecto propone CLICKOBS: una aplicación de escritorio en Java (cliente JavaFX), respaldada por un backend y una base de datos MySQL on-premise, que integra los datos de consumo y costo de las cuatro herramientas y utiliza inteligencia artificial (vía OpenRouter) para generar recomendaciones de racionalización y alertas de licenciamiento.

Este documento integra, de forma incremental, la Fase de Inicio (TP1), la Fase de Elaboración (TP2: etapas de análisis, diseño, implementación y pruebas, modelo de datos, scripts SQL y comunicaciones del sistema) y la Fase de Construcción (TP3: desarrollo del prototipo en Java/JavaFX).

Datos clave del proyecto

Cliente	CLICKOBS S.A. - Fintech regional con 4 tribus de ingeniería
Objetivo del proyecto	Reducir en al menos un 20% el gasto anual en licenciamiento de observabilidad y disponer de una visión consolidada y proyectable de ese gasto
Duración	6 meses (24 semanas)
Inicio estimado	Lunes 5 de octubre de 2026
Fin estimado	Viernes 19 de marzo de 2027
Metodología	Proceso Unificado de Desarrollo (PUD) - iterativo e incremental
Tecnología propuesta	Java 21: cliente de escritorio JavaFX, backend Spring Boot (API REST), base de datos MySQL 8 e IA vía OpenRouter
Arquitectura	Cliente-servidor en tres capas desplegada on-premise, con una capa de ingesta hacia las 4 herramientas de observabilidad y una capa de IA
Prototipo	Aplicación JavaFX conectada por JDBC a MySQL (subconjunto de la arquitectura objetivo)
Módulos principales	7 módulos: Usuarios y Seguridad, Herramientas y Conectores, Ingesta y Normalización, Comparación de Costos, Recomendaciones IA, Alertas de Licenciamiento, Administración y Reportes
Marco regulatorio	Ley N.° 25.326 de Protección de Datos Personales; políticas internas de seguridad de la información; restricción de residencia de datos para el Core Bancario
Usuarios alcanzados	CTO, Head de Infraestructura, SRE Leads y DevOps de las 4 tribus, Finanzas / Procurement
Repositorio	https://github.com/jorgejcorrea/clickobs

Principales beneficios esperados

- Visibilidad y control centralizado del costo de observabilidad de toda la organización, con información oportuna para decisiones de presupuesto.
- Reducción de costos operativos mediante la detección y eliminación de solapamientos de cobertura entre herramientas.
- Proyección presupuestaria confiable para el ejercicio fiscal, al unificar cuatro modelos de facturación distintos en un esquema común.
- Recomendaciones automatizadas de racionalización generadas por IA, con trazabilidad histórica de cada sugerencia.
- Alertas proactivas de licenciamiento que evitan sorpresas en el cierre de facturación mensual.
- Escalabilidad para incorporar nuevas herramientas de observabilidad sin modificar el núcleo del sistema.
- Alineación con las iniciativas corporativas de FinOps y sostenibilidad IT ya en curso en CLICKOBS S.A.

Alcance del presente documento

El documento se organiza en cuatro partes, que corresponden a las fases del Proceso Unificado de Desarrollo (PUD) cubiertas hasta el momento:

- **Parte I, Fase de Inicio:** secciones 1 a 11. Contexto y problemática, justificación, objetivos, límites, elicitación, conocimiento del negocio, modelo de dominio, procesos de negocio, propuesta de solución, requerimientos, casos de uso y plan de proyecto. Incluye los ajustes surgidos de la revisión de esta fase.
- **Parte II, Fase de Elaboración:** secciones 12 a 21. Criterios de diseño, fundamento del uso de IA, etapas de análisis, diseño, implementación y pruebas, modelo de datos con su diagrama entidad-relación, scripts SQL con evidencia de ejecución, requerimientos de comunicación y repositorio del proyecto.
- **Parte III, Fase de Construcción (TP3):** sección 22. Desarrollo del prototipo en Java/JavaFX: alcance, organización del código, los cuatro pilares de la programación orientada a objetos, estructuras de datos y algoritmos, manejo de excepciones, coherencia de la interfaz con los mockups, pruebas, decisiones y repositorio.
- **Parte IV, Versión final integradora (TP4):** sección 23. Patrón de diseño DAO, persistencia y consulta en MySQL, excepciones de base de datos, clases abstractas e interfaces, arreglos con ArrayList, archivos, pruebas y repositorio.

La fase de Transición del PUD se abordará en una etapa posterior del proyecto.

PARTE I — FASE DE INICIO (TP1)

1. Introducción

El presente documento describe la solución informática a desarrollar para CLICKOBS S.A., en el marco del proyecto de gestión de costos y licenciamiento de plataformas de observabilidad. La iniciativa surge ante la necesidad de centralizar la gobernanza de las cuatro herramientas de observabilidad utilizadas por las tribus de ingeniería, hoy administradas de manera independiente y sin criterios de costo unificados.

1.1 Antecedentes

CLICKOBS S.A. es una fintech regional (LATAM) que ofrece billetera digital, préstamos personales online y un marketplace de inversión, con una arquitectura de microservicios desplegada en una nube híbrida (AWS para los productos digitales y on-premise para el core bancario, por requisitos regulatorios).

A lo largo de tres años de crecimiento, cada tribu de ingeniería adoptó su propia herramienta de observabilidad de forma autónoma, sin que existiera una política corporativa que unificara criterios:

Tribu	Herramienta	Modalidad	Motivo de adopción original
Infraestructura / Plataforma	Dynatrace	SaaS	Auto-discovery e IA causal (Davis)
Pagos	Datadog	SaaS	Integración rápida con AWS
Core Bancario	Grafana + Prometheus	On-premise (self-hosted)	Requisito de dato on-premise por compliance
Mobile / Apps	Elastic (ELK)	On-premise (self-hosted)	Ya se usaba Elastic para búsqueda en el producto

Actualmente, el área de Arquitectura estima que aproximadamente el 35% de los servicios están monitoreados por más de una herramienta de forma simultánea, sin que exista un criterio documentado que lo justifique. El Head de Infraestructura encomienda liderar el equipo de desarrollo que construirá CLICKOBS: un sistema que centralice la visibilidad de costo y licenciamiento de las cuatro herramientas e incorpore inteligencia artificial para generar recomendaciones de racionalización.

1.2 Descripción del área problemática

El relevamiento inicial sobre las cuatro tribus permitió identificar los siguientes problemas concretos en la operación actual:

- **Información descentralizada y desactualizada:** cada tribu gestiona el consumo y el costo de su propia herramienta sin una vista consolidada a nivel organización.
- **Ausencia de criterios estandarizados:** cada tribu decide qué monitorear y con qué herramienta según preferencia técnica, sin análisis de costo-beneficio previo.
- **Solapamiento de cobertura:** aproximadamente el 35% de los servicios están cubiertos por dos o más herramientas de forma simultánea, lo que genera pago duplicado.

- **Imposibilidad de proyectar presupuesto:** los cuatro proveedores facturan bajo modelos distintos (DDU, host/métrica custom, almacenamiento, infraestructura propia), lo que impide a Finanzas consolidar una proyección anual confiable.
- **Riesgo de sobrecosto por licenciamiento:** al no existir alertas centralizadas, los picos de consumo (por ejemplo, en Dynatrace por DDU) se detectan recién en la factura, sin posibilidad de acción preventiva.
- **Dependencia de reportes manuales:** el CTO y Finanzas dependen de reportes ad-hoc que cada tech lead arma de forma independiente para conocer el gasto de su tribu.
- **Restricción regulatoria:** el Core Bancario no puede enviar datos fuera de la región, lo que condiciona qué información de Grafana/Prometheus puede centralizarse o enviarse a servicios de IA externos sin anonimizar previamente.

1.3 Formulación de la problemática

¿Cómo se puede centralizar la visibilidad del costo y licenciamiento de las herramientas de observabilidad de CLICKOBS S.A. para detectar solapamientos de cobertura, proyectar el gasto anual con precisión y generar recomendaciones de optimización mediante inteligencia artificial, respetando al mismo tiempo las restricciones regulatorias de residencia de datos del Core Bancario?

1.4 Marco FinOps de la solución

FinOps es un marco operativo y una práctica cultural que maximiza el valor de negocio de la tecnología, permite decidir a tiempo con datos y crea responsabilidad financiera compartida entre ingeniería, finanzas y negocio (FinOps Foundation, 2026). Su marco organiza el trabajo en tres fases: informar, optimizar y operar. Además, delimita «alcances» por categoría de tecnología, entre ellas SaaS, centros de datos propios e inteligencia artificial (Markanastasakis, 2026).

La compañía tiene los tres alcances a la vez. Dynatrace y Datadog se consumen como servicio; Grafana con Prometheus y Elastic corren en infraestructura propia; y la capa de IA, vía OpenRouter, agrega un consumo variable por uso. La compañía no disponía de una solución que mirara ese conjunto en una sola vista, y la ausencia de gobierno unificado se traducía en solapamiento de cobertura, imposibilidad de proyectar el gasto y decisiones sin análisis de costo-beneficio. Por eso la solución dejó de ser una mejora deseable y pasó a ser una necesidad imperiosa.

La ecuación que el sistema busca balancear tiene dos lados: el **valor operativo** de la observabilidad (cobertura, retención y capacidad de detectar y resolver incidentes) y su **costo financiero** (licencias, consumo, infraestructura y operación). Una herramienta de más duplica cobertura y gasto; una de menos deja puntos ciegos. El objetivo es la **previsión**, anticipar el gasto, y la **reacción**, actuar a tiempo ante desvíos. El cuadro siguiente relaciona las fases de FinOps con lo que hace el sistema y su estado.

Fase	Qué hace el sistema	Estado
Informar	Dashboard con costo total y costo por mes, consumo por herramienta y por tribu, comparación de costos y exportación a CSV	Prototipo (TP4)
Optimizar	Detección de cobertura duplicada y recomendaciones de racionalización con ahorro potencial (IA simulada)	Prototipo (TP4); modelo real por OpenRouter en la versión final
Operar	Umbrales mensuales configurables, cola de alertas, administración de conectores y bitácora	Prototipo (TP4)

Fase	Qué hace el sistema	Estado
Prever	Proyección anual del gasto a partir del histórico	Construcción final
Reaccionar	Detección de anomalías de consumo, comparando cada período con su comportamiento histórico	Construcción final
IA global	Registro y control del consumo de servicios de IA junto al de observabilidad	Construcción final

La detección de anomalías complementa a los umbrales actuales: el umbral avisa cuando se cruza un valor fijo, mientras que la detección de anomalías avisa cuando el consumo se aparta de su patrón aunque no llegue al umbral.

2. Justificación

En el marco de la problemática descrita, surge la necesidad de contar con un sistema con capacidad para operar y administrar la información de costo y consumo de las herramientas de observabilidad de CLICKOBS S.A. Las razones que justifican el proyecto se fundamentan en los siguientes ejes:

Valor para la organización

- **Visibilidad y control centralizado:** la solución permitirá al CTO, al Head de Infraestructura y a Finanzas conocer, en cualquier momento, el consumo y el costo consolidado de las cuatro herramientas, mejorando la calidad de la toma de decisiones presupuestarias.
- **Reducción de costos operativos:** la detección de solapamientos de cobertura permitirá racionalizar el uso de herramientas y eliminar pagos duplicados por la misma cobertura de monitoreo.
- **Proyección presupuestaria confiable:** al normalizar los cuatro modelos de facturación en un esquema común, Finanzas podrá proyectar con precisión el gasto en observabilidad del próximo ejercicio fiscal.
- **Escalabilidad:** la arquitectura basada en conectores permitirá incorporar nuevas herramientas de observabilidad sin modificar el núcleo del sistema, acompañando el crecimiento de la organización.
- **Gobierno y trazabilidad:** cada recomendación generada por IA y cada alerta de licenciamiento quedarán registradas, facilitando las auditorías internas del gasto en tecnología.

Beneficios para los usuarios

- **SRE Lead y DevOps engineers:** visibilidad del consumo de su propia tribu y de posibles solapamientos con otras tribus, sin depender de reportes manuales.
- **Head de Infraestructura:** información consolidada para decidir la racionalización de herramientas con respaldo de datos y de IA.
- **CTO:** recomendaciones de optimización en lenguaje natural, con estimación de ahorro, para fundamentar decisiones estratégicas ante el Directorio.
- **Finanzas / Procurement:** alertas proactivas de licenciamiento y reportes exportables antes del cierre de facturación, en lugar de reaccionar a facturas ya emitidas.

Valor tecnológico

Al tratarse de una aplicación de escritorio conectada a un backend centralizado, con conectores desacoplados por herramienta, se evita la dependencia de un único proveedor y se garantiza que toda la organización trabaje sobre la misma versión de la información de costos. La adopción de tecnologías estándar (Java y MySQL) asegura una solución mantenible y escalable, mientras que el uso de OpenRouter como capa de enrutamiento de IA permite seleccionar el modelo más económico según la complejidad de cada tarea de análisis, aplicando a la propia capa de IA el mismo principio de FinOps que el sistema promueve para las herramientas de observabilidad.

3. Definiciones del proyecto

Se distinguen a continuación tres planos que no deben mezclarse: los fines de la organización, el objetivo general del proyecto y el objetivo del sistema. Los dos primeros se definen en esta sección; el objetivo del sistema, en la sección 4.1.

3.1 Fines de la organización

CLICKOBS S.A. busca sostener el crecimiento rentable de sus servicios financieros digitales (billetera, préstamos personales y marketplace de inversión), cuidando la eficiencia de sus costos operativos, la disponibilidad de su plataforma y el cumplimiento de las regulaciones que le aplican. La gestión ordenada del gasto tecnológico es una condición para ese fin, pero no es el fin en sí mismo.

3.2 Objetivo general del proyecto

Reducir en al menos un 20% el gasto anual en licenciamiento de observabilidad de CLICKOBS S.A. y disponer de una visión consolidada y proyectable de ese gasto, dentro de los 6 meses posteriores al inicio del proyecto.

El objetivo se formula en términos de qué se busca lograr (un resultado para el negocio) y no de cómo se va a construir el sistema: el cómo corresponde al objetivo del sistema (sección 4.1) y a las etapas de diseño e implementación de la Parte II.

3.3 Objetivos específicos del proyecto

Se enuncian como logros concretos derivados del objetivo general, no como etapas de desarrollo del software:

- Contar con una vista única y actualizada del consumo y del costo de las cuatro herramientas, por tribu, por servicio y por período.
- Identificar y cuantificar los servicios monitoreados por más de una herramienta (hoy, cerca del 35%) y reducir ese solapamiento.
- Disponer de una proyección anual confiable del gasto en observabilidad, que hoy no existe.
- Recibir alertas de licenciamiento antes del cierre de facturación, en lugar de detectar el sobrecosto en la factura.
- Contar con recomendaciones de racionalización, con ahorro estimado y trazabilidad histórica, para que el CTO decida con datos.
- Garantizar que ningún dato del Core Bancario llegue a servicios de IA externos sin haber sido anonimizado.

- Reemplazar los reportes de costo armados manualmente por cada tribu por reportes consolidados y exportables.

El relevamiento, el diseño, el desarrollo y las pruebas forman parte del plan de proyecto (sección 11) y no se enuncian como objetivos.

3.4 Distinción entre fines, objetivo del proyecto y objetivo del sistema

Plano	Qué responde	En CLICKOBS
Fines de la organización	¿Para qué existe la organización y qué busca sostener?	Crecer con eficiencia de costos, disponibilidad y cumplimiento regulatorio.
Objetivo general del proyecto	¿Qué resultado de negocio se quiere lograr con este proyecto?	Reducir al menos un 20% el gasto anual en licenciamiento de observabilidad y poder proyectarlo.
Objetivo del sistema	¿Qué función cumple el software una vez construido?	Consolidar el consumo y el costo de las 4 herramientas y generar recomendaciones y alertas.

4. Definiciones del sistema y elicitación

4.1 Objetivo general del sistema

Proveer a CLICKOBS S.A. una aplicación de escritorio que permita consultar y comparar el consumo y el costo consolidado de las cuatro herramientas de observabilidad, y que genere recomendaciones de optimización mediante inteligencia artificial y alertas de licenciamiento, con distintos niveles de acceso según el rol del usuario.

4.2 Límites del proyecto y del sistema

Límite	Desde	Hasta
Proyecto	Relevamiento inicial con el CTO y las tribus (05/10/2026).	Puesta en producción, capacitación y soporte inicial (19/03/2027).
Sistema	Ingesta periódica de los datos de consumo y costo mediante los conectores de cada herramienta.	Presentación al usuario de la comparación, la recomendación o la alerta correspondiente, y su registro en el historial.

Quedan fuera de los límites del sistema los datos de telemetría en sí (métricas, logs y trazas de las aplicaciones monitoreadas): el sistema opera solamente sobre sus metadatos de consumo y costo.

4.3 Alcance

El sistema CLICKOBS abarca la administración, actualización y consulta del consumo y costo consolidado de las cuatro herramientas de observabilidad. Comprende los siguientes módulos:

- **Gestión de usuarios y roles:** alta, baja y modificación de usuarios con perfiles diferenciados (CTO, Head de Infraestructura, SRE Lead, DevOps, Finanzas), incluyendo autenticación y control de acceso.
- **Gestión de herramientas y conectores:** alta de herramientas de observabilidad, configuración segura de credenciales de API y monitoreo del estado de sincronización.
- **Ingesta y normalización:** sincronización periódica de datos de consumo desde cada herramienta y normalización a un esquema común de costo/uso.
- **Comparación de costos:** comparación de costos entre herramientas por servicio y período, y detección de solapamientos de cobertura.
- **Recomendaciones con IA:** generación de recomendaciones de optimización mediante un modelo de IA seleccionado vía OpenRouter.
- **Alertas de licenciamiento:** configuración de umbrales de gasto y generación de notificaciones automáticas.
- **Administración y reportes:** exportación de reportes gerenciales y configuración de parámetros del sistema.

4.4 No contempla

- El proceso de negociación o contratación de licencias con los proveedores (Dynatrace, Datadog, Elastic).
- El análisis del contenido de la telemetría monitoreada (métricas, logs, trazas de las aplicaciones de negocio).
- La migración técnica de servicios de una herramienta de observabilidad a otra (queda a criterio de cada tribu, en base a la recomendación).
- La gestión contable o financiera general de CLICKOBS S.A., más allá del gasto específico en observabilidad.
- El desarrollo de interfaces web o móviles: el acceso al sistema es exclusivamente mediante el cliente de escritorio.

4.5 Elicitación de requerimientos

La elicitación de requerimientos es la actividad mediante la cual se identifican, descubren y comprenden las necesidades, expectativas y restricciones de los distintos actores involucrados en el sistema a construir. Para este proyecto se aplicaron las siguientes técnicas, seleccionadas en función del perfil de los interlocutores y del tipo de información a relevar.

Técnica 1: Entrevistas individuales

Se realizaron entrevistas semiestructuradas con los principales actores del negocio: CTO (1 entrevista de 90 minutos), Head de Infraestructura (2 entrevistas de 60 minutos), Finanzas (2 entrevistas de 60 minutos) y Tech Leads de las 4 tribus (1 entrevista de 45 minutos cada uno).

Resultados obtenidos: identificación de los objetivos estratégicos del CTO (racionalización de costos, cumplimiento de FinOps, escalabilidad); relevamiento del nivel de autonomía que cada tribu espera conservar; detección de resistencia al cambio en tribus con alta satisfacción con su herramienta actual.

Técnica 2: Observación directa

Se realizaron sesiones de observación con los DevOps engineers de las 4 tribus durante su operación habitual, relevando cómo consultan hoy sus dashboards y qué reportes de costo arman manualmente.

Resultados obtenidos: confirmación de la heterogeneidad de criterios entre tribus; identificación de reportes manuales redundantes que el sistema debería reemplazar; comprensión del flujo de trabajo diario de cada perfil técnico.

Técnica 3: Análisis de documentación existente

Se relevaron las facturas de los últimos 12 meses de los cuatro proveedores, los contratos de licenciamiento vigentes y la política interna de seguridad de la información de CLICKOBS S.A.

Resultados obtenidos: identificación de los campos mínimos necesarios para normalizar el costo de cada herramienta; detección de fechas de renovación de contratos; listado de restricciones de compliance a cubrir por el sistema.

Técnica 4: Cuestionarios al personal técnico

Se distribuyó un cuestionario estructurado a los SRE y DevOps de las 4 tribus (32 personas en total), orientado a relevar dificultades actuales y expectativas respecto de la nueva solución.

Resultados obtenidos: el 78% manifestó dificultades para comparar el costo de su herramienta con el de otras tribus; el 65% desconocía si su servicio estaba monitoreado por más de una herramienta; el 88% expresó interés en contar con alertas automáticas de gasto.

Técnica 5: Workshop de priorización

Se realizó un taller con la participación conjunta de los 4 Tech Leads, el Head de Infraestructura y un referente de Finanzas, con el objetivo de validar los procesos relevados y priorizar funcionalidades.

Resultados obtenidos: consenso sobre los criterios de solapamiento aceptable frente a redundancia a eliminar; priorización de las alertas de licenciamiento como funcionalidad de primera versión; acuerdo sobre el nivel de autonomía que cada tribu conserva sobre su propia herramienta.

Síntesis de hallazgos

El proceso de elicitación permitió consolidar los siguientes hallazgos transversales, que dieron sustento a la definición de los requerimientos funcionales y no funcionales del sistema:

- Necesidad ineludible de centralizar la información de costo y consumo en una única base de datos.
- Existencia de roles claramente diferenciados que requieren niveles de acceso distintos.
- Demanda concreta de alertas automáticas de gasto y de licenciamiento.
- Necesidad de información consolidada y de reportes para el CTO y Finanzas.
- Restricción regulatoria de residencia de datos que condiciona el envío de información del Core Bancario a servicios de IA externos.
- Importancia crítica de preservar la autonomía operativa de cada tribu sobre su propia herramienta.

5. Conocimiento del negocio

5.1 Descripción de la organización

CLICKOBS S.A. es una fintech regional constituida para operar en el mercado LATAM, dedicada a la oferta de billetera digital, préstamos personales online y un marketplace de inversión. La organización opera bajo la dirección de un Directorio encabezado por su CEO y su CFO, con una Gerencia de Tecnología (CTO) responsable de las cuatro tribus de ingeniería que componen la plataforma.

5.2 Estructura organizacional

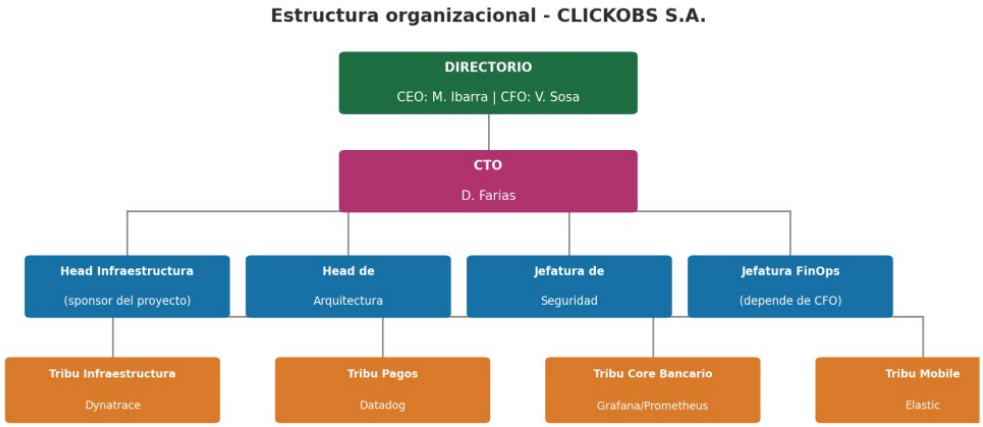


Figura 1. Estructura organizacional de CLICKOBS S.A.

5.3 Modelo de negocio

El modelo de negocio de CLICKOBS S.A. se basa en la prestación de servicios financieros digitales, con tres líneas principales: billetera digital (transacciones y pagos), préstamos personales online (originación y cobranza) y marketplace de inversión (comisiones por operación). La disponibilidad y el rendimiento de estos servicios dependen críticamente de la observabilidad de la plataforma: una degradación no detectada a tiempo implica pérdida de transacciones y daño reputacional.

El costo de observabilidad, sin embargo, no es percibido hoy como parte del costo operativo total de cada línea de negocio, ya que se factura y se gestiona por separado en cada tribu, sin trazabilidad hacia el producto que efectivamente protege.

5.4 Procesos de negocio principales

La organización opera bajo los siguientes procesos clave, de los cuales el sistema impactará directamente sobre el primero:

- **Gestión de costos de observabilidad (alcance del proyecto):** negociación de licencias, consumo por tribu, facturación, comparación y racionalización.
- **Desarrollo y operación de microservicios:** relacionado pero fuera del alcance directo (genera el consumo que el sistema debe medir).
- **Gestión de FinOps de infraestructura cloud:** proceso hermano ya en curso, con el que CLICKOBS deberá integrarse conceptualmente a futuro.
- **Gestión de compliance y seguridad de la información:** define las restricciones de residencia de datos que el sistema debe respetar.

5.5 Cómo dará respuesta el sistema

- **Centralización de la información:** una única base de datos consolidada, accesible desde el cliente de escritorio, con el consumo y el costo de las 4 herramientas.
- **Estandarización de datos:** normalización de los distintos modelos de facturación a un esquema común de costo por servicio y período.
- **Automatización de alertas:** notificaciones automáticas al superar umbrales de gasto configurables por herramienta.
- **Recomendaciones inteligentes:** generación de sugerencias de racionalización mediante IA, con estimación de ahorro.
- **Control de acceso por rol:** cada usuario accede únicamente a las funciones que le correspondan según su perfil.
- **Tablero gerencial:** información consolidada para el CTO y Finanzas, con indicadores clave para decisiones estratégicas.

6. Diagrama de dominio (UML)

El diagrama de dominio representa las clases conceptuales del negocio con sus atributos principales y las relaciones que existen entre ellas. No describe el diseño técnico del sistema, sino el modelo conceptual del problema que se busca resolver.

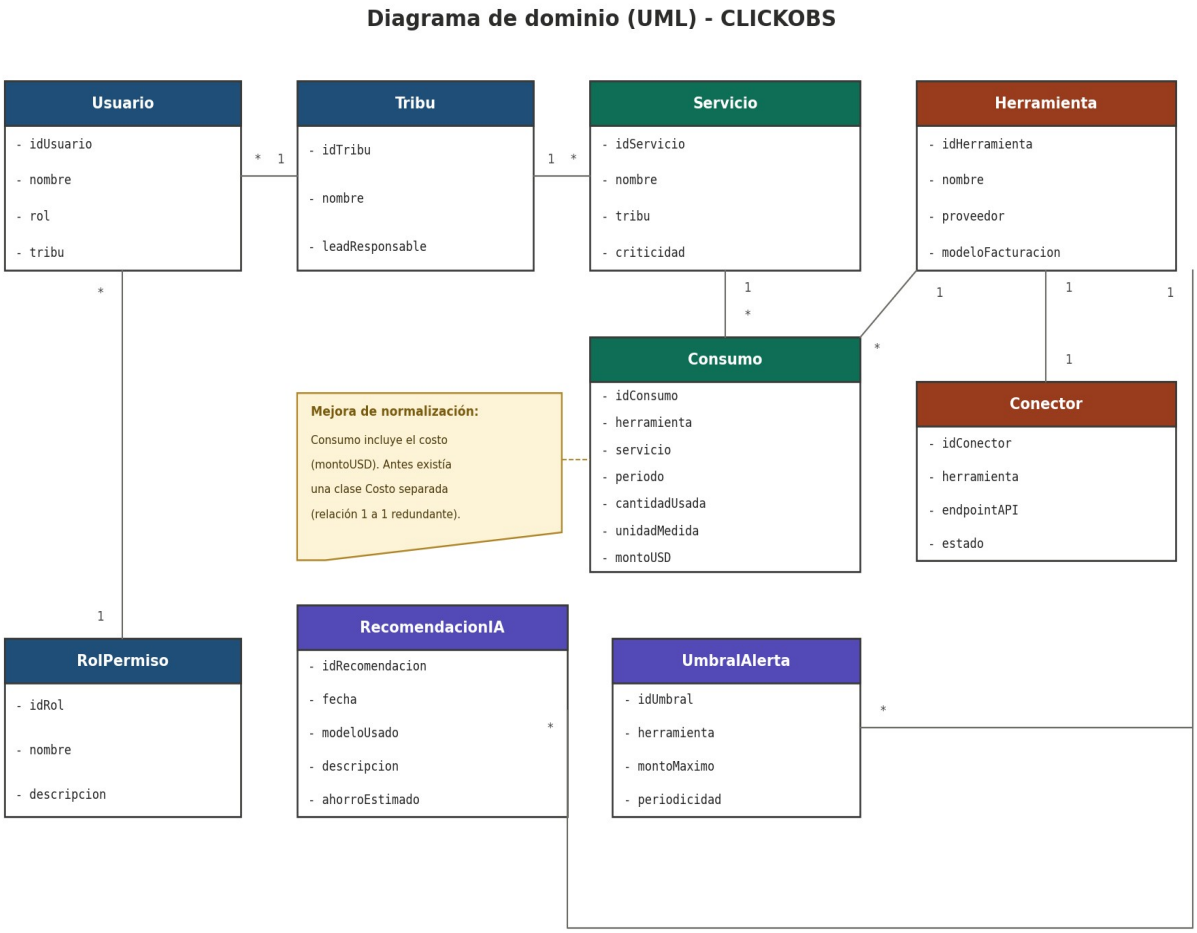


Figura 2. Diagrama de dominio del sistema CLICKOBS

6.1 Descripción de las clases del dominio

- **Usuario:** representa a toda persona que interactúa con el sistema. Atributos: idUsuario, nombre, rol, tribu.
- **RolPermiso:** define los permisos y el nivel de acceso. Atributos: idRol, nombre, descripcion.
- **Tribu:** cada uno de los equipos de ingeniería de CLICKOBS S.A. Atributos: idTribu, nombre, leadResponsable.
- **Servicio:** microservicio o sistema monitoreado. Atributos: idServicio, nombre, tribu, criticidad.
- **Herramienta:** plataforma de observabilidad (Dynatrace, Datadog, Grafana/Prometheus, Elastic). Atributos: idHerramienta, nombre, proveedor, modeloFacturacion.
- **Consumo:** registro periódico de uso de una herramienta sobre un servicio, con su costo valorizado. Atributos: idConsumo, herramienta, servicio, periodo, cantidadUsada, unidadMedida, montoUSD.
- **Conector:** integración técnica con la API de una herramienta. Atributos: idConector, herramienta, endpointAPI, estado.

- **UmbralAlerta:** configuración de un límite de gasto. Atributos: idUmbral, herramienta, montoMaximo, periodicidad.
- **RecomendacionIA:** sugerencia de optimización generada por el modelo de IA. Atributos: idRecomendacion, fecha, modeloUsado, descripcion, ahorroEstimado.

6.2 Relaciones principales

- Un Usuario tiene un RolPermiso y pertenece a una Tribu.
- Una Tribu es responsable de múltiples Servicios.
- Un Servicio genera múltiples registros de Consumo, cada uno asociado a una Herramienta.
- Cada Consumo incluye su costo valorizado (montoUSD); no existe una clase Costo separada.
- Cada Herramienta tiene un Conector que gestiona la integración con su API.
- Cada Herramienta puede tener múltiples UmbralAlerta configurados.
- Las RecomendacionIA se generan a partir del análisis agregado de Consumo y UmbralAlerta.

Mejora de normalización: en la primera versión de este diagrama, Costo era una clase separada de Consumo, con una relación 1 a 1. Tras revisar la cardinalidad y la posible redundancia (que condiciona el rendimiento de las consultas comparativas), ambas se unificaron en una sola clase. El fundamento completo se presenta en la sección 18.2.

7. Diagramas de procesos de negocio

Los siguientes diagramas modelan los procesos clave que el sistema deberá soportar. Se modelaron en notación BPMN simplificada para facilitar la lectura por parte de los usuarios funcionales y técnicos.

7.1 Proceso: ingesta y normalización de datos de consumo

Describe el flujo desde que se dispara la sincronización periódica hasta que los datos de consumo quedan normalizados y persistidos.

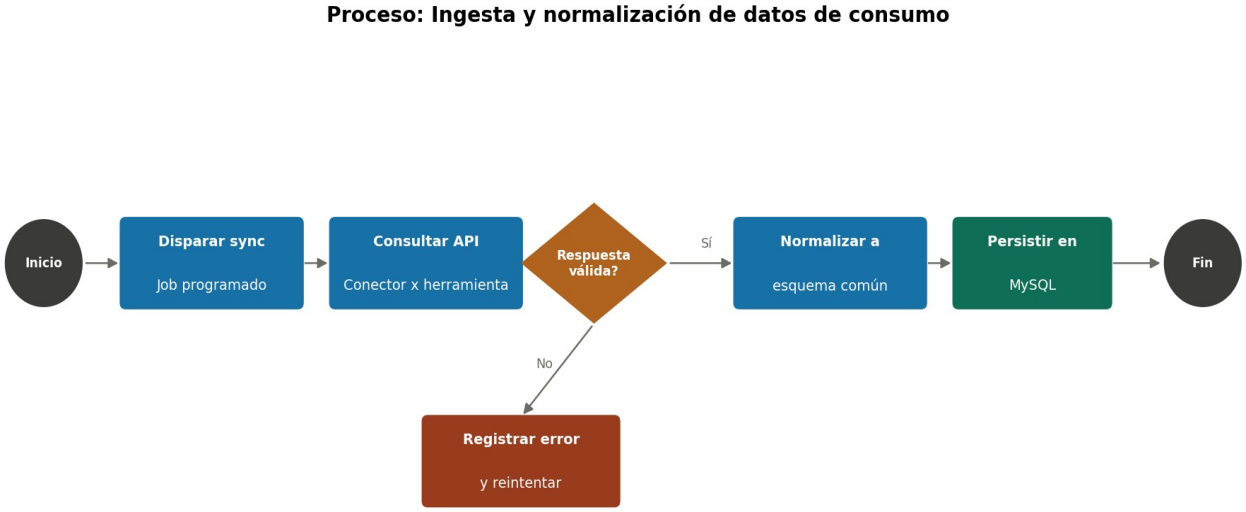


Figura 3. Proceso de ingesta y normalización de datos de consumo

Pasos principales:

1. Un job programado dispara la sincronización periódica.
2. El conector correspondiente consulta la API de la herramienta.
3. Si la respuesta no es válida: se registra el error y se reintenta (excepción).
4. Si la respuesta es válida: los datos se normalizan al esquema común.
5. Los datos normalizados se persisten en la base de datos MySQL.

7.2 Proceso: generación de recomendación IA y alerta de licenciamiento

Modela el flujo cuando un usuario solicita un análisis y el sistema evalúa si corresponde disparar una alerta de licenciamiento.

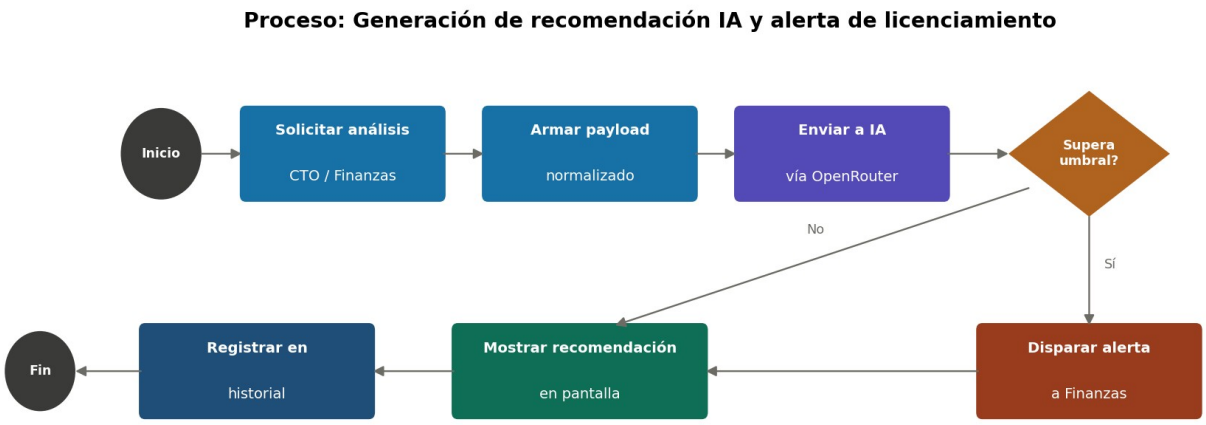


Figura 4. Proceso de generación de recomendación IA y alerta de licenciamiento

Pasos principales:

1. El CTO o Finanzas solicita un análisis.
2. El sistema arma un payload normalizado con los datos consolidados.
3. El payload se envía al modelo de IA seleccionado vía OpenRouter.
4. Si el consumo analizado supera el umbral configurado: se dispara una alerta a Finanzas.
5. Si no supera el umbral: se muestra la recomendación en pantalla.
6. En ambos casos, la recomendación o la alerta queda registrada en el historial.

8. Propuesta de solución

8.1 Metodología de desarrollo: Proceso Unificado de Desarrollo (PUD)

Para el desarrollo de CLICKOBS se adopta el Proceso Unificado de Desarrollo (PUD), una metodología iterativa e incremental ampliamente utilizada en proyectos de ingeniería de software. El PUD se caracteriza por ser dirigido por casos de uso, centrado en la arquitectura e iterativo e incremental, lo que resulta adecuado para un proyecto con incertidumbre inicial respecto de cuánto se puede racionalizar sin afectar cobertura crítica (Kendall y Kendall, 2011).

El proyecto se estructura en las cuatro fases clásicas del PUD:

- **Fase de Inicio:** definición del alcance, identificación de actores y casos de uso principales, y elaboración del plan general del proyecto. Cubierta por la Parte I de este documento (secciones 1 a 11).
- **Fase de Elaboración:** profundización del análisis y del diseño, modelo de datos, arquitectura y prototipo. Cubierta por la Parte II (secciones 12 a 21).
- **Fase de Construcción:** desarrollo iterativo de los conectores, el módulo de IA, el backend y el cliente JavaFX. Cada iteración entrega un incremento funcional probado.
- **Fase de Transición:** piloto con la tribu de Infraestructura, ajustes según feedback y despliegue general a las 4 tribus.

8.2 Propuesta funcional

La propuesta funcional describe qué hará el sistema para responder a las necesidades del negocio relevadas. La solución contempla los siguientes módulos funcionales:

Módulo 1: Gestión de usuarios y seguridad

- Alta, baja y modificación de usuarios.
- Asignación de roles diferenciados por perfil (CTO, Head de Infraestructura, SRE Lead, DevOps, Finanzas).
- Autenticación mediante usuario y contraseña, con recuperación por correo electrónico.
- Registro de auditoría (log) de operaciones por usuario.

Módulo 2: Gestión de herramientas y conectores

- Alta de herramientas de observabilidad y configuración segura de sus credenciales de API.
- Monitoreo del estado de sincronización de cada conector.
- Administración de umbrales de reintento ante fallos de conexión.

Módulo 3: Ingesta y normalización de consumo

- Sincronización periódica programada por herramienta.
- Normalización de los datos de consumo a un esquema común de costo/uso.
- Registro de errores de sincronización para su seguimiento.

Módulo 4: Comparación de costos y solapamientos

- Comparación de costos entre herramientas por servicio y período.
- Detección automática de servicios cubiertos por más de una herramienta.
- Histórico de consumo y costo para análisis de tendencia.

Módulo 5: Recomendaciones con IA

- Generación de recomendaciones de optimización mediante un modelo de IA seleccionado vía OpenRouter.
- Selección del modelo según la complejidad de la tarea (económico para clasificación, más robusto para la narrativa gerencial).
- Historial de recomendaciones generadas, con trazabilidad.

Módulo 6: Alertas de licenciamiento

- Configuración de umbrales de gasto por herramienta y periodicidad.

- Generación de notificaciones automáticas al superar el umbral.

Módulo 7: Administración y reportes

- Exportación de reportes gerenciales a PDF y Excel.
- Configuración de parámetros del sistema y de catálogos auxiliares (tribus, herramientas).
- Respaldo y restauración de datos.

8.3 Propuesta técnica

La propuesta técnica describe cómo se construirá el sistema, definiendo la arquitectura, las tecnologías y los componentes principales. La Figura 5 sintetiza la arquitectura objetivo en una infografía funcional; a continuación se describe cada capa.

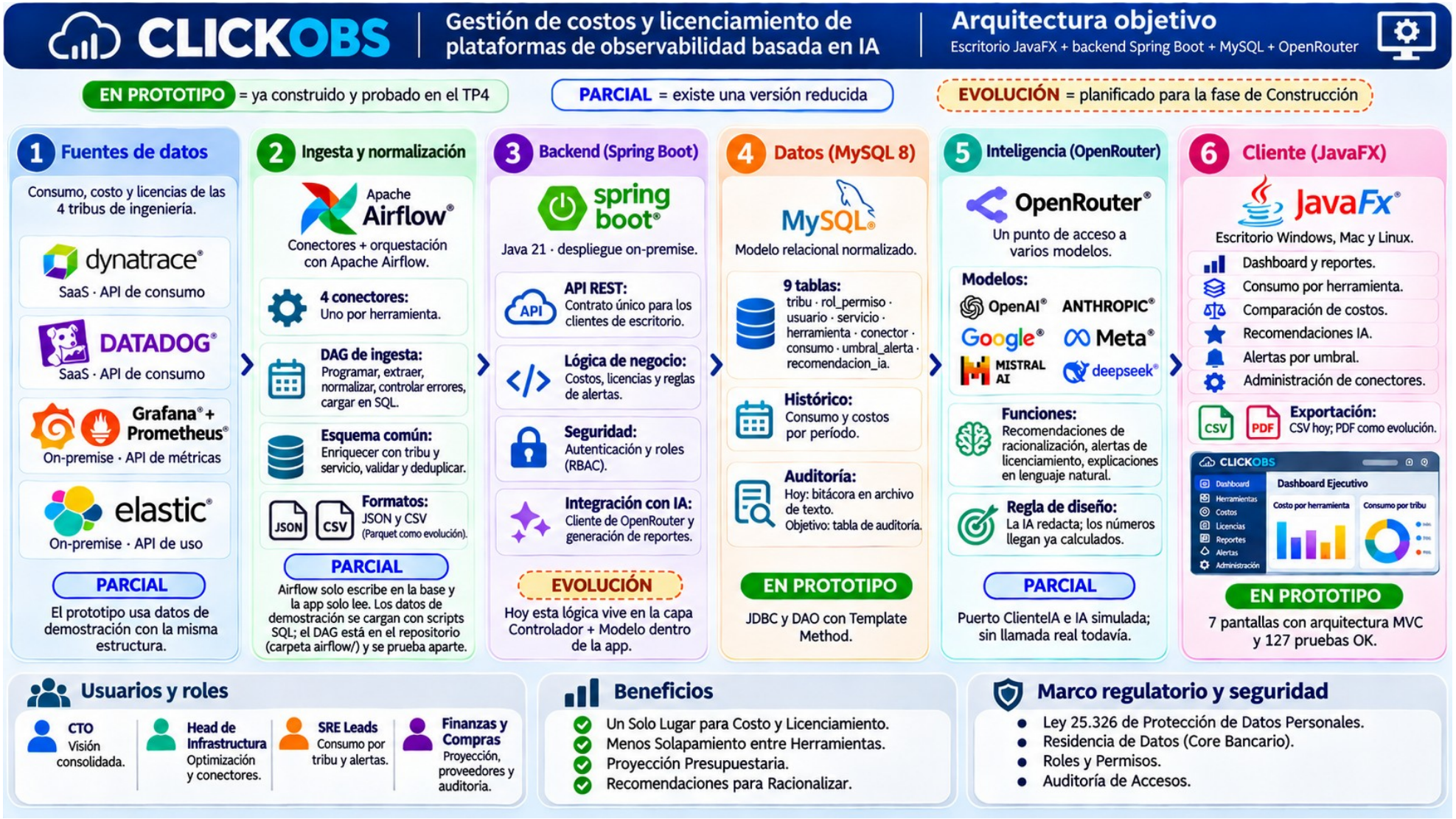


Figura 5. Infografía de la arquitectura objetivo de la solución. Cada columna indica si el componente está en el prototipo, es parcial o es una evolución; el modelo relacional vigente se presenta en la sección 18.3 y la arquitectura del prototipo, en la Figura 43.

Arquitectura objetivo

- **Fuentes de datos:** APIs de Dynatrace y Datadog (SaaS) y de Grafana/Prometheus y Elastic (self-hosted, on-premise).
- **Capa de ingesta y normalización:** conectores por herramienta, con orquestación de las ingestas mediante Apache Airflow y normalización (ETL) a un esquema común.
- **Capa de datos:** base de datos MySQL 8 on-premise, con el modelo relacional descripto en la sección 18.
- **Backend:** aplicación Spring Boot que expone una API REST, concentra la lógica de negocio, la seguridad por roles (RBAC) y la integración con OpenRouter.
- **Capa de IA:** OpenRouter como gateway hacia modelos de lenguaje de terceros; opera sobre métricas ya agregadas por el backend (sección 13).
- **Cliente de escritorio:** aplicación JavaFX para Windows, macOS y Linux, con dashboards, gestión de herramientas, alertas, recomendaciones y exportación de reportes.

Stack tecnológico

- Lenguaje: Java 21 (LTS).
- Interfaz gráfica: JavaFX (FXML y hojas de estilo de JavaFX).
- Backend: Spring Boot, con JPA / Hibernate como ORM.
- Base de datos: MySQL 8.x (acceso por JDBC en el prototipo).
- Orquestación de ingestas: Apache Airflow.
- Formatos de intercambio: JSON y CSV.
- Integración de IA: OpenRouter (API unificada de modelos de lenguaje).
- Control de versiones: Git y GitHub. Gestor de dependencias: Maven.
- Pruebas: JUnit para pruebas unitarias y Postman para pruebas de API.

Infraestructura

- Backend y base de datos desplegados on-premise, en el data center corporativo.
- Cliente de escritorio instalado en las estaciones de trabajo de los usuarios, con comunicación HTTPS hacia el backend.
- Política de respaldos diarios automatizados de la base de datos.

Seguridad

- Autenticación con contraseñas encriptadas (BCrypt).
- Autorización basada en roles (RBAC).
- Comunicación cifrada mediante HTTPS / TLS.
- Anonimización de los datos del Core Bancario antes de cualquier envío al módulo de IA.
- Registro de auditoría de operaciones críticas.

El prototipo de esta entrega implementa un subconjunto de esta arquitectura (sección 16.1).

9. Requerimientos

9.1 Requerimientos funcionales (RF)

Gestión de usuarios y seguridad

ID	Requerimiento
RF-01	El sistema debe permitir el alta, la baja y la modificación de usuarios.
RF-02	El sistema debe permitir la asignación de roles diferenciados (CTO, Head de Infraestructura, SRE Lead, DevOps, Finanzas).
RF-03	El sistema debe autenticar a los usuarios mediante usuario y contraseña antes de permitir su acceso.
RF-04	El sistema debe restringir el acceso a las funcionalidades según el rol del usuario autenticado.
RF-05	El sistema debe registrar un log de auditoría con cada operación crítica.

Gestión de herramientas y conectores

ID	Requerimiento
RF-06	El sistema debe permitir dar de alta una herramienta de observabilidad y configurar sus credenciales de API de forma segura.
RF-07	El sistema debe mostrar el estado de sincronización de cada conector.
RF-08	El sistema debe reintentar automáticamente ante un fallo de sincronización, con registro del error.

Ingesta, normalización y comparación

ID	Requerimiento
RF-09	El sistema debe conectar y sincronizar datos de consumo desde las APIs de Dynatrace, Datadog, Prometheus y Elastic.
RF-10	El sistema debe normalizar los datos de costo y consumo de cada herramienta a un esquema común.
RF-11	El sistema debe comparar costos entre herramientas por servicio y período.
RF-12	El sistema debe detectar y reportar solapamientos de monitoreo entre herramientas.

Recomendaciones IA y alertas

ID	Requerimiento
RF-13	El sistema debe generar recomendaciones de optimización mediante un modelo de IA vía OpenRouter.
RF-14	El sistema debe mantener un historial de recomendaciones para su trazabilidad.
RF-15	El sistema debe permitir configurar umbrales de gasto por herramienta y periodicidad.
RF-16	El sistema debe generar alertas automáticas cuando se supere un umbral configurado.

Reportes y administración

ID	Requerimiento
RF-17	El sistema debe permitir exportar reportes gerenciales en formato PDF o Excel.
RF-18	El sistema debe permitir administrar catálogos auxiliares (tribus, herramientas).

9.2 Requerimientos no funcionales (RNF)

Se contemplan tiempos de respuesta, usabilidad, portabilidad y las normas aplicables al sistema:

ID	Categoría	Requerimiento
RNF-01	Rendimiento	Las consultas comparativas sobre períodos de hasta 12 meses deben responder en menos de 3 segundos.
RNF-02	Disponibilidad	El sistema (backend y base de datos) debe tener una disponibilidad igual o superior al 99% en horario laboral.
RNF-03	Seguridad	Las credenciales de las APIs externas deben almacenarse cifradas (AES-256), nunca en texto plano.
RNF-04	Seguridad	Las contraseñas deben almacenarse encriptadas mediante BCrypt o equivalente.
RNF-05	Usabilidad	La interfaz debe ser utilizable por usuarios no técnicos (Finanzas) sin capacitación previa; un usuario nuevo debe poder realizar las consultas básicas tras 2 horas de capacitación.
RNF-06	Mantenibilidad	El código debe organizarse en capas claramente separadas, siguiendo estándares de codificación Java.
RNF-07	Portabilidad	El cliente de escritorio debe ejecutarse en Windows, macOS y Linux; el backend y la base de datos se despliegan on-premise.
RNF-08	Escalabilidad	La arquitectura debe permitir agregar nuevos conectores sin modificar el núcleo (patrón Adapter).
RNF-09	Costo	El uso de la capa de IA debe tener un tope de presupuesto mensual configurable.
RNF-10	Compliance	Los datos del Core Bancario no deben salir de la infraestructura local ni enviarse a IA externa sin anonimizar.
RNF-11	Cumplimiento legal	El sistema debe cumplir con la Ley N.° 25.326 de Protección de Datos Personales.
RNF-12	Rendimiento (IA)	La recomendación generada por IA debe presentarse en menos de 30 segundos, informando el progreso al usuario mientras se procesa.

10. Casos de uso

10.1 Identificación de actores

Un actor representa un rol externo al sistema que interactúa con él. Para CLICKOBS se identifican los siguientes actores:

- **SRE Lead:** consulta el consumo de su tribu, compara costos entre herramientas y administra conectores.
- **DevOps engineer:** consulta el consumo por herramienta de los servicios que opera.
- **CTO:** solicita recomendaciones de IA y exporta reportes gerenciales para el Directorio.
- **Finanzas / Procurement:** configura alertas de licenciamiento y exporta reportes para el cierre presupuestario.
- **APIs de las herramientas de observabilidad (actor externo):** Dynatrace, Datadog, Prometheus y Elastic, con los que el sistema intercambia datos de consumo mediante sus conectores.
- **OpenRouter (actor externo):** motor de enrutamiento de IA al que el sistema envía los payloads de análisis y del que recibe las recomendaciones.

10.2 Diagrama de casos de uso

El diagrama representa las interacciones entre los actores humanos y el sistema.

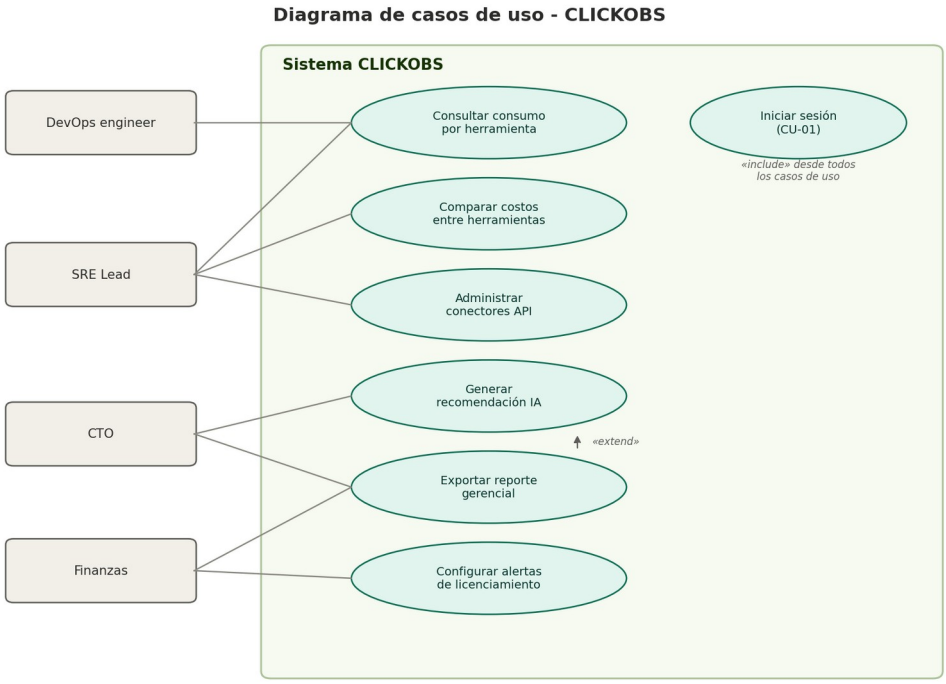


Figura 6. Diagrama de casos de uso del sistema CLICKOBS

«include» indica que un caso de uso siempre invoca a otro como parte obligatoria de su flujo (por ejemplo, todos los casos de uso incluyen «Iniciar sesión»). «extend» indica que un caso de uso extiende opcionalmente el comportamiento de otro bajo ciertas condiciones (por ejemplo,

«Exportar reporte gerencial» extiende a «Generar recomendación IA», ya que la exportación es opcional).

10.3 Especificación de casos de uso

A continuación se especifican en detalle los casos de uso más representativos del sistema.

CU-01 - Iniciar sesión

Identificador	CU-01
Nombre	Iniciar sesión
Actor principal	Cualquier usuario del sistema
Descripción	Permite a un usuario autenticarse para acceder a las funcionalidades del sistema según su rol.
Precondiciones	El usuario debe estar dado de alta en el sistema y tener una cuenta activa.
Postcondiciones	El usuario queda autenticado y se le habilitan las funcionalidades correspondientes a su rol.

Flujo principal:

1. El sistema muestra la pantalla de inicio de sesión.
2. El usuario ingresa su nombre de usuario y su contraseña.
3. El sistema valida las credenciales.
4. El sistema identifica el rol del usuario y carga sus permisos.
5. El sistema muestra la pantalla principal correspondiente a su rol.

Flujos alternativos: si las credenciales son inválidas, el sistema muestra un mensaje de error; si el usuario olvidó su contraseña, puede solicitar su recuperación por correo electrónico.

CU-02 - Comparar costos entre herramientas

Identificador	CU-02
Nombre	Comparar costos entre herramientas
Actor principal	SRE Lead
Descripción	Permite comparar el costo de cobertura entre las 4 herramientas para un período dado.
Precondiciones	Las 4 herramientas tienen sus conectores configurados y con datos ingeridos del último período.
Postcondiciones	El usuario puede identificar visualmente qué servicios tienen cobertura duplicada.
Incluye	CU-01 (Iniciar sesión)

Flujo principal:

1. El usuario selecciona un período.
2. El sistema consulta los datos normalizados en MySQL.
3. El sistema presenta una tabla comparativa de costo por servicio y por herramienta.

Flujo alternativo: si una herramienta no tiene datos del período, el sistema la marca como «sin datos» en lugar de fallar.

CU-03 - Generar recomendación IA

Identificador	CU-03
Nombre	Generar recomendación IA
Actor principal	CTO
Descripción	Permite solicitar un análisis de optimización generado por un modelo de IA.
Precondiciones	Existe al menos un período de datos consolidados.
Postcondiciones	La recomendación queda registrada en el historial para su trazabilidad.
Incluye	CU-01 (Iniciar sesión)
Extendido por	Exportar reporte gerencial

Flujo principal:

1. El usuario solicita un análisis.
2. El sistema arma un payload normalizado con los datos consolidados.
3. El sistema envía el payload vía OpenRouter al modelo de IA seleccionado.
4. El sistema recibe y muestra la recomendación en lenguaje natural, con el ahorro estimado.

CU-04 - Configurar alertas de licenciamiento

Identificador	CU-04
Nombre	Configurar alertas de licenciamiento
Actor principal	Finanzas
Descripción	Permite definir umbrales de gasto por herramienta y recibir notificaciones automáticas.
Precondiciones	El usuario tiene permisos de Finanzas.
Postcondiciones	Finanzas recibe alertas proactivas antes del cierre de facturación.
Incluye	CU-01 (Iniciar sesión)

Flujo principal:

1. El usuario define un umbral mensual para una herramienta.
2. El sistema monitorea el consumo acumulado de esa herramienta.
3. Si se supera el umbral, el sistema dispara una notificación a Finanzas.

11. Plan de proyecto (Gantt)

Se prevé un plan de ejecución de 6 meses (24 semanas) estructurado en seis fases:

Fase	Descripción	Período	Duración
Fase 1	Inicio y relevamiento	05/10 → 23/10	3 semanas
Fase 2	Análisis de requerimientos	26/10 → 13/11	3 semanas

Fase	Descripción	Período	Duración
Fase 3	Diseño (arquitectura, base de datos, UML, BPMN)	16/11 → 11/12	4 semanas
Fase 4	Construcción (desarrollo)	14/12 → 05/02	8 semanas
Fase 5	Pruebas (unitarias, integración, aceptación)	08/02 → 26/02	3 semanas
Fase 6	Puesta en producción y capacitación	01/03 → 19/03	3 semanas

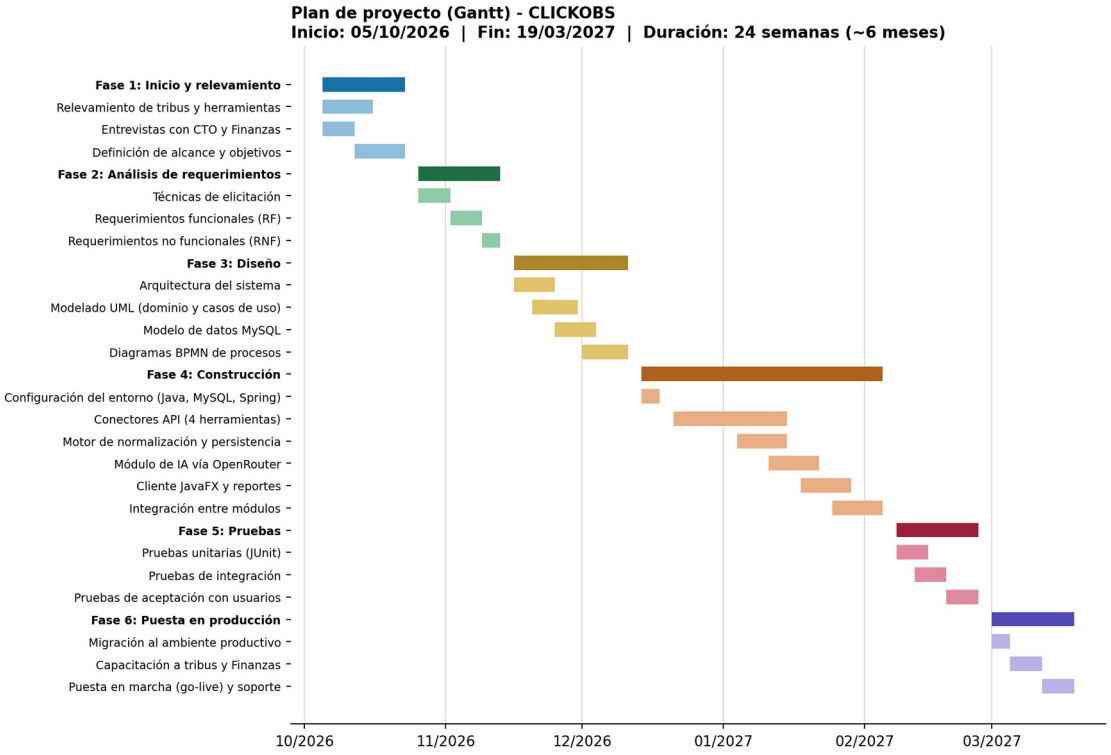


Figura 7. Diagrama de Gantt del proyecto CLICKOBS

PARTE II — FASE DE ELABORACIÓN (TP2)

12. Criterios de diseño

Antes de entrar en los artefactos de cada etapa, se explicitan los criterios que guiaron las decisiones de diseño. Se los formaliza aquí porque atraviesan más de una sección del documento y conviene que no queden implícitos en cada decisión puntual.

12.1 Justificación antes que solución técnica

Toda decisión de este proyecto parte del problema de negocio antes que de la solución técnica: la justificación de CLICKOBS se construyó sobre el costo real del solapamiento de cobertura entre herramientas, no sobre una preferencia tecnológica. El criterio se sostiene en la Parte II fundamentando cada elección de diseño —unificación de tablas, elección de framework, alcance del prototipo— en una razón de negocio o de rendimiento concreta.

12.2 Normalización orientada a las consultas más frecuentes

El modelo de datos se normalizó priorizando la consulta más frecuente del sistema (comparar costos entre herramientas) y no la pureza teórica del modelo de dominio. Por eso Consumo y Costo, que inicialmente eran clases separadas con una relación 1 a 1, se unificaron en una sola tabla: mantenerlas separadas exigía un JOIN adicional en cada consulta comparativa, sin ningún beneficio de integridad a cambio (desarrollo completo en la sección 18.2).

12.3 IA como capa de interpretación, no de cálculo

La inteligencia artificial se usa exclusivamente para interpretar y priorizar datos ya calculados de forma determinística, nunca para reemplazar el cálculo financiero en sí. El backend calcula comparaciones, solapamientos y ahorro estimado con SQL; OpenRouter solo redacta la recomendación en lenguaje natural. El criterio se desarrolla en la sección 13.3.

12.4 Simplicidad del prototipo frente a la arquitectura objetivo

El prototipo operacional de esta entrega prioriza la simplicidad de implementación (cliente JavaFX conectado por JDBC directo a MySQL) por sobre la arquitectura objetivo completa (cliente JavaFX, backend Spring Boot, orquestación de ingestas e integraciones reales) descripta en la sección 8.3. La decisión responde a la propia definición de prototipo de Kendall y Kendall (2011): validar el modelo de datos y la lógica de negocio sin construir todavía toda la infraestructura. Los elementos omitidos se incorporan en la fase de Construcción del PUD (sección 16.1).

12.5 Trazabilidad entre interfaz y casos de uso

Cada pantalla del prototipo (sección 16.3) se diseñó a partir de un caso de uso específico y no al revés: primero se definió el caso de uso y su flujo principal, y luego se derivó la pantalla que lo implementa. Esto garantiza trazabilidad entre requerimiento funcional, caso de uso, clase de interfaz (diagrama de secuencia, sección 14) y pantalla final.

13. Fundamento del uso de IA para el análisis de FinOps

Antes de detallar los diagramas, se fundamenta por qué es viable que CLICKOBS delegue parte del análisis de costos a un modelo de IA vía OpenRouter y cómo se garantiza que ese análisis sea confiable.

13.1 Aclaración sobre la arquitectura de IA

OpenRouter no es en sí mismo un modelo de IA, sino un gateway de enrutamiento: reenvía cada solicitud a un modelo de lenguaje de terceros (por ejemplo, Claude, GPT o Llama) según el criterio de selección configurado y devuelve la respuesta. No existe una «IA propia» entrenada ni dedicada a CLICKOBS: cada llamada es sin estado y el contexto de negocio se incluye completo en el payload de cada invocación. Quien orquesta el análisis es el backend de CLICKOBS (Gestor de Recomendaciones); OpenRouter y el modelo subyacente son un servicio externo que se consulta puntualmente.

13.2 Viabilidad práctica

Incorporar IA al análisis de costos no es una propuesta experimental: las prácticas de FinOps para 2026 evolucionan hacia una optimización predictiva y automatizada apoyada en IA, con capacidades como el análisis predictivo del gasto, la detección de anomalías y la recomendación de configuraciones más eficientes en costo. Además, la propia disciplina FinOps debe hoy gestionar el gasto en IA, lo que respalda el criterio de este proyecto de acotar el costo de la capa de IA (RNF-09). El diseño de CLICKOBS se alinea con esa tendencia: usa la IA para interpretar y priorizar datos ya calculados, no para reemplazar el análisis financiero.

13.3 Salvaguardas de diseño

Para que el análisis sea confiable, el flujo completo (Figura 8) aplica tres principios:

- **Cálculo determinístico primero:** la comparación de costos, la detección de solapamientos y el ahorro estimado se calculan con SQL y lógica de backend; no se le piden a la IA. El modelo de lenguaje solo recibe métricas ya agregadas y devuelve una narrativa explicativa, nunca los números finales que se muestran al usuario.
- **Datos mínimos y agregados:** al payload que sale hacia OpenRouter solo viajan montos, nombres de herramienta y servicio y período; nunca telemetría cruda ni datos que identifiquen clientes. Para el Core Bancario, además, esos datos se anonimizan antes de salir (RNF-10).
- **Decisión humana final:** la recomendación se muestra en la aplicación para que el CTO decida; el sistema nunca ejecuta automáticamente una baja o migración de herramienta.

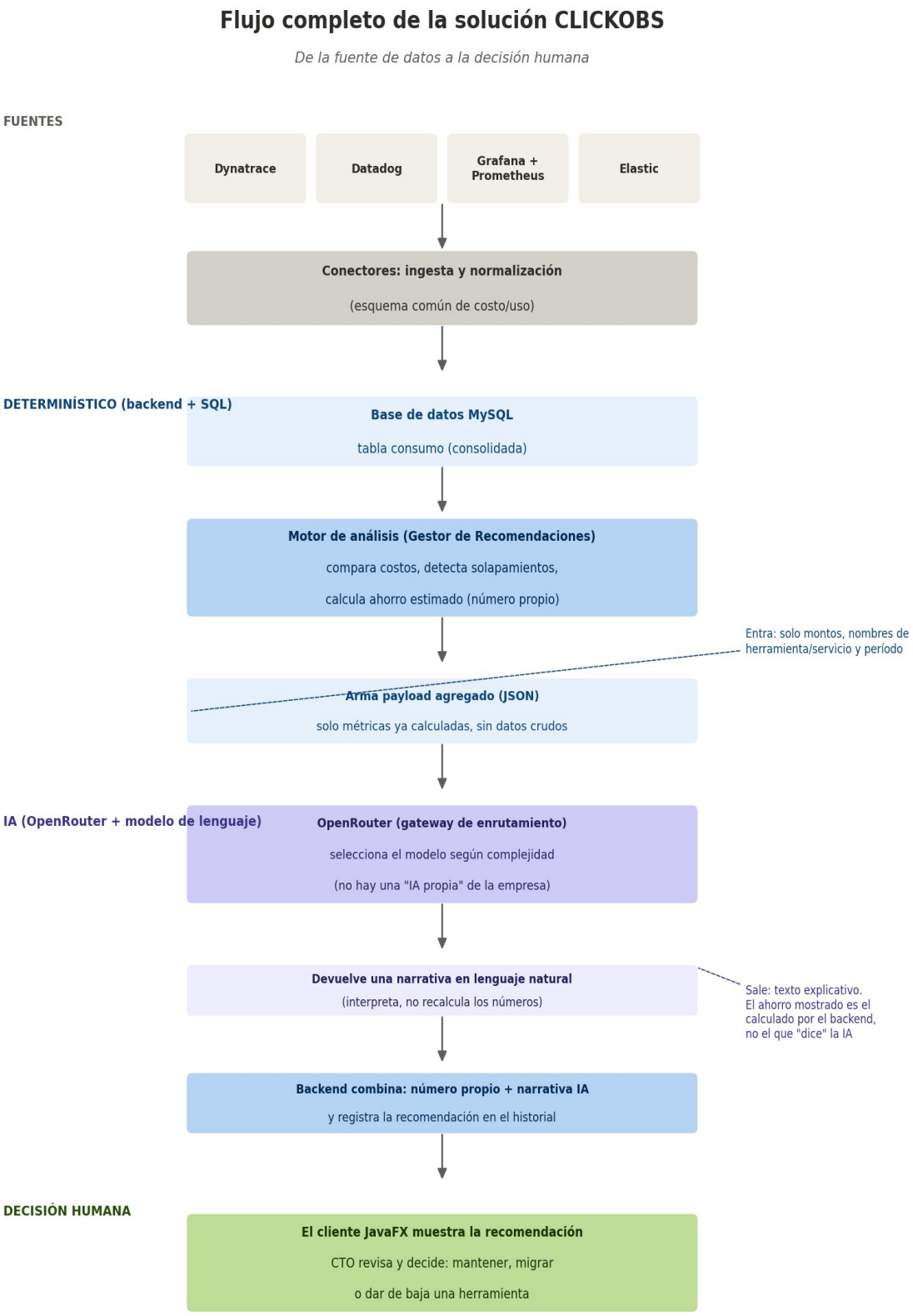


Figura 8. Flujo de la solución: de la fuente de datos a la decisión humana

14. Etapa de análisis: diagrama de secuencia

Se modela el flujo de interacción entre objetos para el caso de uso CU-03 (Generar recomendación IA). El diagrama distingue los tres estereotipos de clases de análisis que propone UML (Rumbaugh et al., 2006): una clase de interfaz (Panel de Recomendaciones), una clase de control (Gestor de Recomendaciones) y una clase de entidad (Consumo); además incluye la interacción con OpenRouter como sistema externo y detalla, en cada clase, un atributo clave.

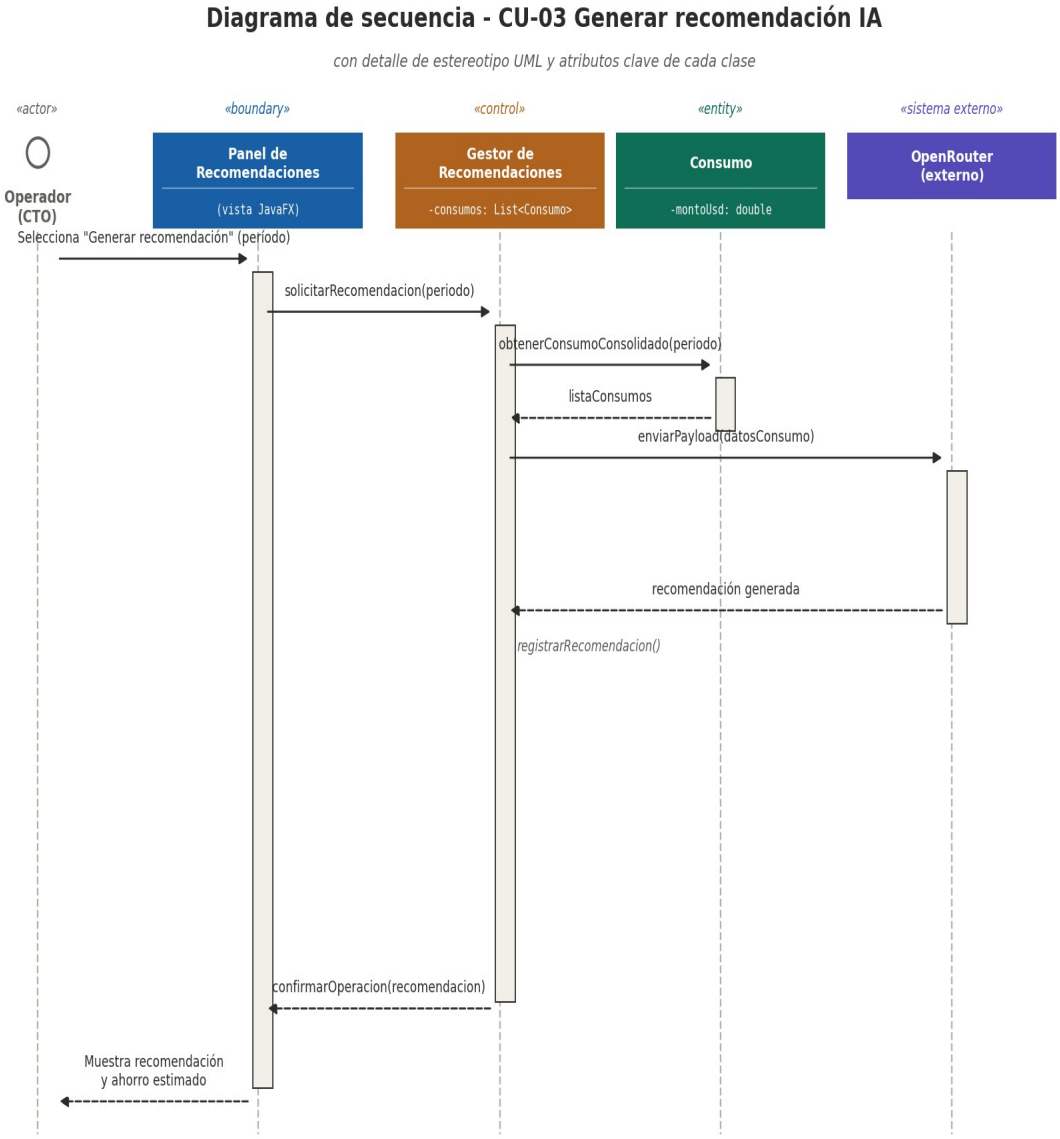


Figura 9. Diagrama de secuencia correspondiente al CU-03

El flujo comienza cuando el CTO selecciona la opción de generar una recomendación para un período determinado. El Panel de Recomendaciones delega en el Gestor de Recomendaciones, que consulta el consumo consolidado, arma el payload y lo envía a OpenRouter. La respuesta se registra y se devuelve al usuario junto con el ahorro estimado.

15. Etapa de diseño: diagrama de clases

El diagrama de clases de diseño representa la estructura estática de las clases involucradas en el CU-03, con sus atributos, métodos y relaciones (Booch, 1996). Incluye una jerarquía de herencia que resuelve una necesidad concreta del dominio: las herramientas de observabilidad de CLICKOBS S.A. se facturan de dos maneras fundamentalmente distintas (SaaS, con costo reportado por el proveedor, o self-hosted, con costo estimado desde la infraestructura propia), por lo que el cálculo del costo por período se modela como un método que cada subclase implementa de forma diferente.

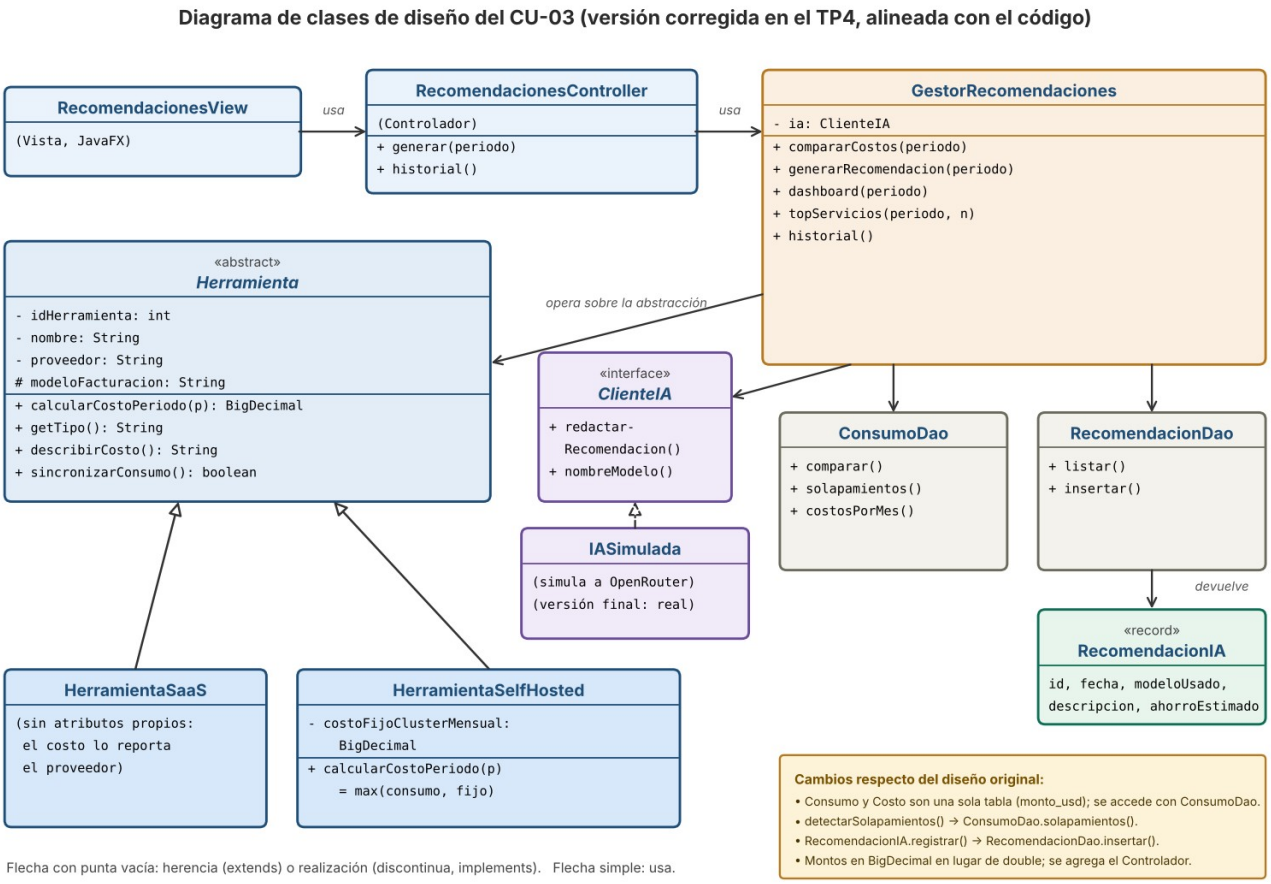


Figura 10. Diagrama de clases de diseño del CU-03 de CLICKOBS (corregido en el TP4 para coincidir con el código)

La clase abstracta Herramienta define el contrato común (los métodos abstractos calcularCostoPeriodo, getTipo y describirCosto, y sincronizarConsumo). HerramientaSaaS lo implementa consultando el costo reportado por la API del proveedor; HerramientaSelfHosted lo implementa estimándolo a partir del costo de infraestructura propia declarado. El Gestor de Recomendaciones opera sobre la abstracción Herramienta sin conocer la subclase concreta, lo que permite incorporar nuevas herramientas sin modificar la lógica de orquestación. La tabla consumo, que unifica consumo y costo mediante el campo monto_usd, es una mejora de normalización (sección 18.2).

Nota de la fase de Construcción (TP3): este diagrama es el de diseño. Al programar se hicieron ajustes (montos en BigDecimal, costoFijoClusterMensual en la herramienta self-hosted, endpoint y token fuera de las clases); el diagrama de las clases realmente implementadas se presenta en la sección 22.2 (Figura 21).

Corrección de la versión final (TP4). Se corrigió la Figura 10 para que el diseño coincida con lo implementado: el diagrama original mostraba una clase Consumo, un método detectarSolapamientos() en el gestor, un método registrar() en la recomendación y montos en double, y en el código esos elementos se resolvieron de otra forma (la tabla consumo se accede con ConsumoDao, los solapamientos los calcula ConsumoDao.solapamientos(), la recomendación se guarda con RecomendacionDao.insertar() y los montos son BigDecimal). También se incorporan el controlador, que mantiene a la pantalla separada del gestor (sección 23.5.1), y la interfaz ClienteIA con su implementación simulada, que será reemplazada por OpenRouter en la versión final.

16. Etapa de implementación

La etapa de implementación traduce los modelos de análisis y diseño en un prototipo operacional, según la definición de Kendall y Kendall (2011): «la creación de un modelo operacional que incluya solo algunas características del sistema final». El sistema final es la aplicación de escritorio JavaFX con backend Spring Boot descripta en la sección 8.3; el prototipo implementa un subconjunto de esa arquitectura.

16.1 Arquitectura del prototipo

El prototipo es una aplicación de escritorio en Java (JavaFX o Swing como framework de interfaz gráfica) que se conecta directamente a la base de datos MySQL mediante JDBC (MySQL Connector/J). Respecto de la arquitectura objetivo se omiten deliberadamente tres elementos: el backend Spring Boot (por ahora la lógica de negocio vive en el propio cliente), la orquestación de ingestas con Airflow (los datos se cargan con los scripts SQL de la sección 19) y las integraciones reales con las APIs de las herramientas y con OpenRouter (se simulan con datos de prueba). Estos elementos se incorporan en la fase de Construcción del PUD. La simplificación permite validar el modelo de datos, la lógica de comparación de costos y la experiencia de usuario antes de construir toda la infraestructura.

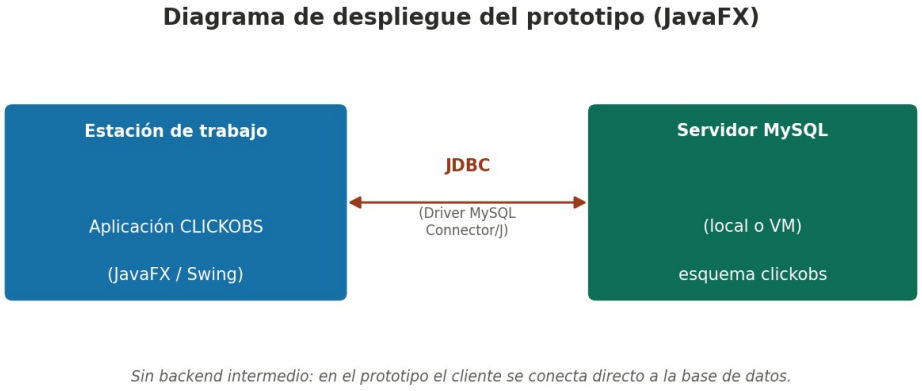


Figura 11. Diagrama de despliegue del prototipo (JavaFX + JDBC)

16.2 Requerimientos de implementación

ID	Requerimiento
RI-01	El prototipo debe desarrollarse en Java, utilizando JavaFX o Swing para la interfaz gráfica de escritorio.

ID	Requerimiento
RI-02	El prototipo debe conectarse a la base de datos MySQL mediante JDBC (MySQL Connector/J).
RI-03	El prototipo debe ejecutarse como aplicación de escritorio independiente, sin requerir un servidor web.
RI-04	El código fuente y los archivos asociados deben publicarse en un repositorio de GitHub (sección 21).

16.3 Prototipo de interfaz gráfica y su vinculación con los casos de uso

A continuación se presentan tres pantallas del prototipo (mockups), cada una vinculada de forma directa a un caso de uso definido en la sección 10.3. Esta trazabilidad entre requerimiento funcional, caso de uso y clase de interfaz es la que exige el modelo de análisis del PUD: cada pantalla materializa la clase de interfaz que ya aparecía en el diagrama de secuencia (sección 14) y en el diagrama de casos de uso (Figura 6).

Pantalla 1: Iniciar sesión (CU-01)

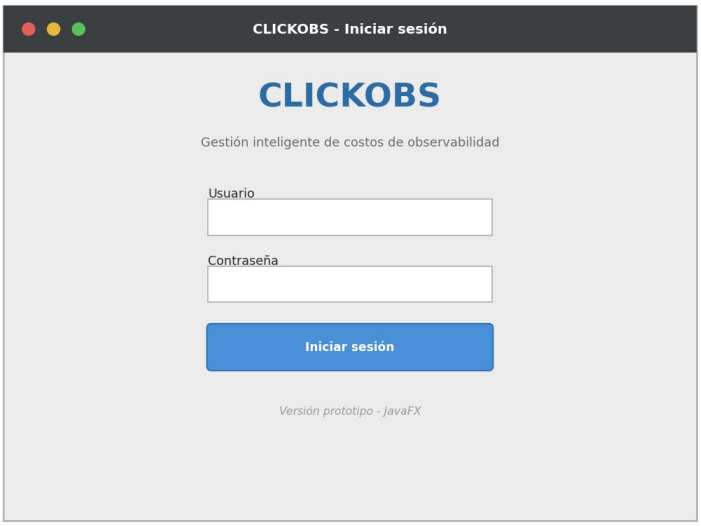


Figura 12. Mockup de interfaz: pantalla de inicio de sesión

Se vincula con CU-01 (Iniciar sesión), incluido por todos los demás casos de uso. Implementa el flujo de autenticación de su especificación: el usuario ingresa usuario y contraseña, y el sistema valida las credenciales antes de habilitar el resto de las pantallas según el rol (RF-03 y RF-04).

Pantalla 2: Comparar costos entre herramientas (CU-02)

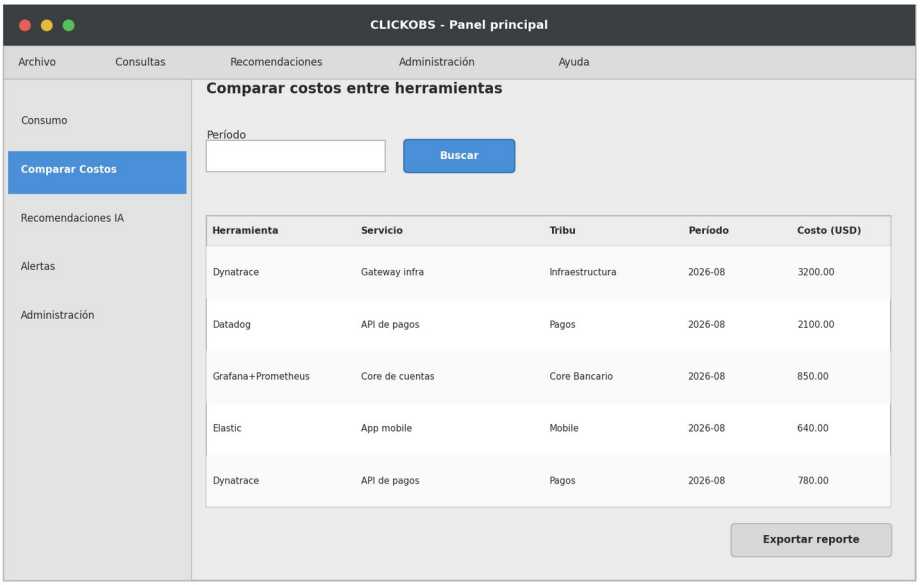


Figura 13. Mockup de interfaz: pantalla de comparación de costos

Materializa el caso de uso CU-02: el usuario (SRE Lead) selecciona un período y el sistema presenta la tabla comparativa de costo por herramienta y servicio, tal como se especificó en su flujo principal y se validó con la consulta SQL de la sección 19.3. El botón «Exportar reporte» corresponde al RF-17.

Pantalla 3: Generar recomendación IA (CU-03)

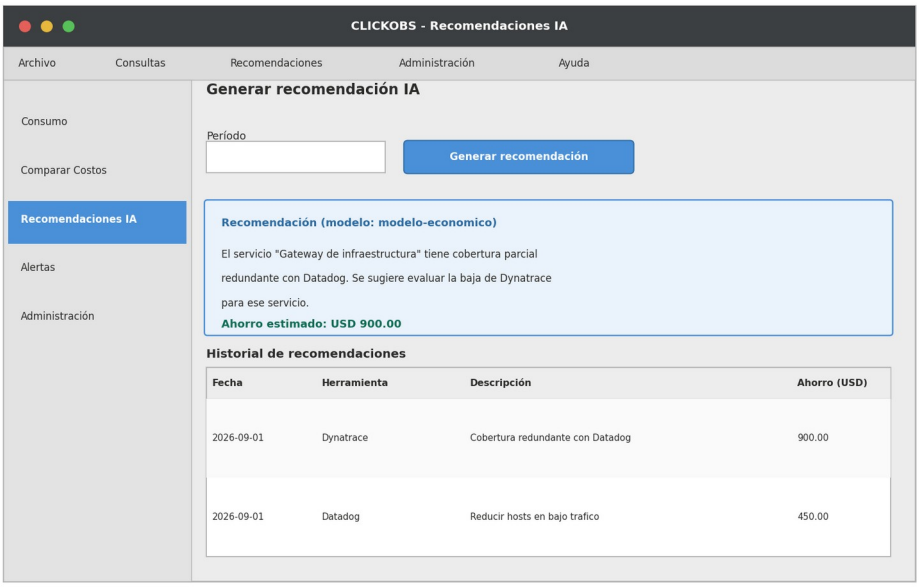


Figura 14. Mockup de interfaz: pantalla de recomendaciones IA

Es la clase de interfaz «Panel de Recomendaciones» del diagrama de secuencia (sección 14), expresada como pantalla. Implementa el flujo completo de CU-03: selección de período, botón para disparar la generación (que dispara la llamada a OpenRouter descripta en la sección 13), tarjeta con la narrativa y el ahorro estimado, e historial de recomendaciones previas (RF-14).

Nota de la fase de Construcción (TP3): el programa respeta estos tres mockups, con los cambios y agregados descriptos en la sección 22.7; las capturas de las pantallas implementadas figuran allí (Figuras 22 a 30).

17. Etapa de pruebas

Siguiendo los artefactos de la etapa de pruebas del PUD, se presentan el plan de pruebas, los casos de prueba (con técnicas de caja blanca y de caja negra), el tratamiento de defectos y la evaluación de prueba correspondientes al caso de uso CU-02 (Comparar costos entre herramientas).

17.1 Plan de pruebas

El plan indica qué tipos de prueba se realizan, en qué orden y cuáles son los criterios de aceptación. Se utiliza el código de prueba con la nomenclatura CP (caso de prueba) + letra de tipo de prueba + número correlativo.

Caso de uso	Código prueba	Tipo de prueba	Técnica	Observaciones
CU-02	CPC01	Componente	Caja blanca	Cobertura del método compararCostos() de GestorRecomendaciones
CU-02	CPI01	Integración	Caja negra	Verifica que la consulta con JOIN devuelva filas consistentes entre consumo, herramienta, servicio y tribu
CU-02	CPS01	Sistema	Caja negra	Ejecuta el flujo completo desde la pantalla (Figura 13) hasta la base de datos

Criterio de aceptación: el caso de uso se considera aprobado si los tres niveles de prueba (componente, integración y sistema) se ejecutan sin excepciones no controladas y no quedan defectos de severidad alta o crítica sin resolver.

17.2 Casos de prueba (caja blanca y caja negra)

CPC01 aplica la técnica de caja blanca: se conoce y se prueba la lógica interna del método compararCostos(), incluyendo sus caminos condicionales (período con datos, sin datos y con datos parciales). CPI01 y CPS01 aplican caja negra: se prueban entradas y salidas sin considerar la implementación interna, verificando únicamente que el resultado visible (la grilla de la Figura 13) sea el esperado para cada escenario.

#	Código	Técnica	Escenario	Resultado esperado
1	CPC01	Caja blanca	Período sin datos cargados para ninguna herramienta	El sistema muestra la tabla vacía, sin error
2	CPC01	Caja blanca	Período con datos en las 4 herramientas	La grilla muestra la comparación completa, ordenada por período y herramienta
3	CPC01	Caja blanca	Una herramienta sin datos en el período (las otras sí)	Las herramientas con datos se listan igual; la faltante no genera excepción

#	Código	Técnica	Escenario	Resultado esperado
4	CPI01	Caja negra	Un servicio cubierto por más de una herramienta en el mismo período	Ambas filas aparecen por separado, permitiendo detectar visualmente el solapamiento
5	CPS01	Caja negra	Un usuario sin rol SRE Lead intenta acceder a la pantalla	El sistema deniega el acceso según el rol (RF-04)

17.3 Tratamiento de defectos

Se registran los defectos detectados durante la ejecución de los casos de prueba anteriores, con su severidad y su estado de resolución.

ID	Caso de prueba	Descripción	Severidad	Estado
DEF-01	CPC01 (#3)	El campo periodo no valida el formato AAAA-MM al insertar, lo que permitiría cargar valores inconsistentes	Baja	Abierto: se agregará la validación en la próxima iteración
DEF-02	CPS01 (#5)	El mensaje de acceso denegado no especifica qué rol se requiere para la pantalla solicitada	Baja	Abierto: mejora de usabilidad, no bloquea la entrega

17.4 Evaluación de prueba

De los 3 casos de prueba planificados (CPC01, CPI01 y CPS01) y los 5 escenarios ejecutados, no se detectaron defectos de severidad alta o crítica: los dos defectos registrados (DEF-01 y DEF-02) son de severidad baja y no impiden el uso del prototipo. Según el criterio de aceptación definido en el plan de pruebas (sección 17.1), el caso de uso CU-02 se considera aprobado para esta entrega; ambos defectos quedan documentados para su resolución en la próxima iteración del PUD (fase de Construcción).

18. Modelo de datos relacional

18.1 Definición de base de datos para el sistema

Para el desarrollo del sistema de gestión y monitoreo de costos de CLICKOBS S.A. se requiere una base de datos relacional. Tal como se especifica en los requerimientos de implementación (RI-02), el prototipo utiliza una base MySQL, que ofrece buen rendimiento, flexibilidad y escalabilidad para el volumen de datos manejado.

El uso de una base de datos es fundamental para lograr la persistencia de los datos clave del negocio: el consumo y el costo de cada herramienta por servicio y período, los umbrales de alerta configurados y el historial de recomendaciones generadas por IA. Esto es esencial para mantener un histórico completo que permita comparar costos entre períodos y detectar solapamientos de cobertura, que son el valor central de CLICKOBS.

18.2 Decisión de diseño: unificación de Consumo y Costo

En la primera versión del diagrama de dominio, Consumo y Costo se modelaron como clases separadas con una relación 1 a 1. Al revisar la cardinalidad para el modelo relacional —evaluando si convenía mantenerlas separadas o si había redundancia real— se detectó que la separación era redundante: el campo fecha de Costo duplicaba al campo periodo de Consumo, y mantener las tablas separadas obligaba a un JOIN adicional en la consulta más frecuente del sistema (comparar costos entre herramientas), sin que existiera una razón de negocio (dueño, ciclo de vida o frecuencia de acceso distintos) que justificara la separación.

Por lo tanto, se unificaron ambas entidades en una única tabla consumo, que incorpora el campo monto_usd directamente. La decisión reduce la complejidad de las consultas comparativas y elimina el campo de fecha redundante, sin perder ninguna capacidad funcional. El diagrama de dominio (sección 6) y el diagrama de clases de diseño ya reflejan esta mejora.

18.3 Diagrama entidad-relación (DER)

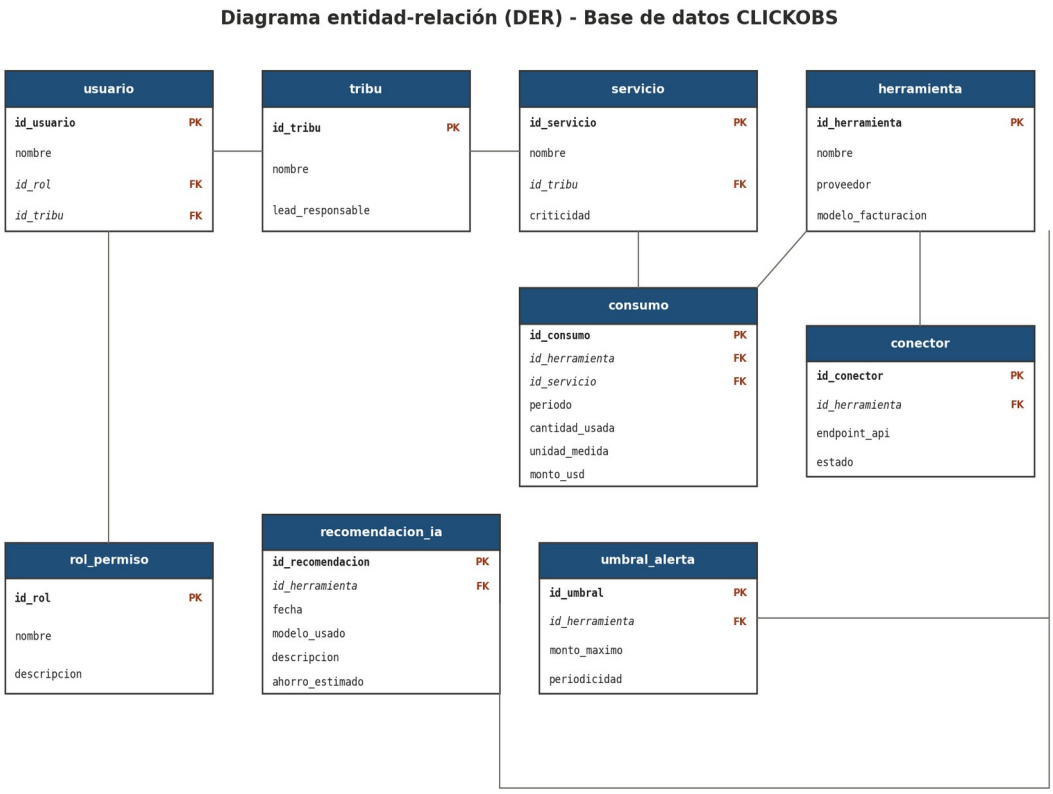


Figura 15. Diagrama entidad-relación de la base de datos CLICKOBS

18.4 Descripción de las tablas

Tabla	Descripción
tribu	Equipos de ingeniería de CLICKOBS S.A. (Infraestructura, Pagos, Core Bancario, Mobile).
rol_permiso	Roles de acceso al sistema (CTO, SRE Lead, DevOps, Finanzas).

Tabla	Descripción
usuario	Personas que usan el sistema, con un rol y una tribu asignados.
servicio	Microservicios o sistemas monitoreados, asociados a una tribu.
herramienta	Las 4 plataformas de observabilidad (Dynatrace, Datadog, Grafana/Prometheus, Elastic).
conector	Integración técnica con la API de cada herramienta.
consumo	Consumo y costo (unificados) por herramienta, servicio y período.
umbral_alerta	Límites de gasto configurados por herramienta.
recomendacion_ia	Recomendaciones generadas por el modelo de IA, con su ahorro estimado.

18.5 Verificación de reglas de negocio, normalización e integridad

Al cerrar el diseño de la base se revisó el modelo contra cuatro criterios: que las relaciones respeten las reglas de negocio de los casos de uso, el nivel de normalización y su efecto en las consultas más frecuentes, la definición de claves primarias y foráneas con su integridad referencial, y que las consultas devuelvan lo esperado con los datos de prueba.

Relaciones y reglas de negocio. Cada servicio pertenece a una tribu y cada usuario tiene un rol y una tribu (claves foráneas obligatorias). La cobertura de un servicio por varias herramientas, que es la base de la detección de solapamientos del CU-03, se resuelve con la tabla consumo, que actúa como relación muchos a muchos entre herramienta y servicio. Cada herramienta tiene un umbral de alerta (CU-04) y un conector. La recomendación IA admite herramienta nula, porque puede referirse a un conjunto de herramientas.

Normalización. El modelo está en tercera forma normal con dos decisiones deliberadas. (1) Consumo y Costo se unificaron (relación 1 a 1), de modo que la consulta más frecuente (CU-02, comparar costos) une solo cuatro tablas en lugar de cinco. (2) La unidad de medida se mantiene en consumo aunque en los datos de prueba dependa de la herramienta, porque una misma herramienta puede facturar en varias unidades (por ejemplo, Datadog por host y por métrica personalizada); separarla agregaría otro JOIN sin beneficio. Como todas las consultas filtran por período, se agrega un índice sobre esa columna.

Integridad. Se probaron operaciones inválidas contra la base. Las claves primarias y foráneas rechazaron todo lo que debían, pero la revisión encontró reglas que cumplía solo la aplicación y no la base: consumos duplicados, períodos o montos inválidos y más de un umbral o conector por herramienta. Se corrigió con un script opcional (db/Endurecimiento_CLICKOBS.sql: restricciones UNIQUE, CHECK y NOT NULL) que se ejecuta después de la creación de tablas; no cambia columnas ni datos de prueba. La evidencia de las figuras 16 a 19 corresponde a los scripts originales.

Prueba de integridad	Modelo original	Con hardening
Consumo con una herramienta inexistente	Rechazado (FK)	Rechazado
Consumo con un servicio inexistente	Rechazado (FK)	Rechazado
Usuario con un rol inexistente	Rechazado (FK)	Rechazado
Borrar una tribu o herramienta con datos asociados	Rechazado (FK)	Rechazado

Prueba de integridad	Modelo original	Con hardening
Clave primaria repetida	Rechazado (PK)	Rechazado
Consumo repetido (misma herramienta, servicio y período)	**Aceptado (defecto)**	Rechazado (UNIQUE)
Período «basura» o monto negativo	**Aceptado (defecto)**	Rechazado (CHECK)
Segundo umbral para la misma herramienta	**Aceptado (defecto)**	Rechazado (UNIQUE)
Segundo conector para la misma herramienta	**Aceptado (defecto)**	Rechazado (UNIQUE)

Consultas. La consulta del CU-02 (sección 19.3) devuelve con los datos de prueba las cinco filas esperadas, ordenadas por herramienta, y los costos suman USD 7.570 (3.200 + 2.100 + 850 + 640 + 780). La detección de solapamientos encuentra el servicio «API de pagos», cubierto por Datadog (USD 2.100) y Dynatrace (USD 780), y estima un ahorro de USD 780, que es lo que muestra la pantalla de recomendaciones (Figura 30).

19. Scripts SQL

Se presentan los cuatro scripts que implementan el modelo relacional en MySQL: creación de tablas, inserción de datos de prueba, consulta comparativa y borrado con verificación. Los cuatro fueron ejecutados en un entorno MySQL (con MySQL Workbench como cliente), cuya evidencia se incluye en cada apartado.

19.1 Creación de tablas

```
-- CLICKOBS - Creación de tablas
-- Nota de diseño: Consumo y Costo se unificaron en una sola tabla (consumo),
-- ya que tenían una relación 1 a 1 y el campo "fecha" de Costo duplicaba
-- al campo "periodo" de Consumo. Mantenerlas separadas obligaba a un JOIN
-- adicional en la consulta más frecuente del sistema (comparar costos).

CREATE TABLE tribu (
    id_tribu INT PRIMARY KEY,
    nombre VARCHAR(100),
    lead_responsable VARCHAR(150)
);

CREATE TABLE rol_permiso (
    id_rol INT PRIMARY KEY,
    nombre VARCHAR(100),
    descripcion VARCHAR(255)
);

CREATE TABLE usuario (
    id_usuario INT PRIMARY KEY,
    nombre VARCHAR(150),
    id_rol INT,
    id_tribu INT,
    FOREIGN KEY (id_rol) REFERENCES rol_permiso(id_rol),
    FOREIGN KEY (id_tribu) REFERENCES tribu(id_tribu)
);

CREATE TABLE servicio (
    id_servicio INT PRIMARY KEY,
    nombre VARCHAR(150),
    id_tribu INT,
    criticidad VARCHAR(50),
```

```
    FOREIGN KEY (id_tribu) REFERENCES tribu(id_tribu)
);

CREATE TABLE herramienta (
    id_herramienta INT PRIMARY KEY,
    nombre VARCHAR(100),
    proveedor VARCHAR(150),
    modelo_facturacion VARCHAR(100)
);

CREATE TABLE conector (
    id_conector INT PRIMARY KEY,
    id_herramienta INT,
    endpoint_api VARCHAR(255),
    estado VARCHAR(50),
    FOREIGN KEY (id_herramienta) REFERENCES herramienta(id_herramienta)
);

CREATE TABLE consumo (
    id_consumo INT PRIMARY KEY,
    id_herramienta INT,
    id_servicio INT,
    periodo VARCHAR(20),
    cantidad_usada DECIMAL(12, 2),
    unidad_medida VARCHAR(50),
    monto_usd DECIMAL(12, 2),
    FOREIGN KEY (id_herramienta) REFERENCES herramienta(id_herramienta),
    FOREIGN KEY (id_servicio) REFERENCES servicio(id_servicio)
);

CREATE TABLE umbral_alerta (
    id_umbral INT PRIMARY KEY,
    id_herramienta INT,
    monto_maximo DECIMAL(12, 2),
    periodicidad VARCHAR(50),
    FOREIGN KEY (id_herramienta) REFERENCES herramienta(id_herramienta)
);

CREATE TABLE recomendacion_ia (
    id_recomendacion INT PRIMARY KEY,
    id_herramienta INT,
    fecha DATETIME,
    modelo_usado VARCHAR(100),
    descripcion VARCHAR(500),
    ahorro_estimado DECIMAL(12, 2),
    FOREIGN KEY (id_herramienta) REFERENCES herramienta(id_herramienta)
);
```

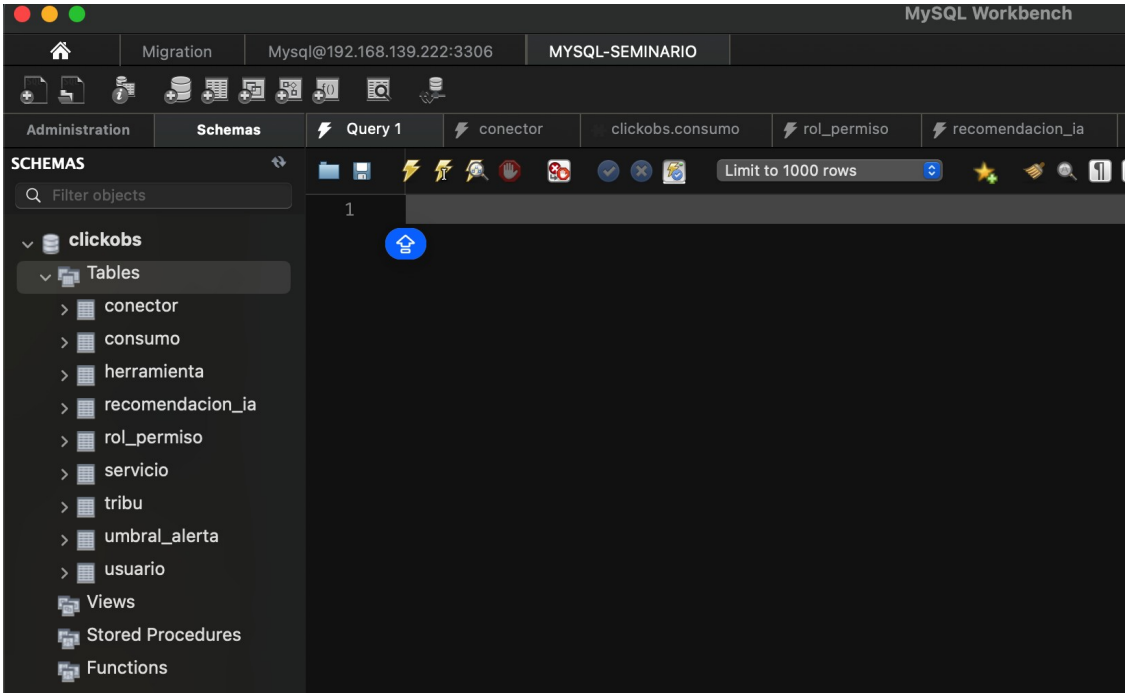


Figura 16. Evidencia de ejecución: creación de tablas

19.2 Inserción de datos de prueba

Los datos de prueba se cargaron sin tildes para evitar problemas de codificación de caracteres entre el cliente y el servidor MySQL.

```
INSERT INTO tribu (id_tribu, nombre, lead_responsable)
VALUES
    (1, 'Infraestructura', 'Head de Infraestructura'),
    (2, 'Pagos', 'Tech Lead Pagos'),
    (3, 'Core Bancario', 'Tech Lead Core Bancario'),
    (4, 'Mobile', 'Tech Lead Mobile');


INSERT INTO rol_permiso (id_rol, nombre, descripcion)
VALUES
    (1, 'CTO', 'Acceso a recomendaciones IA y reportes gerenciales'),
    (2, 'SRE Lead', 'Consulta y comparacion de costos, administracion de
conectores'),
    (3, 'DevOps', 'Consulta de consumo por herramienta'),
    (4, 'Finanzas', 'Configuracion de alertas y exportacion de reportes');


INSERT INTO usuario (id_usuario, nombre, id_rol, id_tribu)
VALUES
    (1, 'dfarias', 1, 1),
    (2, 'sre.infra', 2, 1),
    (3, 'devops.pagos', 3, 2),
    (4, 'finanzas', 4, 1);


INSERT INTO servicio (id_servicio, nombre, id_tribu, criticidad)
VALUES
    (1, 'API de pagos', 2, 'Alta'),
    (2, 'Core de cuentas', 3, 'Alta'),
    (3, 'App mobile', 4, 'Media'),
    (4, 'Gateway de infraestructura', 1, 'Alta');
```

```
INSERT INTO herramienta (id_herramienta, nombre, proveedor, modelo_facturacion)
VALUES
    (1, 'Dynatrace', 'Dynatrace LLC', 'DDU'),
    (2, 'Datadog', 'Datadog Inc', 'Host/Metrica custom'),
    (3, 'Grafana+Prometheus', 'Self-hosted', 'Infraestructura propia'),
    (4, 'Elastic', 'Elastic NV', 'Almacenamiento');


INSERT INTO conector (id_conector, id_herramienta, endpoint_api, estado)
VALUES
    (1, 1, 'https://api.dynatrace.com/v2/metrics', 'Activo'),
    (2, 2, 'https://api.datadoghq.com/api/v2/usage', 'Activo'),
    (3, 3, 'http://prometheus.clickobs.internal:9090/api/v1', 'Activo'),
    (4, 4, 'https://api.elastic-cloud.com/api/v1/billing', 'Activo');


INSERT INTO consumo (id_consumo, id_herramienta, id_servicio, periodo,
cantidad_usada, unidad_medida, monto_usd)
VALUES
    (1, 1, 4, '2026-08', 1250.00, 'DDU', 3200.00),
    (2, 2, 1, '2026-08', 340.00, 'hosts', 2100.00),
    (3, 3, 2, '2026-08', 980.00, 'GB almacenados', 850.00),
    (4, 4, 3, '2026-08', 210.00, 'GB indexados', 640.00),
    (5, 1, 1, '2026-08', 300.00, 'DDU', 780.00);


INSERT INTO umbral_alerta (id_umbral, id_herramienta, monto_maximo, periodicidad)
VALUES
    (1, 1, 3500.00, 'Mensual'),
    (2, 2, 2500.00, 'Mensual'),
    (3, 4, 700.00, 'Mensual');


INSERT INTO recomendacion_ia (id_recomendacion, id_herramienta, fecha,
modelo_usado, descripcion, ahorro_estimado)
VALUES
    (1, 1, '2026-09-01 10:00:00', 'modelo-economico', 'El servicio Gateway de
infraestructura tiene cobertura parcial redundante con Datadog', 900.00),
    (2, 2, '2026-09-01 10:05:00', 'modelo-robusto', 'La tribu de Pagos podria
reducir hosts monitoreados en horarios de bajo trafico', 450.00);
```

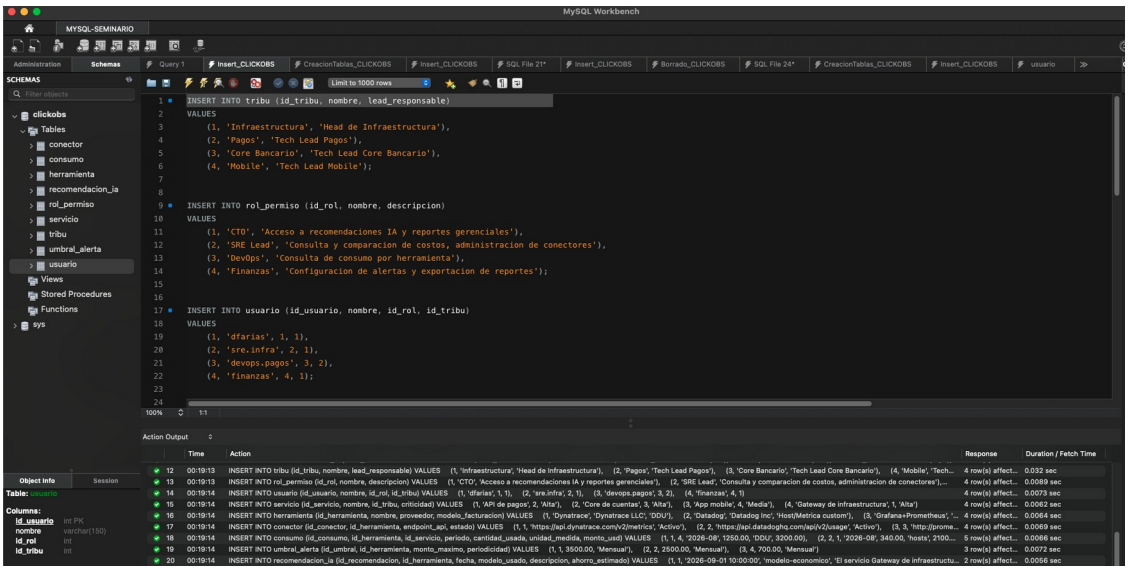


Figura 17. Evidencia de ejecución: inserción de datos y usuarios cargados

19.3 Consulta de datos (CU-02: comparar costos entre herramientas)

```
-- Consulta correspondiente a CU-02 "Comparar costos entre herramientas"
SELECT
    H.nombre AS 'Herramienta',
    S.nombre AS 'Servicio',
    T.nombre AS 'Tribu',
    C.periodo AS 'Periodo',
    C.cantidad_usada AS 'Cantidad Usada',
    C.unidad_medida AS 'Unidad',
    C.monto_usd AS 'Costo (USD)'
FROM
    consumo C,
    herramienta H,
    servicio S,
    tribu T
WHERE
    C.id_herramienta = H.id_herramienta
    AND C.id_servicio = S.id_servicio
    AND S.id_tribu = T.id_tribu
ORDER BY
    C.periodo, H.nombre;
```

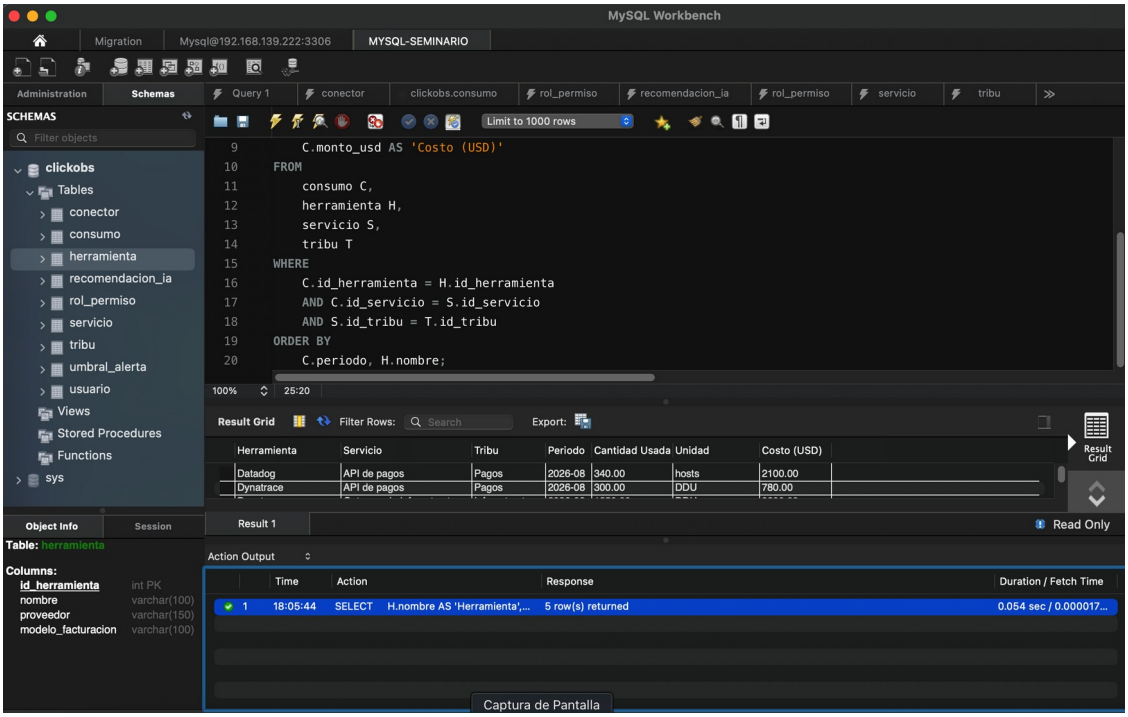


Figura 18. Evidencia de ejecución: consulta comparativa de costos

19.4 Borrado de datos y verificación

```
-- Se desactiva temporalmente el modo Safe Updates de MySQL Workbench,
-- que bloquea los DELETE sin WHERE sobre columna clave.
SET SQL_SAFE_UPDATES = 0;

-- Orden de borrado respetando las dependencias de FOREIGN KEY (hijas antes que
padres)
DELETE FROM recomendacion_ia;
DELETE FROM umbral_alerta;
DELETE FROM consumo;
DELETE FROM conector;
DELETE FROM usuario;
DELETE FROM servicio;
```

```
DELETE FROM herramienta;
DELETE FROM tribu;
DELETE FROM rol_permiso;

SET SQL_SAFE_UPDATES = 1;

SELECT * FROM recomendacion_ia;
SELECT * FROM umbral_alerta;
SELECT * FROM consumo;
SELECT * FROM conector;
SELECT * FROM usuario;
SELECT * FROM servicio;
SELECT * FROM herramienta;
SELECT * FROM tribu;
SELECT * FROM rol_permiso;
```

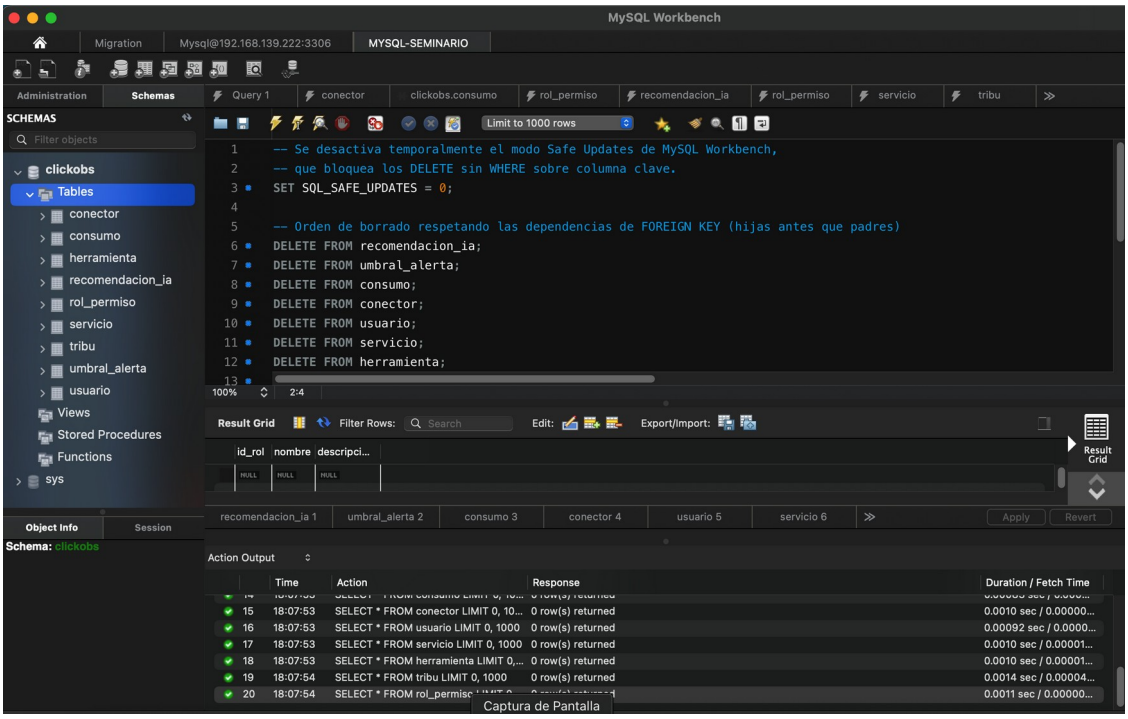


Figura 19. Evidencia de ejecución: borrado y verificación

19.5 Presentación de las consultas SQL

Los cuatro scripts presentados en esta sección (CreacionTablas_CLICKOBS.sql, Insert_CLICKOBS.sql, Consulta_CLICKOBS.sql y Borrado_CLICKOBS.sql) también se adjuntan como archivos .sql independientes en el repositorio de GitHub del proyecto (sección 21), listos para ejecutarse en cualquier entorno de desarrollo integrado (IDE) de MySQL, tal como se hizo en el entorno de prueba de esta entrega.

20. Requerimientos de comunicación del sistema

Se definen a continuación cómo se establecen las interacciones entre los componentes de CLICKOBS y qué protocolos y estándares resultan adecuados para la comunicación con las herramientas de observabilidad, con OpenRouter y con la base de datos.

20.1 Interacciones entre componentes

En la arquitectura objetivo (sección 8.3), el cliente JavaFX solo se comunica con el backend y nunca accede directamente a la base de datos. El backend es el único componente que accede a MySQL, que consulta las herramientas self-hosted de la red interna y que sale hacia servicios externos (APIs SaaS de Dynatrace y Datadog, y OpenRouter). En el prototipo (sección 16.1), el cliente se conecta directamente a MySQL por JDBC dentro de la red local.

20.2 Protocolos y estándares de comunicación

ID	Interacción	Protocolo / estándar	Justificación
RC-01	Cliente JavaFX ↔ Backend	HTTPS (TLS 1.2 o superior), REST/JSON, autenticación por token	Confidencialidad de credenciales y de datos de costo en la red interna
RC-02	Backend ↔ MySQL	JDBC sobre TLS (puerto 3306)	Estándar de conectividad entre Java y la base de datos; en la arquitectura objetivo el cliente no accede a la base
RC-03	Backend ↔ Dynatrace y Datadog (SaaS)	HTTPS, REST/JSON, API token; salida por firewall/proxy con lista de destinos permitidos	Los proveedores exponen su API de uso y costo solo sobre HTTPS
RC-04	Backend ↔ Grafana/Prometheus y Elastic (self-hosted)	HTTP(S), REST, dentro de la red interna	Herramientas propias: el tráfico no atraviesa Internet
RC-05	Backend ↔ OpenRouter	HTTPS, REST/JSON, API key; solo métricas agregadas y anonimizadas	API de terceros: se minimizan los datos enviados y se respeta la residencia de datos
RC-06	Resolución de nombres	DNS interno y público	Necesario para ubicar cada endpoint
RC-07	Segmentación de red	VLAN (IEEE 802.1Q) y firewall entre estaciones, servidores y segmento del Core Bancario	Ley N.° 25.326 y política interna: aislar los datos del Core Bancario

20.3 Diagrama de despliegue y comunicaciones

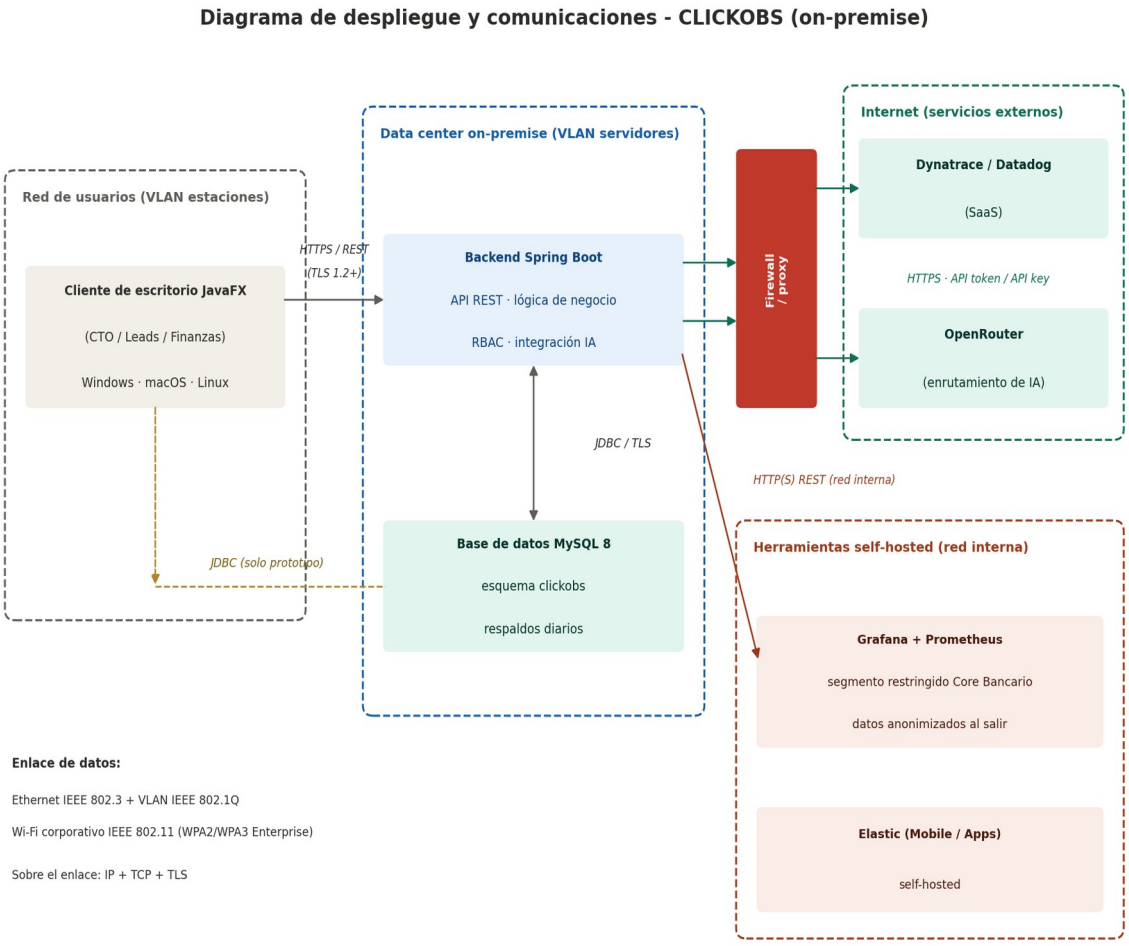


Figura 20. Diagrama de despliegue y comunicaciones de CLICKOBS

20.4 Entorno de red, infraestructura física y control de enlace de datos

- **Entorno de red:** LAN corporativa segmentada en tres VLAN: estaciones de trabajo, servidores (backend y MySQL) y segmento restringido del Core Bancario (Grafana/Prometheus). Un firewall/proxy perimetral controla la salida hacia Internet, permitida solo desde el backend hacia los destinos de las APIs SaaS y de OpenRouter.
- **Infraestructura física:** servidores en el data center corporativo (o un servidor virtualizado con dos máquinas virtuales) para el backend Spring Boot y para MySQL; switches administrables con soporte de VLAN; firewall perimetral; y estaciones de trabajo (Windows, macOS o Linux) con el cliente JavaFX instalado. Para cumplir el RNF-02 (disponibilidad igual o superior al 99% en horario laboral) se prevén respaldos diarios y un procedimiento de restauración documentado.
- **Control de enlace de datos:** en la capa 2 del modelo OSI, estaciones y servidores se conectan por Ethernet (IEEE 802.3) sobre switches con VLAN (IEEE 802.1Q); las estaciones inalámbricas usan Wi-Fi corporativo (IEEE 802.11) con WPA2/WPA3 Enterprise. La detección de errores de trama se resuelve con la secuencia de verificación de trama (FCS/CRC) propia de Ethernet, y el direccionamiento local, con direcciones MAC (resolución IP–MAC mediante ARP). Sobre este enlace operan IP y TCP, y encima TLS para cifrar las comunicaciones.

21. Repositorio del proyecto

Para la persistencia de los datos se utiliza una base de datos MySQL y, para el desarrollo del prototipo, una IDE de Java (IntelliJ IDEA o Eclipse) con JavaFX o Swing para la interfaz gráfica, tal como se detalla en la sección 16. El código fuente completo del prototipo, los cuatro scripts SQL (creación, inserción, consulta y borrado), las imágenes de los diagramas y las infografías utilizadas en este documento se publican, en su versión completa, en el siguiente repositorio de GitHub, bajo la cuenta personal del autor:

Repositorio: https://github.com/jorgejcorrea/clickobs

El repositorio se actualiza de forma incremental conforme avanzan las fases de Construcción y Transición del PUD, incorporando el backend Spring Boot, la orquestación de ingestas y las integraciones reales previstas en la arquitectura objetivo (sección 8.3). El paso a paso para su publicación se detalla en el Anexo A.

PARTE III — FASE DE CONSTRUCCIÓN (TP3)

22. Desarrollo del prototipo en Java/JavaFX

Este capítulo documenta cómo se construyó el programa funcional de CLICKOBS a partir del diseño de las partes I y II. No reproduce el código (completo en el repositorio, sección 22.11), sino que explica cómo se resolvió cada requisito del proyecto y por qué se tomó cada decisión.

22.1 Alcance: producto mínimo viable

El programa cubre el núcleo del sistema: los cuatro casos de uso de mayor valor para el negocio (CU-01 a CU-04) más las pantallas de apoyo que figuran en el menú del mockup de la sección 16.3. Se dejaron fuera, de forma deliberada, los elementos de la arquitectura objetivo que no son necesarios para validar el núcleo del sistema (backend Spring Boot, llamadas reales a OpenRouter y orquestación con Airflow dentro de la aplicación).

Pantalla	Caso de uso / requisito	Estado en el prototipo
Iniciar sesión	CU-01, RF-03	Implementada. Valida que el usuario exista en la base y asigna su rol. La verificación de la contraseña queda para la siguiente etapa (el campo todavía no se valida).
Comparar costos	CU-02, RF-17	Implementada: filtro por período, ordenamiento, ubicación de una herramienta y exportación a CSV.
Recomendaciones IA	CU-03, RF-14	Implementada con una IA simulada detrás de una interfaz. El ahorro se calcula de forma determinística (SQL + Java); la IA solo redacta.
Alertas	CU-04, RF-15 y RF-16	Implementada: umbrales de gasto por herramienta, evaluación de un período y cola de alertas pendientes.
Consumo	Consulta de consumo por herramienta (rol DevOps)	Implementada. El costo lo calcula el objeto de cada herramienta.
Dashboard	Vista gráfica ejecutiva (no estaba en los mockups originales)	Implementada como pantalla adicional: KPIs, barras por herramienta, dona por tribu y alertas.
Administración	Conectores de ingesta	Parcial: activar o desactivar conectores. La ingesta real corresponde al DAG de Airflow de la carpeta airflow/.

22.2 Organización del código y decisión de capas

El código se organizó en capas con una regla simple: la interfaz nunca habla con la base de datos. La Figura 21 muestra las clases realmente implementadas (46 archivos Java, unas 2.700 líneas).

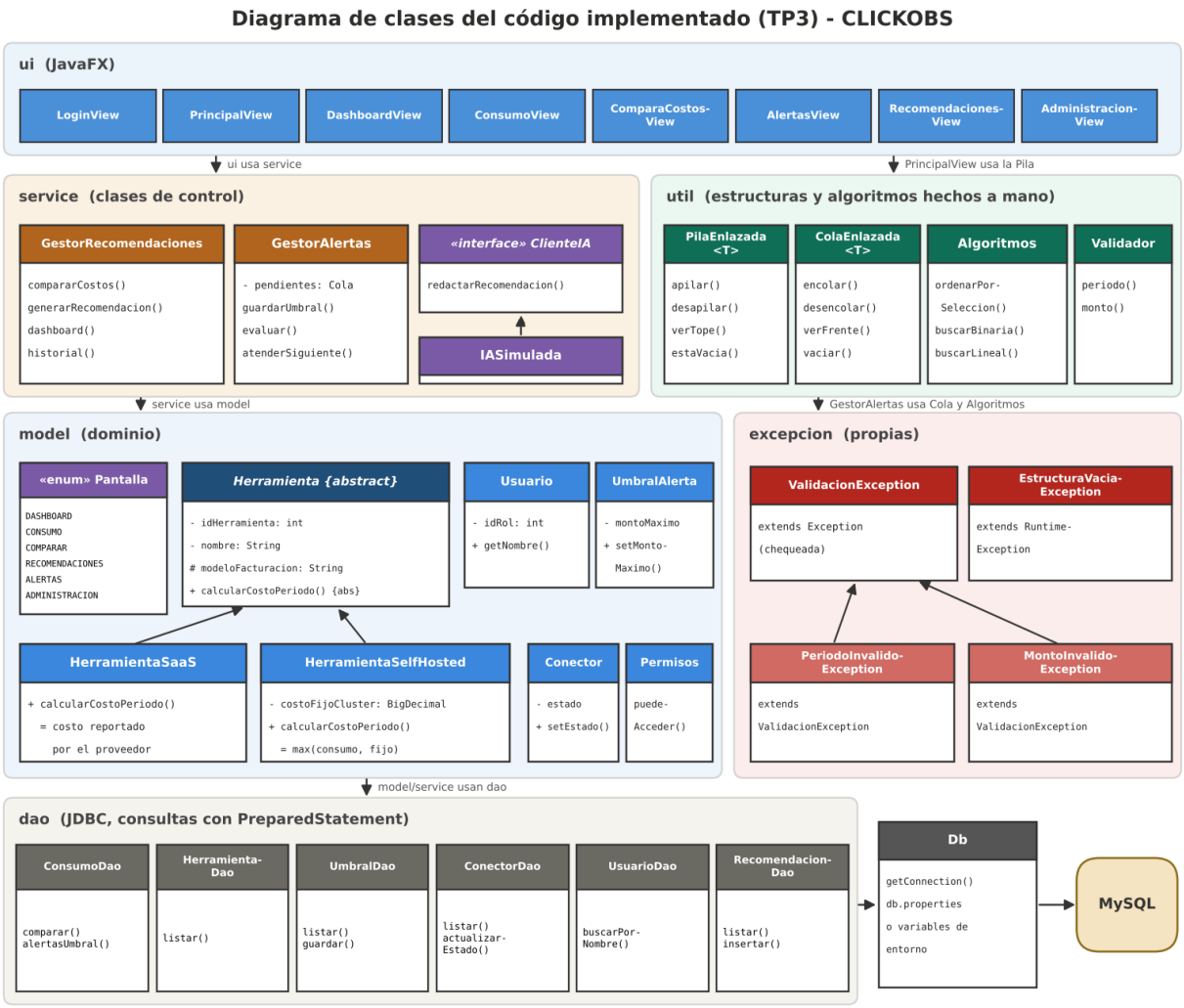


Figura 21. Diagrama de clases del código implementado (TP3)

Nota de la versión final (TP4). En el TP4 se agregó el paquete controller entre ui y service, y las pantallas dejaron de llamar directamente a los DAO y a los gestores: el diagrama de esa estructura es la Figura 39 (sección 23.5.1). Las clases nuevas de acceso a datos (Dao, DaoBase, PersistenciaException) figuran en la Figura 38.

- **ui (JavaFX):** una clase por pantalla. Muestran datos y capturan eventos; no contienen SQL ni reglas de negocio.
- **service:** clases de control (GestorRecomendaciones, GestorAlertas) que orquestan cada caso de uso, igual que la clase de control del diagrama de secuencia (sección 14).
- **model:** clases del dominio y reglas de acceso por rol (Permisos).
- **dao y db:** única capa que usa JDBC. Todas las consultas usan PreparedStatement con parámetros, para evitar la inyección de SQL, y las credenciales se leen de un archivo local o de variables de entorno, nunca del código (RNF-03).
- **util y excepcion:** estructuras de datos propias, validaciones y excepciones del sistema.

Se integró JDBC desde esta etapa para mantener el código simple (por ejemplo, llenar las listas directamente desde la base), por lo que el prototipo lee directamente de MySQL. Todo acceso a la base se ejecuta en un hilo aparte (clase Async) para que la ventana no se congele mientras se consulta. **Cambios respecto del diagrama de la sección 15:** los montos pasaron de double a

BigDecimal (los números binarios no representan bien los centavos); el atributo costoInfraestructuraMensual de la herramienta self-hosted pasó a llamarse costoFijoClusterMensual y se usa de verdad en el cálculo; y apiEndpoint y apiToken salieron de las clases, porque el endpoint vive en la tabla conector y el token nunca debe estar en el código.

22.3 Los cuatro pilares de la programación orientada a objetos

Pilar	Dónde se aplica	Por qué se decidió así
Encapsulamiento	Atributos private en Herramienta, Usuario, UmbralAlerta y Conector, accesibles por getters. Setters con validación (UmbralAlerta.setMontoMaximo, Conector.setEstado). Único atributo protected: modeloFacturacion, que usan las subclases.	Nadie puede dejar un umbral en cero ni un conector en un estado inventado: la única puerta de entrada es el setter, que valida. Protected solo se usa donde la herencia lo necesita.
Herencia	HerramientaSaaS y HerramientaSelfHosted extienden a Herramienta. PeriodoInvalidoException y MontoInvalidoException extienden a ValidacionException.	Las dos clases de herramienta comparten identidad, nombre y proveedor y difieren en cómo calculan el costo. En las excepciones se reutiliza el mensaje y se permite capturar ambas como ValidacionException.
Polimorfismo	calcularCostoPeriodo() se sobreescribe: la herramienta SaaS devuelve lo que reporta el proveedor; la self-hosted nunca baja del costo fijo de su cluster si hubo consumo. También se sobreescriben getTipo() y describirCosto(). Además, IASimulada implementa ClienteIA.	GestorRecomendaciones y la pantalla Consumo trabajan con una lista de Herramienta y llaman al mismo método sin saber la subclase. Al agregar una herramienta nueva no se toca la lógica de orquestación. Con la interfaz, cambiar la IA simulada por OpenRouter no afecta al resto.
Abstracción	Herramienta es abstracta con tres métodos abstractos (calcularCostoPeriodo, getTipo, describirCosto). ClienteIA es una interfaz.	Herramienta define qué debe saber hacer toda herramienta de observabilidad, sin decir cómo. No tiene sentido instanciar una «herramienta genérica», por eso es abstracta.

Constructores. Ninguna clase del dominio usa el constructor por defecto. Herramienta rechaza un id no positivo o un nombre vacío (IllegalArgumentException); HerramientaSelfHosted tiene dos constructores (uno con el costo fijo del cluster y otro sobrecargado que lo toma en cero); UmbralAlerta valida el monto al construirse. **Registros (record):** los objetos que solo transportan el resultado de una consulta (ComparacionFila, AlertaUmbral, Solapamiento, DatosDashboard) son records de Java: inmutables, sin lógica de validación propia. Donde hay reglas que proteger se usan clases comunes con atributos privados.

22.4 Estructuras de datos y algoritmos

Son opcionales para este problema, por lo que se aplicaron solo donde el caso de uso los pide naturalmente. Se escribieron a mano (sin Stack, Deque, Collections.sort ni binarySearch) para controlar y poder explicar su comportamiento.

Estructura	Dónde	Para qué y por qué	Costo
Pila enlazada (LIFO)	PrincipalView	Historial de pantallas: al navegar se apila la pantalla actual y el botón «Volver» desapila. La última pantalla visitada es la primera a la que se vuelve.	O(1) apilar y desapilar
Cola enlazada (FIFO)	GestorAlertas	Alertas pendientes: se atienden en el orden en que están en la cola. Antes de encolar se ordenan de mayor a menor exceso sobre el umbral, así la primera es la más grave.	O(1) encolar y desencolar (puntero al final)
Ordenamiento por selección	Comparar costos, GestorAlertas	Ordena la tabla por herramienta o por costo. Se eligió por simple: son decenas de filas, no millones. Devuelve una copia, la lista original no cambia.	$O(n^2)$
Búsqueda binaria	Comparar costos («Ubicar»)	Ubica una herramienta por nombre. Exige lista ordenada, por eso primero se ordena una copia por nombre. Devuelve la primera coincidencia para poder contar las repetidas.	O(log n)
Búsqueda lineal	Comparar costos	Encuentra la fila encontrada dentro de la tabla visible, que puede estar ordenada por costo y no por nombre (la binaria no sirve ahí).	O(n)

No se reemplazó todo por estructuras propias: ArrayList y LinkedHashMap se siguen usando para los resultados de las consultas, porque ahí una estructura propia no aporta nada. La evidencia de uso está en las capturas: la cola y su orden en la Figura 24, el ordenamiento y la búsqueda binaria en la Figura 26 («1 fila(s) de Datadog (búsqueda binaria)») y la pila en el pie de cada pantalla, que cuenta las pantallas apiladas en el historial.

22.5 Manejo de excepciones

El criterio general es que un error del usuario nunca debe romper el programa y un error de programación debe hacerse notar cuanto antes.

Excepción	Dónde se lanza / captura	Decisión y justificación
NumberFormatException	Se captura dentro de Validador.monto, con try-catch	Es el caso de «ingreso numérico»: si el usuario escribe «abc» al configurar un umbral, la conversión a BigDecimal falla. Se captura esa excepción específica (no Exception) y se traduce a un mensaje claro. Capturar Exception hubiera escondido errores de programación.
MontoInvalidoException (chequeada)	Se lanza en Validador.monto y UmbralAlerta; se captura en la pantalla Alertas	Cubre texto vacío, monto cero o negativo y tope de USD 10.000.000. Es chequeada a propósito: el compilador obliga a quien llama a decidir qué mostrarle al usuario. El mensaje aparece en rojo y el umbral no se modifica (Figura 25).

Excepción	Dónde se lanza / captura	Decisión y justificación
PeriodoInvalidoException (chequeada)	Se lanza en Validador.periodo; se captura en el manejador de fallos de Async	Valida el formato AAAA-MM con una expresión regular antes de armar una consulta. Se muestra como advertencia (Figura 27), distinta de un error del sistema.
SQLException	Se lanza en los DAO; se captura en Async	La interfaz no se cae si la base no responde: muestra un mensaje. Los recursos JDBC se cierran con try-with-resources.
EstructuraVaciaException (no chequeada)	Se lanza al desapilar o desencolar en vacío; no se captura	Es un error de programación, no del usuario. La interfaz siempre pregunta antes si hay elementos (por ejemplo, atenderSiguiente devuelve null si la cola está vacía).
IllegalArgumentException	Se lanza en los constructores del dominio; no se captura	Un objeto mal construido es un defecto del código: es mejor que falle en el acto y no más adelante con datos inconsistentes.

22.6 Tipos de datos y estructuras de control

Se usaron distintos tipos de datos según la naturaleza de cada dato: BigDecimal con dos decimales y redondeo HALF_UP para dinero (con double, 0,1 + 0,2 no da 0,3); int para identificadores; String para el período AAAA-MM; boolean para banderas como «sin datos»; LocalDateTime para la fecha de una recomendación; Integer (que admite null) para la herramienta asociada, que puede no existir; y el enum Pantalla para las pantallas navegables. Las estructuras condicionales están en las reglas de negocio (if/else en el cálculo del costo fijo, switch sobre el enum en Permisos y en la creación de pantallas) y las repetitivas en los algoritmos (for anidados en el ordenamiento, while en la búsqueda binaria y al recorrer los resultados de JDBC, for-each para encolar alertas).

22.7 Interfaz de usuario y coherencia con los mockups

La interfaz programada respeta los mockups de la sección 16.3: la misma barra de menú (Archivo, Consultas, Recomendaciones, Administración, Ayuda), la misma barra lateral y las mismas pantallas de inicio de sesión, comparación de costos y recomendaciones. Al programar se hicieron cuatro cambios, por lo que este capítulo reemplaza a los mockups como diseño de referencia:

Cambio respecto del mockup	Motivo
Se agregó «Dashboard» arriba de la barra lateral	Vista gráfica ejecutiva pedida durante el desarrollo; es la pantalla inicial de CTO, SRE Lead y Finanzas.
Botón «← Volver» en todas las pantallas	Usa la pila de navegación (sección 22.4).
En Comparar costos: selector «Ordenar por» y campo «Ubicar»	Necesarios para aplicar el ordenamiento y la búsqueda sobre la tabla.
Pantallas nuevas: Alertas (CU-04), Consumo y Administración	Figuraban en el menú de los mockups pero no tenían pantalla diseñada. Se construyeron con el mismo estilo.

Los accesos están restringidos por rol (RF-04): el CTO ve Dashboard, Consumo, Recomendaciones IA y Alertas; el SRE Lead, Dashboard, Consumo, Comparar costos y Administración; Finanzas, Dashboard y Alertas; y DevOps solo Consumo. Si un rol entra a una pantalla que no le corresponde, ve un mensaje de acceso denegado.



Figura 22. Inicio de sesión: el prototipo valida que el usuario exista en la base de datos

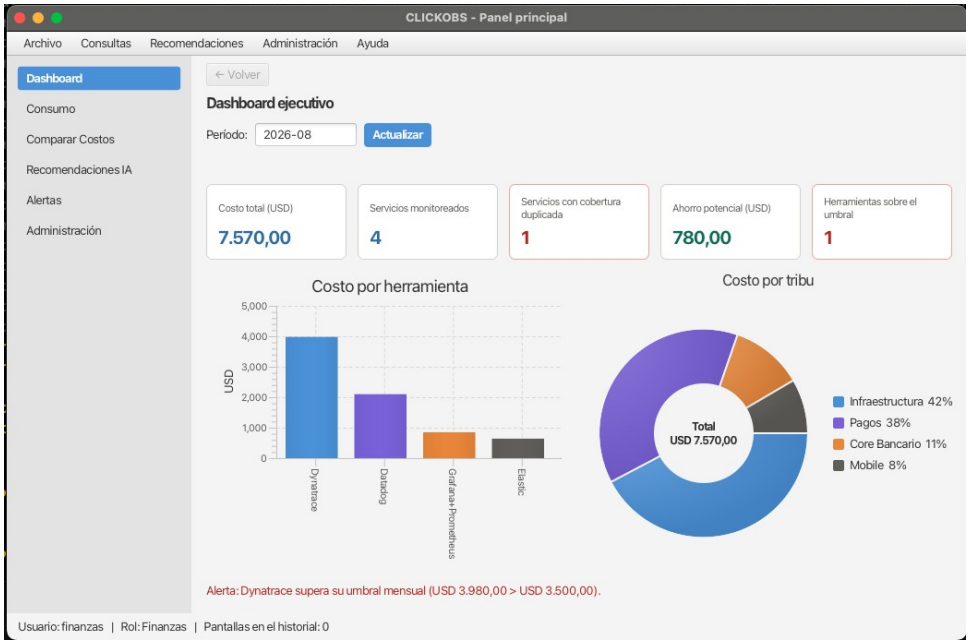


Figura 23. Dashboard ejecutivo del período 2026-08: indicadores, costo por herramienta y por tribu, y alerta de umbral

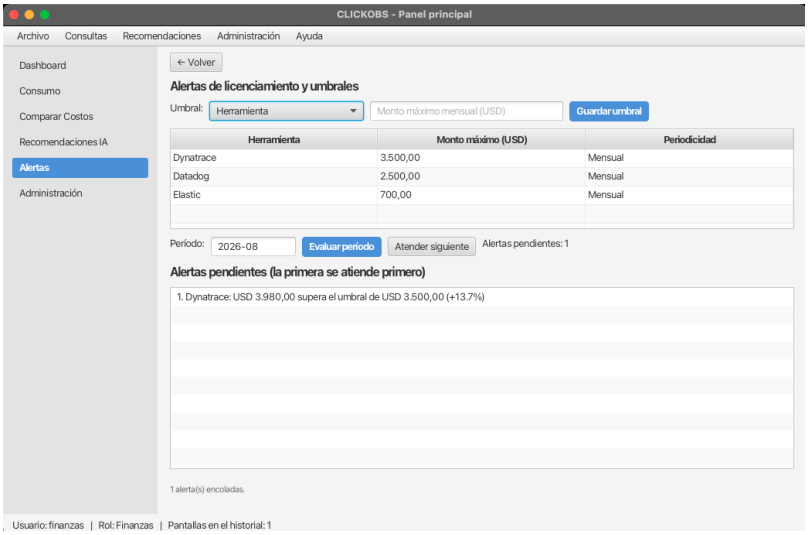


Figura 24. Pantalla Alertas (CU-04): umbrales y cola de alertas pendientes, ordenada por gravedad

Alertas (CU-04). El usuario de Finanzas configura el umbral de cada herramienta, evalúa un período y obtiene las alertas en una cola: Dynatrace supera su umbral en 13,7 % y Datadog en 5,0 %, por eso Dynatrace está primero. «Atender siguiente» saca la primera de la cola.

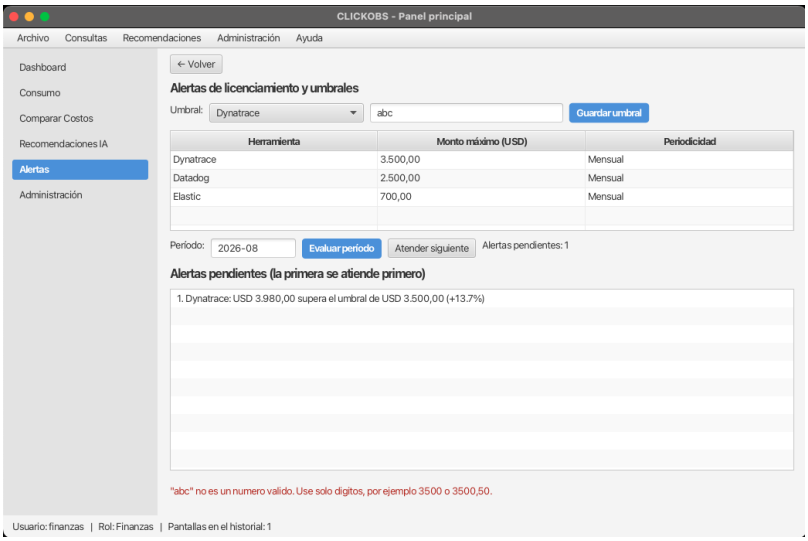


Figura 25. Validación del ingreso numérico: «abc» no es un monto válido y el umbral no cambia

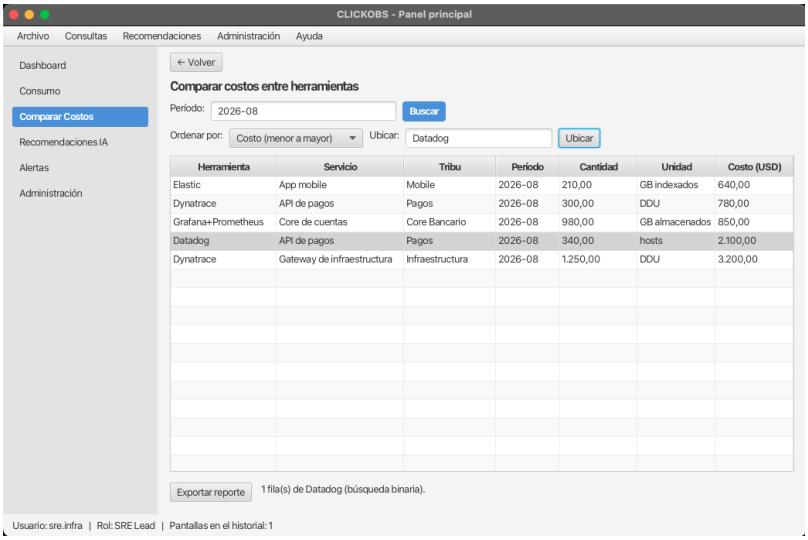


Figura 26. Comparar costos (CU-02): tabla ordenada por costo y búsqueda binaria de «Datadog»

Comparar costos (CU-02). El SRE Lead consulta un período; la tabla se ordena con el algoritmo de selección y la herramienta escrita se ubica con búsqueda binaria. El botón «Exportar reporte» genera un CSV compatible con Excel (RF-17).

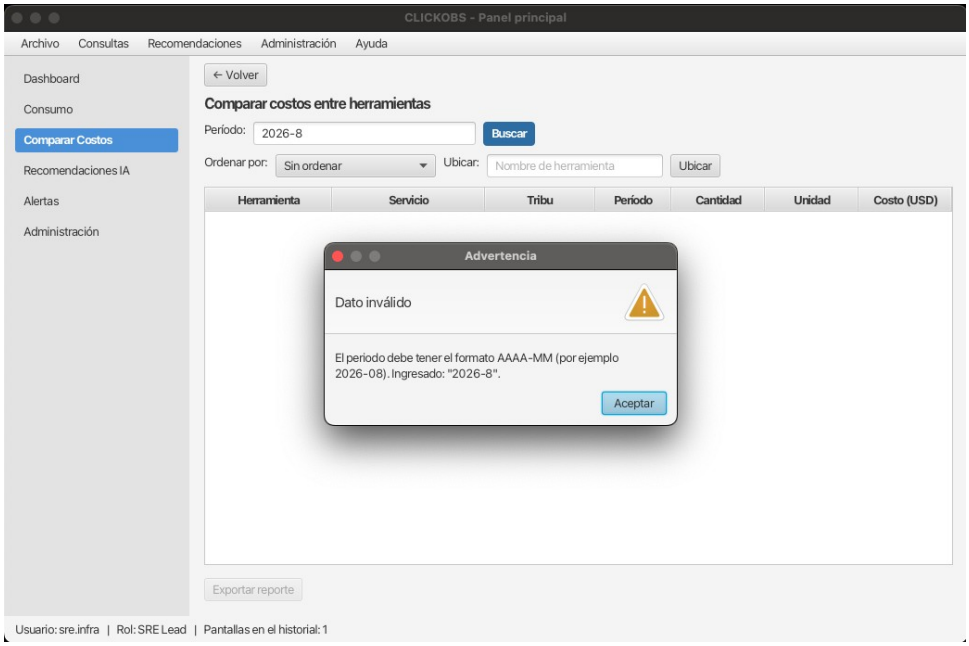


Figura 27. Período mal escrito («2026-8»): advertencia en lugar de error del sistema

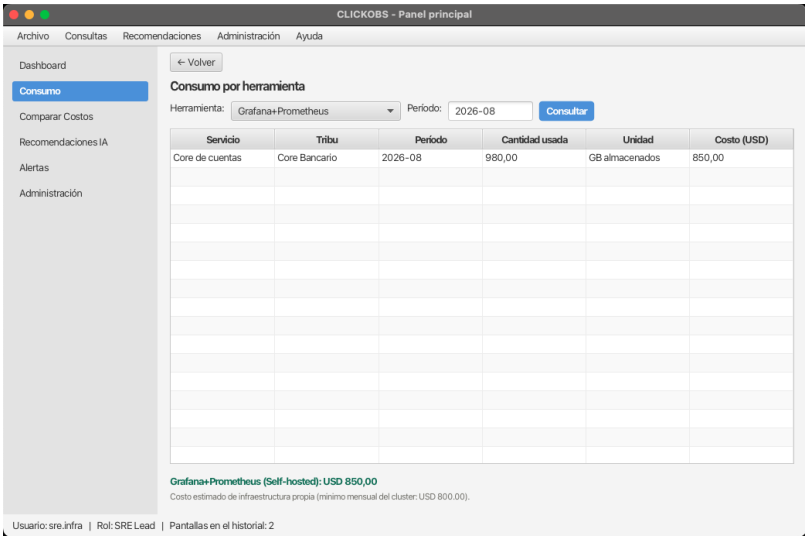


Figura 28. Consumo: herramienta self-hosted; el texto inferior lo aporta el objeto HerramientaSelfHosted

Consumo. Al elegir Grafana+Prometheus el pie dice «Self-hosted» y aclara que es un costo estimado con un mínimo mensual de cluster; con Dynatrace diría que es un costo reportado por el proveedor. Es el polimorfismo visible para el usuario.

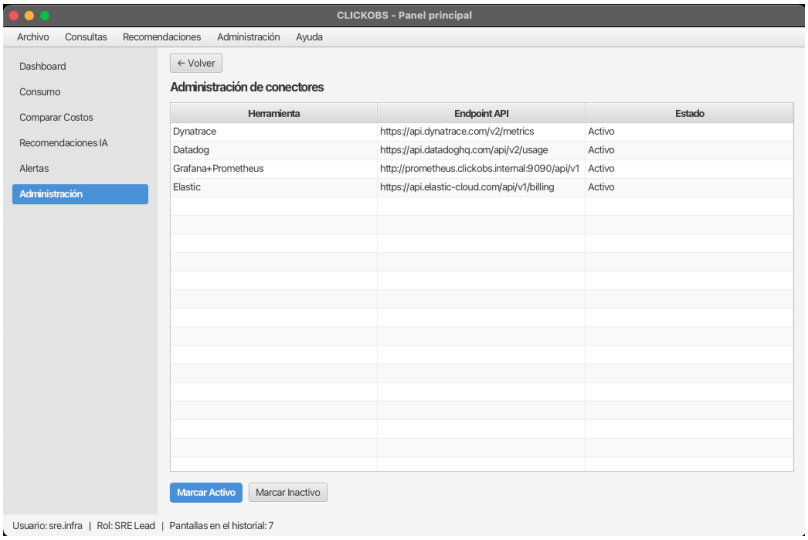


Figura 29. Administración: estado de los conectores (el pie muestra 7 pantallas apiladas en el historial de navegación)

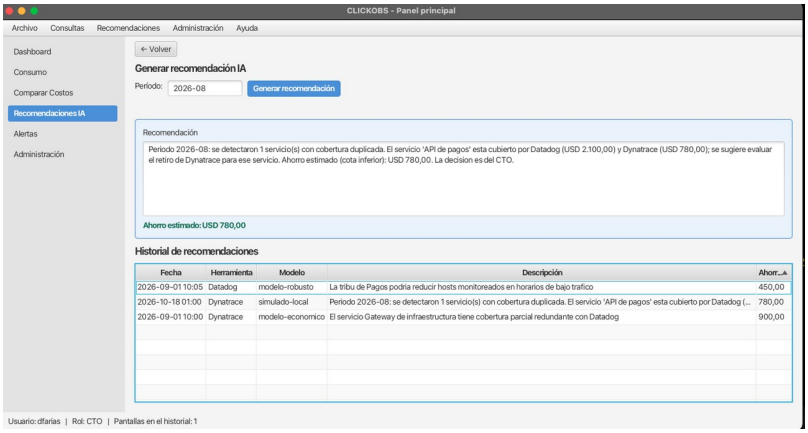


Figura 30. Recomendaciones IA (CU-03): narrativa generada para 2026-08, ahorro calculado e historial

22.8 Pruebas

Se mantiene el plan de pruebas de la sección 17 (caja negra y caja blanca, tratamiento de defectos y evaluación). Como el alcance del proyecto no requiere un framework de pruebas, los casos se escribieron como un programa ejecutable sin dependencias (carpeta pruebas/ del repositorio) que imprime OK o FALLO por cada caso. Los casos de caja negra parten de clases de equivalencia y valores límite (por ejemplo, el monto 0 y el monto 10.000.000 como límites); los de caja blanca recorren las ramas de las estructuras propias (pila y cola vacías, vaciadas y reutilizadas, búsqueda con y sin coincidencia).

Grupo de casos	Cantidad	Resultado
Validador: período y monto (caja negra)	16	16 OK
Pila y cola (caja blanca)	11	11 OK
Ordenamiento y búsqueda	12	12 OK
Modelo: encapsulamiento, constructores, polimorfismo y permisos por rol	16	16 OK
Con base de datos: costos, alertas, conectores y recomendación	22	22 OK
Total	**77**	**77 OK, 0 FALLO**

Defectos encontrados durante el desarrollo y su corrección: (1) el mensaje de validación se veía en gris porque una regla de estilo pisaba a la del error; (2) el diálogo de período inválido cortaba el texto; (3) la pantalla Administración perdía la fila elegida al actualizar; y (4) en una revisión del código se detectó que las dos subclases de Herramienta ejecutaban la misma consulta, lo que no era polimorfismo real: se rediseñó el cálculo de la self-hosted. **Entorno de la corrida:** JDK 25 (compilando con --release 21), JavaFX 21 y la base creada con los mismos scripts del proyecto (sección 19) sobre el servidor MySQL 8.0 del proyecto (en Docker, accedido por red). Durante el desarrollo las pruebas se corrieron también sobre MariaDB 10.11 con el mismo resultado. La Figura 31 muestra la salida final de la corrida contra MySQL 8.0.

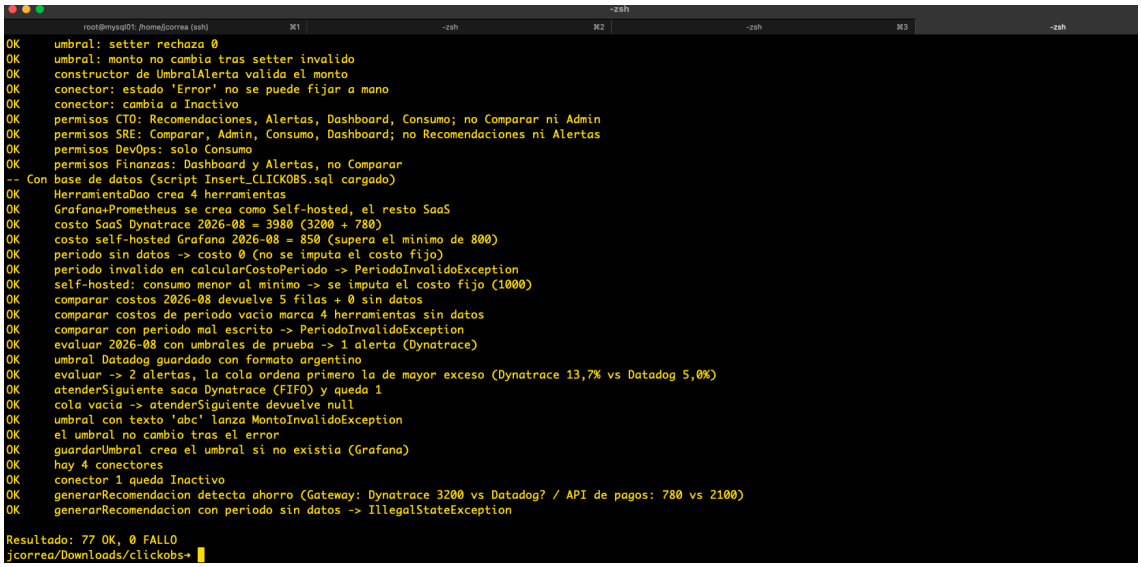


Figura 31. Salida de las 77 pruebas ejecutadas contra el servidor MySQL 8.0 del proyecto: 77 OK, 0 FALLO

22.9 Decisiones y herramientas

- **JavaFX en lugar de Swing:** requisito del proyecto y mejor soporte de tablas y gráficos. La interfaz se arma por código y no con FXML para tener la lógica de cada pantalla en un solo archivo, más simple de explicar.
- **Pila y cola propias en lugar de ArrayDeque:** para tener control total de su comportamiento; se aceptó el costo de mantener código extra.
- **JUnit:** no se exigió; se priorizó la documentación teórica de pruebas y un programa de casos sin dependencias.
- **Spring Boot, OpenRouter y Airflow integrados a la aplicación:** quedaron fuera del producto mínimo viable. Airflow existe como DAG probado en la carpeta airflow/ del repositorio, pero la aplicación todavía lee de la base directamente.
- **Verificación de contraseña y administración de usuarios:** fuera del alcance de esta etapa; es una limitación conocida y declarada.

22.10 Trazabilidad con las entregas anteriores

Los ajustes de las fases anteriores están integrados en el documento: el detalle de clases en el diagrama de secuencia (sección 14), los atributos con mejoras en el diagrama de clases (sección 15), el plan de pruebas con casos de caja blanca y caja negra, los defectos y la evaluación de pruebas (sección 17), los criterios de diseño (sección 12) y el repositorio en GitHub (sección 21). Los ajustes pendientes de la revisión se incorporaron en la versión final integradora (sección 23.2).

22.11 Repositorio y ejecución

Repositorio: https://github.com/jorgejcorrea/clickobs (carpetas src/, db/, airflow/, pruebas/ y docs/). Para ejecutarlo se necesita JDK 21 y Maven: crear la base con los scripts de db/, copiar db.properties.example como db.properties completando usuario y clave (el archivo está en .gitignore y no se sube) y ejecutar «mvn javafx:run».

Usuario de prueba	Rol	Pantallas a las que accede
dfarias	CTO	Dashboard, Consumo, Recomendaciones IA, Alertas
sre.infra	SRE Lead	Dashboard, Consumo, Comparar costos, Administración
devops.pagos	DevOps	Consumo
finanzas	Finanzas	Dashboard, Alertas

22.12 Fragmentos de código de referencia

Se muestran fragmentos breves y reales del código (con su número de línea en el archivo) como referencia de lo explicado en este capítulo; debajo de cada uno se resume la justificación, ampliada en la sección indicada.

```java
/** METODO ABSTRACTO: costo total (USD) de la herramienta en el periodo AAAA-MM. Cada subclase lo implementa distinto. */
public abstract BigDecimal calcularCostoPeriodo(String periodo) throws SQLException, ValidacionException;

/** Tipo legible de herramienta ("SaaS" o "Self-hosted"). */
public abstract String getTipo();

/** Aclaracion que se muestra junto al costo: de donde sale el numero. */
public abstract String describirCosto();
```

Figura 32. Herramienta.java: métodos abstractos de la clase abstracta

Justificación: no existe una «herramienta genérica»; la clase define qué debe saber hacer toda herramienta (calcular su costo, decir su tipo y de dónde sale el número) y cada subclase decide cómo (sección 22.3, abstracción).

```java
public BigDecimal calcularCostoPeriodo(String periodo) throws SQLException, ValidacionException {
    BigDecimal registrado = consumoDao.costoTotal(getIdHerramienta(), Validador.periodo(periodo));
    if (registrado.signum() > 0 && registrado.compareTo(costoFijoClusterMensual) < 0) {
        return costoFijoClusterMensual;
    }
    return registrado;
}
```

Figura 33. HerramientaSelfHosted.java: sobreescritura de calcularCostoPeriodo

Justificación: el mismo mensaje da un resultado distinto al de la herramienta SaaS: el cluster propio cuesta un mínimo mensual aunque se use poco. El piso se aplica solo si hubo consumo, para no inventar costo en períodos sin datos (sección 22.3, polimorfismo).

```java
BigDecimal valor;
try {
    valor = new BigDecimal(limpio);
} catch (NumberFormatException e) {
    throw new MontoInvalidoException("\"" + texto.trim() + "\" no es un numero valido. Use solo digitos, por ejemplo 3500 o 3500,50.", e);
}
```

Figura 34. Validador.java: try-catch específico para el ingreso numérico

Justificación: se captura solo NumberFormatException, la que produce un texto no numérico, y se traduce a una excepción propia con un mensaje que el usuario entiende. Capturar Exception habría escondido errores de programación (sección 22.5).

```java
public void apilar(T dato) {
    tope = new Nodo<>(dato, tope);
    tamanio++;
}

public T desapilar() {
    if (tope == null) {
        throw new EstructuraVaciaException("pila");
    }
    T dato = tope.dato;
    tope = tope.siguiente;
    tamanio--;
    return dato;
```

Figura 35. PilaEnlazada.java: apilar y desapilar

Justificación: la pila guarda el historial de pantallas; con una lista enlazada y un puntero al tope, ambas operaciones son O(1) y la pila vacía lanza una excepción propia (sección 22.4).

```java
public void encolar(T dato) {
    Nodo<T> nuevo = new Nodo<>(dato);
    if (fin == null) {
        frente = nuevo;
    } else {
        fin.siguiente = nuevo;
    }
    fin = nuevo;
    tamanio++;
}

public T desencolar() {
    if (frente == null) {
        throw new EstructuraVaciaException("cola");
    }
    T dato = frente.dato;
    frente = frente.siguiente;
    if (frente == null) {
        fin = null;
    }
    tamanio--;
```

Figura 36. ColaEnlazada.java: encolar y desencolar

Justificación: las alertas se atienden en orden de llegada (FIFO). El puntero al final evita recorrer la lista para encolar, y al vaciarse la cola se limpia también ese puntero para poder reutilizarla (sección 22.4).

```java
public static <T, K> int buscarBinaria(List<T> ordenada, K clave, Function<T, K> extraerClave,
                                       Comparator<? super K> comparador) {
    int bajo = 0;
    int alto = ordenada.size() - 1;
    int encontrado = -1;
    while (bajo <= alto) {
        int medio = (bajo + alto) >>> 1;
        int cmp = comparador.compare(extraerClave.apply(ordenada.get(medio)), clave);
        if (cmp == 0) {
            encontrado = medio;
            alto = medio - 1;   // sigue hacia la izquierda: puede haber una coincidencia anterior
        } else if (cmp < 0) {
            bajo = medio + 1;
        } else {
            alto = medio - 1;
        }
    }
    return encontrado;
```

Figura 37. Algoritmos.java: búsqueda binaria que devuelve la primera coincidencia

Justificación: exige una lista ordenada, por eso la pantalla ordena antes una copia por nombre. Al encontrar una coincidencia sigue hacia la izquierda para devolver la primera y poder contar las repetidas (sección 22.4).

22.13 Conclusión

El prototipo aplica los cuatro pilares de la POO sobre un caso real (herramientas con distinta forma de facturación), usa pila, cola, ordenamiento y búsqueda donde el problema los necesita, valida los ingresos del usuario sin romper el programa y se mantiene coherente con los mockups. Las limitaciones declaradas (contraseña, IA real, backend) son las que se abordan en la continuación del proyecto.

PARTE IV — VERSIÓN FINAL INTEGRADORA (TP4)

23. Versión final integradora del prototipo (TP4)

23.1 Alcance y entrega incremental

Esta entrega retoma el proyecto definido en las Partes I a III y avanza hacia la versión final integradora, manteniendo el enfoque incremental del Proceso Unificado de Desarrollo: no se reescribe lo ya entregado, se agregan los elementos que pide la consigna y se documentan las correcciones. El prototipo sigue siendo un modelo operacional con un subconjunto de las características del sistema final (Kendall y Kendall, 2011): cada módulo funciona y se integra en la versión final.

La tabla indica dónde se resuelve cada requisito de la consigna y qué archivo del repositorio lo evidencia.

Requisito de la consigna	Sección	Evidencia en el repositorio
Corrección de las observaciones de las entregas anteriores	23.2	Secciones 3, 12, 14, 15, 17, 18, 19 y 21 de este documento
Análisis del modelo de negocios que define y justifica el proyecto	23.3	Partes I y II; síntesis en 23.3
Aplicación del Proceso Unificado de Desarrollo	23.4	Fases e iteraciones; casos de uso CU-01 a CU-04
Selección del patrón de diseño y justificación	23.5	controller/*Controller.java, dao/Dao.java, dao/DaoBase.java, dao/*Dao.java
Persistencia y consulta en MySQL (conexión, consultas, actualización, resultados en la interfaz)	23.6	db/Db.java, dao/*, ui/*View.java, db/*.sql
Excepciones en la interacción con MySQL	23.7	excepcion/PersistenciaException.java
Clases abstractas o interfaces	23.8	model/Herramienta.java, dao/DaoBase.java, dao/Dao.java, service/ClienteIA.java
Arreglos y ArrayList de forma complementaria	23.9	model/ResumenAnual.java, dao/ConsumoDao.java, service/GestorRecomendaciones.java
Archivos para guardar y recuperar información	23.10	service/Bitacora.java, service/ExportadorCsv.java, db/Db.java
Pruebas, repositorio y ejecución	23.11 a 23.13	pruebas/PruebasClickobs.java, README.md

23.2 Corrección de las observaciones de las entregas anteriores

Las entregas TP1 y TP3 fueron calificadas con 100/100. Se relevaron las entregas anteriores y se detectaron mejoras, que se incorporaron al proyecto (sección 22.10); se resumen aquí para que el docente pueda verificarlas en un solo lugar.

Entrega	Mejora detectada en el relevamiento	Corrección y dónde se ve
TP1	Separar los fines de la organización, el objetivo general del proyecto y el objetivo del sistema	Reescritos en las secciones 3 y 4.1; los tres planos se presentan por separado
TP1	Definir una arquitectura realista para un prototipo de escritorio	Cliente JavaFX conectado a MySQL, desplegado on-premise (secciones 8, 16 y 20); la arquitectura objetivo se mantiene en la infografía de la sección 8.3
TP1 y TP2	Consumo y Costo tenían relación 1 a 1 y duplicaban el período, lo que obligaba a un JOIN extra	Unificadas en la tabla consumo con el campo monto_usd (sección 18)
TP2	Detalle de las clases en el diagrama de secuencia y atributos del diagrama de clases	Secciones 14 y 15
TP2	El diagrama de clases de diseño (sección 15) tenía elementos que se implementaron de otra forma: la clase Consumo, detectarSolapamientos(), registrar() y los montos en double	Figura 10 corregida en la sección 15: Consumo y Costo son una tabla accedida con ConsumoDao, y los montos son BigDecimal. Se agregan el controlador y la IA como interfaz
TP2	Plan de pruebas con casos de caja blanca y caja negra, defectos y evaluación	Sección 17 y sección 22.8
TP2	Integridad de la base de datos (restricciones faltantes)	Script opcional db/Endurecimiento_CLICKOBS.sql: UNIQUE, CHECK, NOT NULL e índices (sección 19)
TP3	Calificada con 100/100. Se conservan las limitaciones declaradas en 22.1 (contraseña, IA real y backend)	Se mantienen como alcance de la Construcción final y se justifican en 23.12

23.3 Análisis del modelo de negocios que justifica el proyecto

CLICKOBS S.A. es una fintech regional (LATAM) con cuatro tribus de ingeniería que adoptaron, cada una por su cuenta, una herramienta de observabilidad distinta: Dynatrace (SaaS), Datadog (SaaS), Grafana + Prometheus (self-hosted) y Elastic. Cada herramienta factura de una manera diferente, el área de Arquitectura estima que cerca del 35 % de los servicios están monitoreados por más de una herramienta y no existe una visión consolidada del gasto (secciones 1 y 2).

El proyecto se justifica por el valor que genera: **visibilidad centralizada** del costo, **detección de solapamientos** de cobertura, **alertas** de licenciamiento antes del cierre de facturación y **recomendaciones de racionalización** redactadas por IA sobre datos calculados de forma determinística. El objetivo general es reducir al menos un 20 % el gasto anual en licenciamiento de observabilidad dentro de los seis meses posteriores al inicio (sección 3.2).

El prototipo muestra esa lógica de negocio con los datos de prueba de agosto de 2026: el costo total es de USD 7.570, el sistema detecta **1 servicio con cobertura duplicada** y calcula un **ahorro potencial de USD 780** (alrededor del 10,3 % del total) con una consulta SQL, sin intervención de la IA, y salta una alerta porque Dynatrace supera su umbral mensual (USD 3.980 contra USD 3.500). El 10,3 % es un resultado parcial sobre un solo servicio y un solo mes: el 20 % anual requiere los períodos completos, que aportará la ingesta.

23.4 Aplicación del Proceso Unificado de Desarrollo

El Proceso Unificado de Desarrollo (PUD) es dirigido por casos de uso, centrado en la arquitectura e iterativo e incremental. Esta entrega lo aplica de la siguiente manera:

Fase del PUD	Entrega	Disciplinas y artefactos	Estado
Inicio	TP1	Alcance, actores, requerimientos, casos de uso y plan de proyecto (secciones 1 a 11)	Completa
Elaboración	TP2	Análisis, diseño, modelo de datos, scripts SQL, mockups y arquitectura (secciones 12 a 21)	Completa
Construcción	TP3	Iteración 1: núcleo orientado a objetos, estructuras de datos, algoritmos y validación (sección 22)	Completa, 77 pruebas
Construcción	TP4	Iteración 2: patrones MVC y DAO, persistencia en MySQL, excepciones de base de datos, arreglos con ArrayList y archivos (sección 23)	Esta entrega, 127 pruebas
Transición	Posterior	Piloto con la tribu de Infraestructura, ajustes según feedback y despliegue a las cuatro tribus	Pendiente

- **Dirigido por casos de uso:** cada pantalla implementa un caso de uso (CU-01 acceso, CU-02 comparación de costos, CU-03 recomendaciones, CU-04 alertas y umbrales) y las pruebas se organizan por caso de uso.
- **Centrado en la arquitectura:** el patrón elegido (23.5) respeta las capas definidas en la sección 16.1 y el RNF-06; ninguna pantalla importa dao, service ni db: habla solo con su controlador (23.5.1), regla que comprueba una prueba automática.
- **Iterativo e incremental:** cada iteración termina con un incremento ejecutable y probado; la iteración del TP4 no rompe las 77 pruebas del TP3, que se siguen ejecutando.

23.5 Selección del patrón de diseño y justificación

Patrones aplicados: MVC para la estructura general (23.5.1) y DAO (Data Access Object) para la persistencia. Un DAO encapsula todo el acceso a una tabla detrás de una interfaz, de modo que el resto del sistema trabaja con objetos del dominio y no con JDBC (Alur, Crupi y Malks, 2003). Se lo complementa con **Template Method** (Gamma et al., 1994) dentro de una clase base abstracta, y con una **fábrica simple** que ya existía en HerramientaDao.

El criterio de selección fue la coherencia con tres cosas: la arquitectura cliente-servidor en capas definida en la Parte II, el requerimiento RNF-06 (capas claramente separadas) y el problema concreto del TP4, que es conectar, consultar y actualizar MySQL desde varias pantallas sin repetir código.

Patrón candidato	Qué resuelve	Decisión y motivo
MVC (Modelo-Vista-Controlador)	Separa los datos, la presentación y el manejo de las acciones del usuario	**Aplicado** a toda la aplicación; las responsabilidades se identifican en 23.5.1

Patrón candidato	Qué resuelve	Decisión y motivo
DAO	Separa el acceso a datos de la lógica de negocio y de la interfaz	**Elegido.** Es exactamente el problema del TP4; encaja con la capa de datos; cambiar de motor solo afecta a los DAO
Template Method	Define el esqueleto de una operación y deja pasos variables a las subclases	**Complementario.** DaoBase fija los pasos de JDBC; cada DAO aporta solo el SQL y cómo armar el objeto
Fábrica simple	Crea el objeto correcto según un dato	**Ya aplicado.** HerramientaDao crea HerramientaSaaS o HerramientaSelfHosted según el modelo de facturación
Repository	Presenta los datos como una colección del dominio	Descartado: agrega una capa de agregados del dominio que no se justifica con nueve tablas
Active Record	Cada objeto se guarda a sí mismo	Descartado: mezcla el modelo con SQL y rompería la separación en capas
Singleton (conexión)	Una única instancia compartida	Descartado: la interfaz usa hilos (Async); compartir una conexión obliga a sincronizar. Se abre una conexión por operación
Adapter	Adapta interfaces incompatibles	Reservado para los conectores de las herramientas (RNF-08), que pertenecen a la ingesta con Airflow

Cómo está aplicado. La interfaz Dao<T> define el contrato (listar). La clase abstracta DaoBase implementa una sola vez los pasos repetitivos de JDBC: abrir la conexión, preparar la consulta, enlazar los parámetros, recorrer el resultado, cerrar los recursos y traducir los errores. Cada DAO concreto hereda de DaoBase y entrega solamente la sentencia SQL y una función que convierte una fila en un objeto. La Figura 38 muestra la estructura.

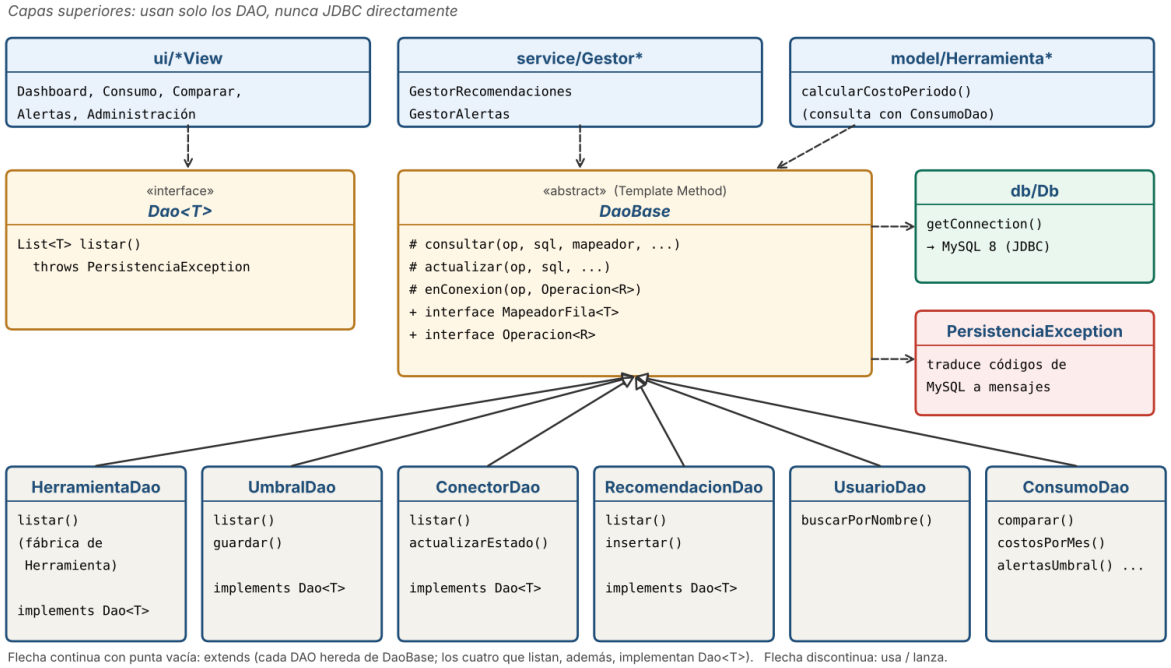


Figura 38. Patrón DAO en CLICKOBS: interfaz Dao<T>, clase abstracta DaoBase (Template Method), seis DAO concretos, Db y PersistenciaException

Fragmento 1. DaoBase: el esqueleto compartido (DaoBase.java, líneas 33 a 69)

```
    /** SELECT con parametros: devuelve las filas como un ArrayList de objetos del
dominio. */
    protected <T> ArrayList<T> consultar(String operacion, String sql, MapeadorFila<T>
mapeador, Object... parametros)
            throws PersistenciaException {
        return enConexion(operacion, c -> {
            try (PreparedStatement ps = c.prepareStatement(sql)) {
                enlazar(ps, parametros);
                try (ResultSet rs = ps.executeQuery()) {
                    ArrayList<T> filas = new ArrayList<>();
                    while (rs.next()) {
                        filas.add(mapeador.mapear(rs));
                    }
                    return filas;
                }
            }
        });
    }

    /** INSERT / UPDATE / DELETE con parametros: devuelve la cantidad de filas
afectadas. */
    protected int actualizar(String operacion, String sql, Object... parametros)
throws PersistenciaException {
        return enConexion(operacion, c -> {
            try (PreparedStatement ps = c.prepareStatement(sql)) {
                enlazar(ps, parametros);
                return ps.executeUpdate();
            }
        });
    }

    /** Abre una conexion, ejecuta el bloque, la cierra siempre y traduce cualquier
SQLException. */
    protected <R> R enConexion(String operacion, Operacion<R> trabajo) throws
PersistenciaException {
        try (Connection c = Db.getConnection()) {
            return trabajo.ejecutar(c);
        } catch (SQLException e) {
            throw PersistenciaException.de(operacion, e);
        }
    }
```

Fragmento 2. HerramientaDao: lo único que cambia en cada DAO (HerramientaDao.java, líneas 18 a 29)

```
    public List<Herramienta> listar() throws PersistenciaException {
        String sql = "SELECT id_herramienta, nombre, proveedor, modelo_facturacion
FROM herramienta ORDER BY id_herramienta";
        return consultar("consultar las herramientas", sql, rs -> {
            int id = rs.getInt("id_herramienta");
            String nombre = rs.getString("nombre");
            String proveedor = rs.getString("proveedor");
            String modelo = rs.getString("modelo_facturacion");
            if ("Infraestructura propia".equalsIgnoreCase(modelo)) {
                return new HerramientaSelfHosted(id, nombre, proveedor, modelo,
COSTO_FIJO_CLUSTER);
            }
            return new HerramientaSaaS(id, nombre, proveedor, modelo);
        });
    }
```

Resultado medible. Antes del TP4 cada método de los DAO abría y cerraba su propia conexión y repetía el manejo de SQLException. Ahora la conexión se abre en un único lugar del código de acceso a datos (DaoBase.enConexion()). Los seis DAO concretos pasaron de 435 a 367 líneas aun después de agregar la consulta por mes (costosPorMes); el código de infraestructura nuevo (DaoBase, Dao y PersistenciaException) suma 138 líneas que se escriben una sola vez.

Ventajas: el mismo código de acceso se reutiliza en todas las tablas, los errores se traducen en un único lugar, los DAO se pueden probar sin pantallas y un cambio de motor de base de datos afecta solo a esta capa. **Límites asumidos:** no hay pool de conexiones (se abre una por operación, razonable para un prototipo de escritorio) y las claves sin AUTO_INCREMENT se calculan con MAX + 1; ambas mejoras quedan previstas para la versión final.

23.5.1 Patrón Modelo-Vista-Controlador: responsabilidades y verificación

La aplicación sigue el patrón Modelo-Vista-Controlador (MVC): una solución conocida que separa lo que se muestra, lo que se decide y lo que se guarda. En el TP4 la estructura se completó con un paquete **controller**, de modo que cada pantalla habla únicamente con su controlador. La Figura 39 muestra el reparto de responsabilidades.

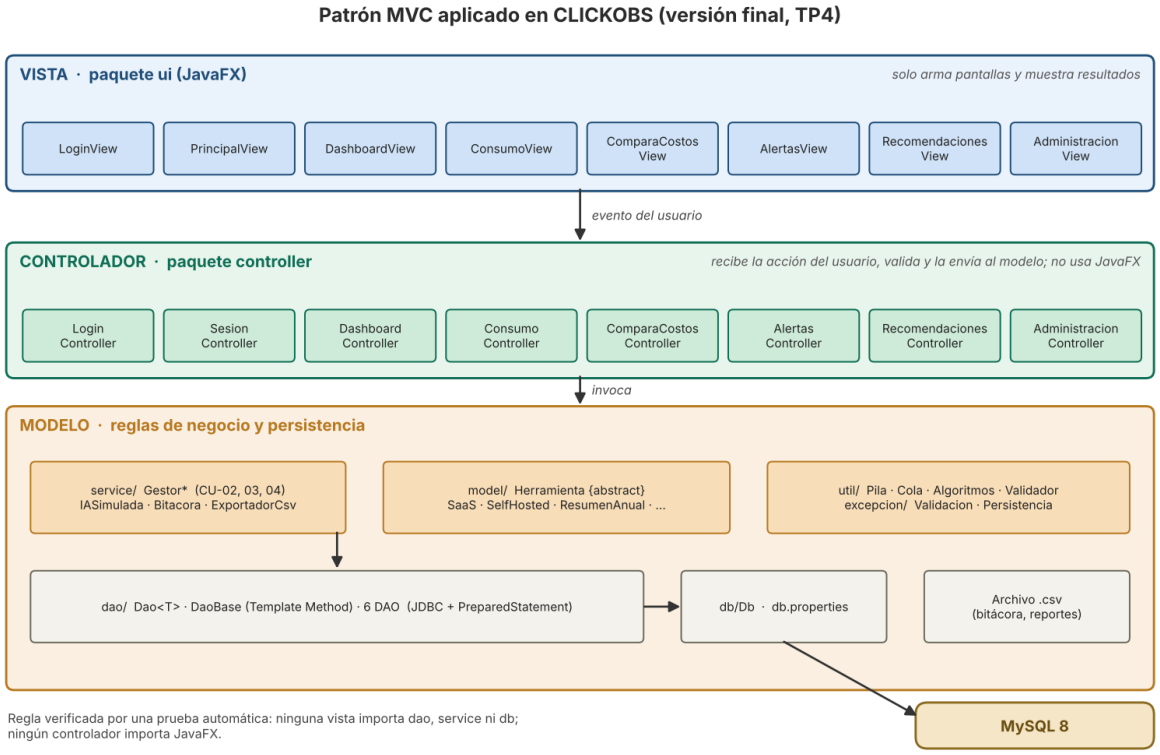


Figura 39. Patrón MVC en CLICKOBS: vistas (ui), controladores (controller) y modelo (service, model, util, dao, db)

Rol MVC	Dónde está en el código	Responsabilidad
Modelo	service/ (GestorAlertas, GestorRecomendaciones y otros), model/ (Herramienta, HerramientaSaaS, HerramientaSelfHosted, ResumenAnual y otras), util/, dao/ y MySQL	Los datos, las reglas de negocio (costo por herramienta, solapamientos, umbrales) y la persistencia. No conoce JavaFX ni a los controladores
Vista	ui/ (LoginView, PrincipalView, DashboardView, ConsumoView, ComparaCostosView, AlertasView, RecomendacionesView, AdministracionView)	Arma la pantalla, recibe los eventos del usuario y muestra los resultados o los errores. No calcula costos, no escribe SQL ni llama a los DAO

Rol MVC	Dónde está en el código	Responsabilidad
Controlador	controller/ (LoginController, SesionController, DashboardController, ConsumoController, ComparaCostosController, AlertasController, RecomendacionesController, AdministracionController)	Recibe la acción del usuario, la valida, la envía al modelo y devuelve un resultado listo para mostrar. No usa JavaFX

Ejemplo del recorrido completo (CU-04, guardar un umbral). El usuario escribe el monto y presiona «Guardar umbral» en AlertasView. El manejador del botón llama a AlertasController.guardarUmbral(); el controlador pasa la orden al GestorAlertas, que valida el monto con el Validador y llama a UmbralDao; el DAO ejecuta el UPDATE en MySQL. El resultado vuelve por el mismo camino y la vista actualiza la tabla. La vista ejecuta la llamada en un hilo aparte (Async) para no congelar la ventana.

Fragmento 3. Controlador: la vista no conoce al gestor ni a los DAO (AlertasController.java, líneas 30 a 33)

```
    /** Valida y guarda el umbral (UPDATE o INSERT en MySQL) y devuelve la lista
actualizada. */
    public List<UmbralAlerta> guardarUmbral(int idHerramienta, String montoTexto)
throws ValidacionException, PersistenciaException {
        gestor.guardarUmbral(idHerramienta, montoTexto);
        return gestor.listarUmbrales();
    }
```

Verificación de que se cumple. La separación no depende de la buena voluntad: dos pruebas automáticas la comprueban sobre el código fuente. La primera verifica que **ninguna clase de ui importe dao, service ni db**; la segunda, que **ningún controlador importe JavaFX**. Además, 16 pruebas ejercitan cada controlador contra MySQL (autenticar, detalle de consumo, guardar un umbral, evaluar y atender alertas, comparar y exportar a CSV, historial, cambiar el estado de un conector y registrar en la bitácora).

Decisiones. Los controladores se crean con sus colaboradores por defecto (new XController()), sin contenedor de inyección, porque el alcance excluye frameworks. AlertasController se crea una sola vez en la ventana principal y se comparte, porque la cola de alertas pendientes debe sobrevivir cuando el usuario cambia de pantalla. Como en cualquier aplicación JavaFX, los manejadores de eventos (setOnAction) permanecen en la vista, pero solo traducen el evento a una llamada al controlador.

23.6 Persistencia y consulta de datos en MySQL

La clase Db abre la conexión con JDBC (MySQL Connector/J) leyendo la URL, el usuario y la clave de db.properties o de las variables de entorno CLICKOBS_DB_URL, CLICKOBS_DB_USER y CLICKOBS_DB_PASSWORD. Las credenciales nunca están en el código (RNF-03) y db.properties está en .gitignore. Todas las sentencias usan PreparedStatement con parámetros, lo que evita la inyección de SQL. El trabajo de base de datos corre en un hilo aparte (clase Async) para que la ventana no se congele.

Operación	Método del DAO	Sentencia	Pantalla
Autenticar al usuario	UsuarioDao.buscarPorNombre	SELECT con JOIN a rol_permiso	Login
Listar herramientas	HerramientaDao.listar	SELECT sobre herramienta	Consumo, Alertas

Operación	Método del DAO	Sentencia	Pantalla
Comparar costos	ConsumoDao.comparar	SELECT con JOIN de cuatro tablas	Comparar costos
Costos agregados	ConsumoDao.costoPorHerramienta, costoPorTribu, costosPorMes	SUM y GROUP BY	Dashboard
Detectar alertas	ConsumoDao.alertasUmbral	HAVING SUM(monto_usd) mayor al umbral	Alertas, Dashboard
Guardar el umbral	UmbralDao.guardar	UPDATE; si no existe, INSERT	Alertas
Cambiar el estado del conector	ConectorDao.actualizarEstado	UPDATE sobre conector	Administración
Registrar la recomendación	RecomendacionDao.insertar	INSERT en recomendacion_ia	Recomendaciones IA

La Figura 40 muestra el Dashboard con todos los resultados que salen de MySQL: las tarjetas con los indicadores, el costo por herramienta, el costo por tribu, el costo mensual del año y el ranking de servicios. La Figura 41 muestra la pantalla de Administración, que actualiza el estado de un conector con UPDATE y recarga la tabla desde la base.

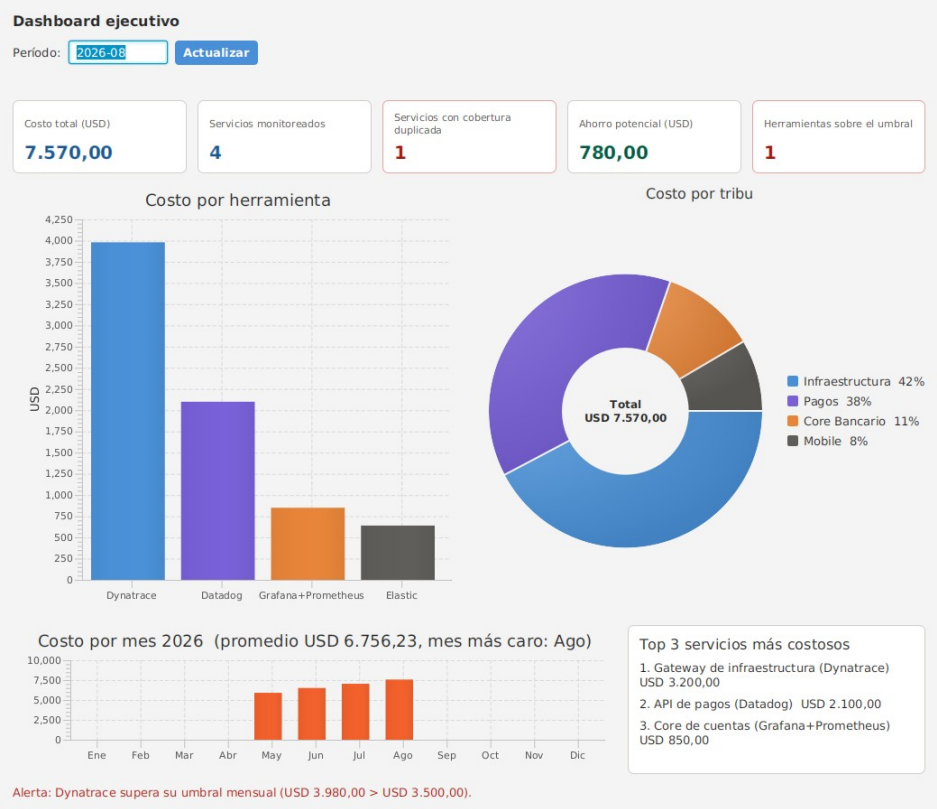


Figura 40. Dashboard ejecutivo (período 2026-08). El gráfico «Costo por mes» incluye el historial de demostración de mayo a julio (db/Historico_demo_CLICKOBS.sql)

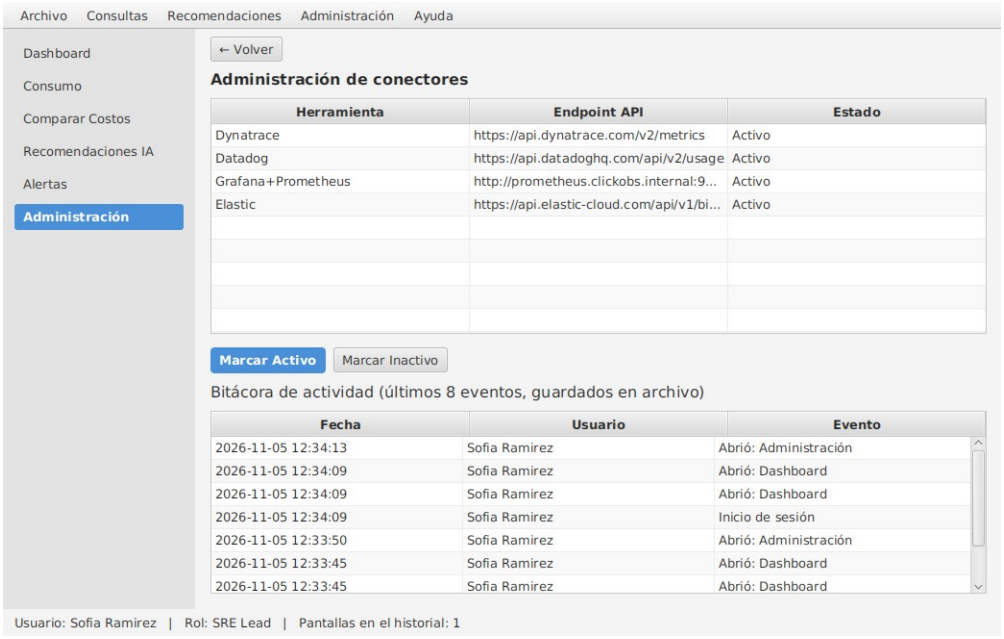


Figura 41. Administración: conectores leídos de MySQL y bitácora de actividad leída de un archivo

23.7 Excepciones en la interacción con MySQL

Una consulta puede fallar por motivos ajenos al programa: clave incorrecta, servidor apagado, tabla inexistente o un dato que viola una restricción. El driver informa esos fallos con SQLException, cuyo texto es técnico y obliga a las capas superiores a conocer java.sql. La decisión fue crear una excepción verificada propia, **PersistenciaException**, y traducir en DaoBase cada SQLException a esa clase, conservando la causa original. Así la interfaz muestra un mensaje claro y el programa sigue funcionando.

Código MySQL	Situación	Mensaje que recibe el usuario
1045	Usuario o clave incorrectos	«... usuario o clave de la base de datos incorrectos.»
1049	La base de datos no existe	«... la base de datos no existe.»
1146	Falta una tabla	«... falta una tabla (ejecute CreacionTablas_CLICKOBS.sql).»
1062	Valor duplicado en una clave única	«... ya existe un registro con ese valor.»
1451 y 1452	Violación de integridad referencial	«... el registro está relacionado con otros datos.»
3819	Una restricción CHECK no se cumple	«... el valor no cumple una restricción de la base de datos.»
Estado SQL 08xxx	No hay conexión con el servidor	«... no hay conexión con la base de datos (revise que el servidor MySQL esté encendido).»
Otro	Cualquier otro error del driver	Se muestra el texto del driver, con el prefijo «No se pudo ...»



Figura 42. Administración con el servidor de base de datos apagado: el mensaje es claro y la aplicación no se cae

La clase de cada excepción responde a una pregunta distinta, y por eso quién la maneja también es distinto:

Excepción	Tipo	Qué representa y quién la maneja
ValidacionException (MontoInvalido, PeriodoInvalido)	Verificada	El usuario escribió un dato inválido. La captura la pantalla y muestra el mensaje
PersistenciaException	Verificada	Falló la base de datos. Los DAO la lanzan, la gestión la propaga y la pantalla la muestra (Async.mensaje)
EstructuraVaciaException	No verificada	Se intentó sacar un elemento de una pila o cola vacía: es un error del programador, no del usuario
IllegalArgumentException / IllegalStateException	No verificada	Constructor con datos imposibles o configuración de base de datos ausente

Los recursos JDBC (Connection, PreparedStatement y ResultSet) se declaran con try-with-resources, de modo que se cierran siempre, incluso cuando la consulta falla.

Fragmento 4. Traducción de errores de MySQL (PersistenciaException.java, líneas 26 a 45)

```
    public static PersistenciaException de(String operacion, SQLException e) {
        int codigo = e.getErrorCode();
        String estado = e.getSQLState() == null ? "" : e.getSQLState();
        String detalle;
        switch (codigo) {
            case 1045: detalle = "usuario o clave de la base de datos incorrectos";
break;
            case 1049: detalle = "la base de datos no existe"; break;
            case 1146: detalle = "falta una tabla (ejecute
CreacionTablas_CLICKOBS.sql)"; break;
            case 1062: detalle = "ya existe un registro con ese valor"; break;
            case 1451:
            case 1452: detalle = "el registro esta relacionado con otros datos";
break;
            case 3819: detalle = "el valor no cumple una restriccion de la base de
datos"; break;
            default:
                if (estado.startsWith("08")) {
                    detalle = "no hay conexion con la base de datos (revise que el
servidor MySQL este encendido)";
                } else {
                    detalle = e.getMessage() == null ? "error de base de datos" :
e.getMessage();
                }
        }
        return new PersistenciaException("No se pudo " + operacion + ": " + detalle +
".", codigo, e);
```

23.8 Clases abstractas e interfaces

Elemento	Tipo	Para qué se usa	Quién lo reutiliza
Herramienta	Clase abstracta	Atributos comunes y los métodos abstractos calcularCostoPeriodo, getTipo y describirCosto	HerramientaSaaS y HerramientaSelfHosted
DaoBase	Clase abstracta	Método plantilla de JDBC: abrir, ejecutar, cerrar y traducir errores	Los seis DAO
Dao<T>	Interfaz	Contrato de los DAO que listan objetos del dominio	HerramientaDao, UmbralDao, ConectorDao, RecomendacionDao
ClienteIA	Interfaz	Contrato de un cliente de IA intercambiable	IASimulada hoy; OpenRouter en la versión final
MapeadorFila<T> y Operacion<R>	Interfaces funcionales	Pasan a DaoBase el comportamiento variable con expresiones lambda	Todos los DAO

Criterio de elección. Se usa una clase abstracta cuando hay estado o código compartido: Herramienta guarda id, nombre y proveedor, y DaoBase contiene el método plantilla. Se usa una interfaz cuando solo se necesita un contrato, sin código común: Dao y ClienteIA. Un DAO puede heredar de una clase y, a la vez, implementar una interfaz, algo que Java permite y que no sería posible con dos clases.

UsuarioDao y ConsumoDao heredan de DaoBase pero no implementan Dao<T>: su operación principal no es «listar todo» (buscan por nombre o agregan por período) y forzar ese método habría sido artificial.

23.9 Arreglos y ArrayList de forma complementaria

El criterio es simple: **arreglo cuando el tamaño es fijo y conocido, ArrayList cuando la cantidad varía.** La base de datos devuelve una cantidad variable de filas, por eso llegan en un ArrayList; los resultados con forma fija (los 12 meses de un año, un Top 3, los tres campos de un evento) se guardan en arreglos.

Dónde	Estructura	Por qué
DaoBase.consultar	ArrayList	La cantidad de filas que devuelve una consulta no se conoce de antemano
ResumenAnual.costoPorMes (BigDecimal[12])	Arreglo	Un año siempre tiene 12 meses; se accede por índice (mes menos 1). El getter devuelve una copia para no romper el encapsulamiento
ResumenAnual.MESES (String[12])	Arreglo	Etiquetas fijas de los meses para el gráfico
ConsumoDao.costosPorMes	Los dos	Lee las filas en un ArrayList y las vuelca en el arreglo de 12 posiciones; los meses sin datos quedan en cero (Arrays.fill)
GestorRecomendaciones.topServicios (ComparacionFila[])	Arreglo	Ranking de tamaño fijo (3); si hay menos datos, el arreglo es más corto
Bitacora.ultimos	ArrayList de arreglos	La cantidad de eventos varía; cada evento tiene siempre tres campos {fecha, usuario, evento}
Algoritmos.ordenarPorSeleccion	ArrayList	Devuelve una copia ordenada de la lista; el ordenamiento escrito a mano es el de la sección 22.4

Fragmento 5. De ArrayList a arreglo de 12 posiciones (ConsumoDao.java, líneas 147 a 159)

```
    public BigDecimal[] costosPorMes(String anio) throws PersistenciaException {
        String sql = "SELECT periodo, SUM(monto_usd) FROM consumo WHERE periodo LIKE ?
GROUP BY periodo ORDER BY periodo";
        List<Object[]> filas = consultar("consultar el costo por mes", sql,
                rs -> new Object[] {rs.getString(1), rs.getBigDecimal(2)}, anio + "-
%");
        BigDecimal[] porMes = new BigDecimal[12];
        Arrays.fill(porMes, BigDecimal.ZERO);
        for (Object[] f : filas) {
            int mes = Integer.parseInt(((String) f[0]).substring(5, 7));
            if (mes >= 1 && mes <= 12) {
                porMes[mes - 1] = (BigDecimal) f[1];
            }
        }
        return porMes;
    }
```

En el Dashboard (Figura 40) el arreglo de 12 posiciones alimenta el gráfico «Costo por mes», y el arreglo Top 3 alimenta el ranking de servicios más costosos. ResumenAnual calcula el total, el promedio de los meses con datos y el mes más caro con bucles for sobre el arreglo.

23.10 Archivos para guardar y recuperar información

El uso de archivos es opcional en la consigna, pero el proyecto lo necesita en tres lugares, con una función distinta cada uno:

Archivo	Operación	Detalle
clickobs-bitacora.csv	Escribir (agregar al final) y leer	Bitácora de actividad: inicio y cierre de sesión, pantallas abiertas y accesos denegados. Se recuperan los últimos 8 eventos en la pantalla Administración
Reporte de comparación (.csv)	Escribir	Exportación de la comparación de costos (RF-17): UTF-8 con BOM y separador «;» para que Excel en configuración argentina lo abra bien
db.properties	Leer	Configuración de la conexión; las variables de entorno la sobrescriben. No se sube al repositorio

Cada línea de la bitácora tiene el formato fecha;usuario;evento, por ejemplo: **2026-11-05 12:33:22;Sofia Ramirez;Abrió: Administración**. Los archivos se manejan con la API java.nio.file y try-with-resources. Si el archivo no se puede escribir, registrar() devuelve false y el programa sigue funcionando: una falla de la bitácora no debe tumbar la aplicación. Si no se puede leer, el error llega a la pantalla como mensaje.

Fragmento 6. Bitacora.registrar (Bitacora.java, líneas 35 a 43)

```
    public boolean registrar(String usuario, String evento) {
        String linea = LocalDateTime.now().format(FORMATO) + ";" + limpiar(usuario) +
";" + limpiar(evento)
                    + System.lineSeparator();
        try {
            Files.writeString(archivo, linea, StandardCharsets.UTF_8,
StandardOpenOption.CREATE, StandardOpenOption.APPEND);
            return true;
        } catch (IOException e) {
            System.err.println("Bitacora: no se pudo escribir en " + archivo + " (" +
e.getMessage() + ")");
            return false;
        }
    }
```

23.11 Pruebas de la iteración

El archivo pruebas/PruebasClickobs.java ahora ejecuta **127 casos** (los 77 del TP3 más 50 nuevos), sin JUnit, y cada caso imprime OK o FALLO. Los casos con base de datos dejan los datos como los encontraron.

Grupo nuevo del TP4	Casos	Qué verifica
ResumenAnual (arreglo de 12 posiciones)	8	Total, promedio, mes más caro, meses sin datos, arreglo de tamaño incorrecto y copia defensiva
Bitácora (archivo)	7	Crear, agregar, leer los últimos n en orden, arreglos de tres campos, separador dentro del texto y ruta imposible
PersistenciaException	4	Traducción de los códigos 1045, 1146 y del estado 08S01; conserva el código y la causa
Patrón DAO (DaoBase y Dao<T>)	4	consultar, actualizar, tabla inexistente que lanza PersistenciaException y un DAO que cumple la interfaz
Patrón MVC (controladores)	20	Regla de dependencias entre ui, controller y el modelo, y el funcionamiento de los ocho controladores contra la base

Grupo nuevo del TP4	Casos	Qué verifica
Ranking y evolución mensual con la base	7	Top 3 ordenado, arreglo vacío sin datos, arreglo más corto, costosPorMes y datos del dashboard

Fragmento 7. Salida de la última corrida (extracto)

```
-- TP4: arreglos, archivos, excepciones de persistencia y patron DAO
OK     ResumenAnual: mes mas caro = Mar (indice 2)
OK     bitacora: ultimos(2) devuelve 2 eventos, el mas reciente primero
OK     traduccion 1045 -> clave incorrecta
OK     DaoBase.consultar: tabla inexistente -> PersistenciaException (no SQLException)
OK     topServicios esta ordenado de mayor a menor costo (3200, 2100, 850)
OK     costosPorMes: arreglo de 12; agosto (indice 7) = 7570; enero = 0
-- TP4: patron MVC (controladores entre las vistas y el modelo)
OK     MVC: ninguna vista importa dao, service ni db (solo habla con su controlador)
OK     MVC: los controladores no importan JavaFX (no dependen de la vista)
OK     AlertasController: guardar el mismo umbral (UPDATE) deja los datos igual
OK     DashboardController: costo total de 2026-08 = 7570

Resultado: 127 OK, 0 FALLO
```

Defectos encontrados y corregidos durante la iteración. (1) Al agregar el gráfico mensual y el ranking, el Dashboard no entraba en la ventana: se envolvió en un panel con barra de desplazamiento. (2) Los mensajes de error de la base, ahora más largos, aplastaban los botones de Administración: el mensaje pasó a su propia línea con ajuste de texto. Ambos se detectaron al ejecutar la pantalla completa y no con las pruebas automáticas, lo que confirma que las pruebas de interfaz siguen siendo manuales (sección 22.8).

23.12 Decisiones, límites y trabajo futuro

- **Sin ORM ni framework:** no se usó Hibernate/JPA ni Spring. El DAO escrito a mano permite mostrar y justificar cada paso de JDBC, que es el contenido de la materia.
- **Una conexión por operación, sin pool:** es adecuado para un cliente de escritorio con pocos usuarios. La versión final agregaría un pool de conexiones.
- **Claves con MAX + 1:** las tablas umbral_alerta y recomendacion_ia no usan AUTO_INCREMENT; la mejora está declarada en el código y se aplicaría en la Construcción final.
- **Datos de demostración opcionales:** el script db/Historico_demo_CLICKOBS.sql agrega consumo de mayo a julio de 2026 solo para que el gráfico mensual muestre más de un mes. No modifica agosto ni las pruebas.
- **Coherencia con el diseño:** el diagrama de clases de diseño de la sección 15 (Figura 10) se corrigió para que coincida con el código (ver 23.2); las clases nuevas del TP4 (Dao, DaoBase, PersistenciaException, ResumenAnual, Bitacora y los controladores) se muestran en las Figuras 38 y 39.
- **Limitaciones heredadas del TP3:** contraseña pendiente de verificar, IA simulada y ausencia de backend intermedio (sección 22.1). Se abordan en la Construcción final y en la Transición.

23.13 Repositorio y ejecución

Repositorio: **https://github.com/jorgejcorrea/clickobs**. El video de presentación de 3 minutos está enlazado en el README del repositorio. Carpetas: src/ (código), db/ (scripts SQL), airflow/ (ingesta), pruebas/ (127 casos) y docs/ (este informe y las figuras).

Para ejecutarlo se necesita JDK 21 y Maven. Se crea la base con db/CreacionTablas_CLICKOBS.sql e Insert_CLICKOBS.sql (y, de forma opcional, Endurecimiento_CLICKOBS.sql e Historico_demo_CLICKOBS.sql), se copia db.properties.example como db.properties completando usuario y clave, y se ejecuta «mvn javafx:run». Las pruebas se corren con los comandos de pruebas/README.md. La corrida de referencia de este informe se hizo contra MySQL 8.0.46 (servidor en contenedor Docker) y dio «Resultado: 127 OK, 0 FALLO»; la misma batería también pasó sobre MariaDB 10.11, lo que confirma que el código no depende de extensiones propias de un motor.

23.14 Arquitectura del prototipo y arquitectura objetivo

La Figura 43 muestra la arquitectura del prototipo tal como quedó construida y probada en el TP4: la cadena Vista, Controlador, Modelo y MySQL, con las 127 pruebas automáticas. Se complementa con la Figura 5 (sección 8.3), que presenta la arquitectura objetivo: allí cada columna lleva una etiqueta (EN PROTOTIPO, PARCIAL o EVOLUCIÓN) que indica cuánto de ese componente existe hoy.

La diferencia principal es que el backend Spring Boot con API REST, la llamada real a OpenRouter, la exportación a PDF y el formato Parquet quedan para la Construcción final. En el prototipo esa lógica vive en la capa de controladores y servicios dentro de la propia aplicación, lo que permite mostrar y probar cada paso sin depender de un framework.

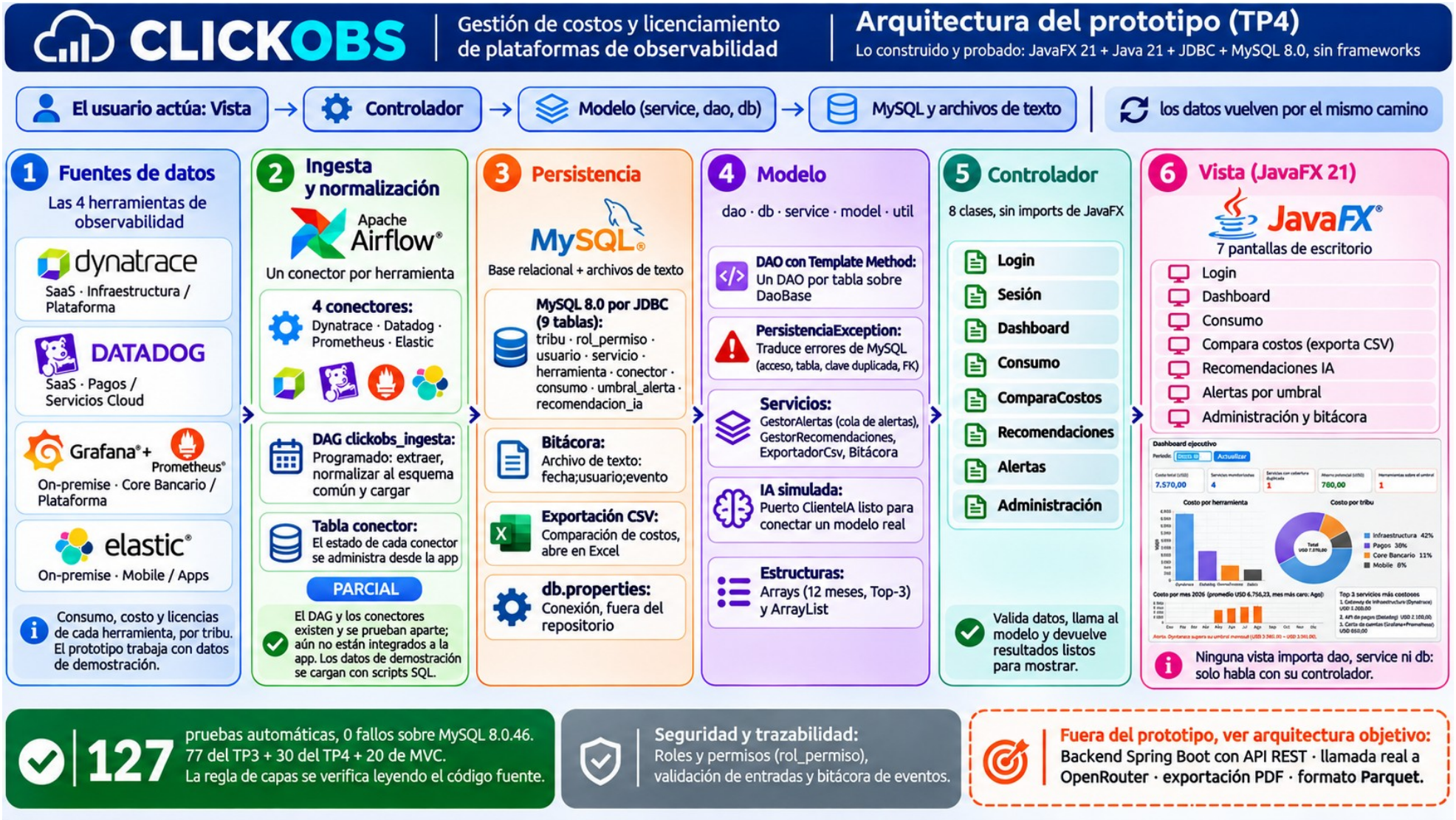


Figura 43. Arquitectura del prototipo (TP4): lo construido y probado, sin frameworks. La arquitectura objetivo se muestra en la Figura 5 (sección 8.3).

23.15 Conclusión

La versión final integradora completa el circuito que pide la consigna: la interfaz consulta y actualiza MySQL a través de un patrón DAO coherente con la arquitectura en capas, los errores de la base se traducen a mensajes claros, las clases abstractas y las interfaces concentran lo reutilizable, los arreglos y el ArrayList se usan cada uno donde corresponde y los archivos guardan y recuperan información relevante. Las correcciones de las entregas anteriores están integradas y verificables, y el prototipo sigue siendo un modelo operacional cuyos módulos se integran en la versión final del sistema.

Referencias

Alur, D., Crupi, J. y Malks, D. (2003). *Core J2EE patterns: Best practices and design strategies* (2.ª ed.). Prentice Hall.

Booch, G. (1996). *Análisis y diseño orientado a objetos, con aplicaciones*. Addison Wesley Iberoamericana S. A.

FinOps Foundation. (2026). *What is FinOps?* FinOps Foundation. Recuperado el 8 de noviembre de 2026, de https://www.finops.org/introduction/what-is-finops/

Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Kendall, K. y Kendall, J. (2011). *Análisis y diseño de sistemas*. Pearson Education.

Markanastasakis, V. (2026, 19 de marzo). *FinOps Framework 2026: Executive strategy, technology categories, and converging disciplines*. FinOps Foundation. https://www.finops.org/insights/2026-finops-framework/

MySQL. (s. f.). *MySQL 8.0 reference manual*. Oracle Corporation. https://dev.mysql.com/doc/refman/8.0/en/

Oracle. (s. f.). *Java SE 21 API specification*. https://docs.oracle.com/en/java/javase/21/docs/api/

Oracle. (s. f.). *JDBC basics (The Java tutorials)*. https://docs.oracle.com/javase/tutorial/jdbc/basics/

Rumbaugh, J., Jacobson, I. y Booch, G. (2006). *El lenguaje unificado de modelado* (2.ª ed.). Pearson-Addison Wesley.

Anexo A. Paso a paso: publicar el proyecto en GitHub

Se detalla el procedimiento para publicar el código del prototipo, los scripts SQL, los diagramas y las infografías en el repositorio personal del autor (github.com/jorgejcorrea).

Paso 1: Crear el repositorio

Ingresar a github.com con la cuenta jorgejcorrea y crear (o reutilizar) el repositorio clickobs, en modo Public para que pueda consultarse sin permisos especiales, con la opción «Add a README file» tildada.

Paso 2: Clonar el repositorio en la máquina local

```
git clone https://github.com/jorgejcorrea/clickobs.git
```

Paso 3: Organizar la carpeta del proyecto

- /src: código fuente del prototipo JavaFX (paquetes Java y archivos .fxml).
- /db: los cuatro scripts SQL (CreacionTablas_CLICKOBS.sql, Insert_CLICKOBS.sql, Consulta_CLICKOBS.sql y Borrado_CLICKOBS.sql).
- /docs: el documento de esta entrega (.docx y .pdf).
- /docs/img: imágenes de los diagramas, infografías (Arquitectura_Clickobs_Infografia_Funcional.png) y capturas de evidencia.
- README.md: descripción breve del proyecto, con la infografía incrustada mediante ![Arquitectura](docs/img/Arquitectura_Clickobs_Infografia_Funcional.png).

Paso 4: Subir los archivos (commit y push)

```
cd clickobs
git add .
git commit -m "TP4: versión final integradora, scripts SQL, pruebas y documento"
git push origin main
```

Paso 5: Autenticarse si GitHub lo solicita

GitHub ya no acepta la contraseña de la cuenta para el push por HTTPS. Si la solicita, usar un Personal Access Token: perfil de GitHub → Settings → Developer settings → Personal access tokens → Generate new token (classic), con el permiso «repo». El token se pega como contraseña; el usuario sigue siendo jorgejcorrea.

Paso 6: Verificar y copiar el link final

Refrescar la página del repositorio y comprobar que se vean las carpetas y el README con la infografía. El link https://github.com/jorgejcorrea/clickobs es el que figura en el Resumen Ejecutivo y en la sección 21.

Paso 7: Actualizaciones posteriores

Cada vez que se agregue código nuevo (por ejemplo, al avanzar en las fases de Construcción y Transición), repetir el Paso 4 desde la misma carpeta local.